



SAPIENZA
UNIVERSITÀ DI ROMA

Autonomous Networking

UNIVERSITY OF ROME "LA SAPIENZA"

FACULTY OF INFORMATION ENGINEERING, COMPUTER
SCIENCE AND STATISTICS

MASTER'S DEGREE IN COMPUTER SCIENCE

Author
LORENZO SABATINO

Based on the lectures of
PROF. GAIA MASELLI

December 5, 2025



Preface

Hello everyone!

My name is Lorenzo Sabatino and I am a student of the Computer Science course at La Sapienza University of Rome. During my academic career, I have faced numerous exams by transcribing the notes taken in class and transforming them into real study books.

I wanted to share this material with you because I believe it can be a valid help for all Computer Science students, both those who are beginners and those who are already advanced in their studies. I have written and formatted these texts with care, using the $\text{\LaTeX}$ markup language to ensure clarity and readability. I have also enriched the content with diagrams and schemes and I have integrated ideas taken from online lessons and textbooks.

However, it is important to underline that these books do not replace the classroom lessons and are not free of errors. My intention was to provide additional support for studying, but it is essential to participate in the lessons and delve into the concepts with the guidance of the teachers. Also, please note that due to the ever-evolving nature of computer science, some information may become outdated over time.

Nevertheless, I decided to share these books because I believe they can be of help to many students and I hope you will find them useful in your academic journey.

If you wish to contact me with any questions, comments or corrections, you can write to me at scrittinformatica@gmail.com. I will be happy to answer your requests.

Thank you for choosing to read these books and good luck with your studies!

Contents

Preface	1
1 Introduction	5
1.1 Why autonomous?	5
1.2 Which technologies?	6
1.3 Autonomous means intelligent	6
2 RFID Systems	9
2.1 RFID communication	9
2.2 MAC protocols	10
2.2.1 Binary Splitting	11
2.2.2 Query Tree	12
2.2.3 Performance of tree protocols	13
2.2.4 Framed Slotted Aloha	13
2.2.5 Tree Slotted Aloha	15
2.2.6 Estimating tag population	16
2.2.7 Binary Splitting Tree Slotted Aloha	17
3 Sensor Networks	19
3.1 Wireless Sensor Networks	19
3.1.1 Applications of Wireless Sensor Networks	20
3.1.2 Characteristics of WSN	20
3.1.3 Wireless signal	22
3.2 Medium Access Control (MAC)	23
3.2.1 MAC protocols	25
3.2.2 CSMA/CA	25
3.2.3 S-MAC	29
3.3 Routing	33
3.3.1 Destination Sequence Distance Vector (DSDV)	34
3.3.2 Reactive Routing Protocols	35
3.3.3 Geographic routing	38
4 Networks of Drones	41
4.1 Unmanned Aerial Vehicle (UAV)	41
4.2 Dronet	42
4.2.1 How do we provide communication in emergency situations where there is no network infrastructure?	42
4.2.2 How to optimize task assignment and trajectory planning for UAVs operating in real-world scenarios?	43
4.2.3 How to perform continuous data offloading when deployed in large and harsh areas?	43
4.2.4 How to perform periodic data offloading when UAVs are deployed in large and harsh areas?	48

5	IoT & Backscattering	51
5.1	What is the Internet of Things (IoT)?	51
5.2	Battery-free smart objects	52
5.3	Infrastructure-based wireless networks	55
5.4	Infrastructure-less networks and MANETs	56
6	Introduction to Reinforcement Learning	59
6.1	Applications of Reinforcement Learning	60
6.2	Rewards	61
6.3	The exploration–exploitation dilemma	61
6.4	General RL framework	62
7	Bandit Algorithm	65
7.1	K-armed bandit	65
7.2	Estimating action values	66
7.3	How to pick an action	66
7.3.1	Random selection	66
7.3.2	Greedy selection	66
7.3.3	ϵ -greedy selection	66
7.4	How to estimate action values	68
7.4.1	Incremental trick	68
7.4.2	Stationary vs nonstationary problems	69
7.4.3	Optimistic initial values	70
8	Performance of methods for action selection	71
8.1	The 10-armed testbed	71
8.2	Optimism in the Face of Uncertainty	73
9	Formalization of sequential decision making	77
9.1	Introduction to Markov Decision Process	77
9.2	Markov Reward Process	79
9.3	Markov Decision Process	81
9.3.1	Policy	82
9.3.2	Value function	82
9.4	Optimal policy	83
9.4.1	Bellman Optimality Equation	83
9.4.2	Solving the Bellman Optimality Equation	84
10	Performance Evaluation	87
10.1	Practical steps	87
10.2	Three rules of validation	88
10.3	Selecting performance metrics	89
10.3.1	Speed, reliability, availability	89
10.4	Summarizing measured data	90
10.4.1	Summarizing data by a single number	90
10.4.2	Summarizing variability	91
10.4.3	What to report	92

Chapter 1

Introduction

Autonomous networking is an advanced course built on existing knowledge. You should already have taken a standard computer networks course and be comfortable with the TCP/IP protocol stack, basic MAC layer mechanisms and routing protocols. Reasonable familiarity with probability and probabilistic thinking is also required, since many techniques we will discuss use stochastic models.

The course explores a variety of wireless, networked communication systems. We will study:

- sensor networks;
- Internet of Things;
- backscattering networks such as RFID;
- unmanned networks (dronets).

A portion of the course focuses on reinforcement learning, because this is currently one of the most practical and compelling ways to design autonomous policies for networking tasks. We will first learn how to formalize reinforcement learning problems in a networking context. Then we will study specific techniques that apply broadly to networking: from bandit algorithms for simple decision problems to tabular and function-approximation variants of Q-learning for stateful environments.

1.1 Why autonomous?

Autonomy in networks carries multiple connotations.

- *Self-Governance*: the system makes independent decisions based on predefined rules and real-time data rather than waiting for human intervention.
- *Self-Sufficiency*: an autonomous network should be able to operate as an independent organism, managing its own resources and orchestrating tasks across nodes.
- *Embedded intelligence*: autonomous systems integrate advanced algorithms, including machine learning, so they can interpret diverse environmental inputs, learn from past experiences and adapt their behavior as conditions change.
- *Proactive Response*: an autonomous network continuously monitors its environment, anticipates disruptions and takes action to preserve or restore performance and reliability.

Consider a smart home with a lot of connected devices sharing a common wireless channel. Normally, the network carries routine traffic: temperature sensors, smart appliances and background media streaming. Imagine that someone starts a video stream that occupies most of the available bandwidth. Now imagine the smoke detector in the kitchen senses smoke. Devices monitoring the kitchen must be able to send their emergency data with minimal delay. An autonomous network should detect that this traffic is critical and change scheduling, prioritization or access policies so that kitchen sensors obtain channel access quickly and reliably. This is the kind of reactive and proactive behavior autonomous networking aims to provide.

1.2 Which technologies?

Most of our focus will be on wireless technologies that commonly require autonomy to satisfy strict constraints on latency, energy and reliability. These include:

- *Backscattering networks used in RFID systems:* Radio Frequency Identification, or RFID, is one of the simplest examples of a backscattering communication system. In RFID, very low-power tags communicate by reflecting a carrier transmitted by a reader (see Figure 1.1).



Figure 1.1: Radio Frequency Identification - RFID.

- *Sensor and IoT:* Wireless sensor networks consist of a monitored area where spatially distributed sensors collect environmental data and forward it — often through multiple hops — to a central sink. The sink then relays information to servers or the Internet (see Figure 1.2).

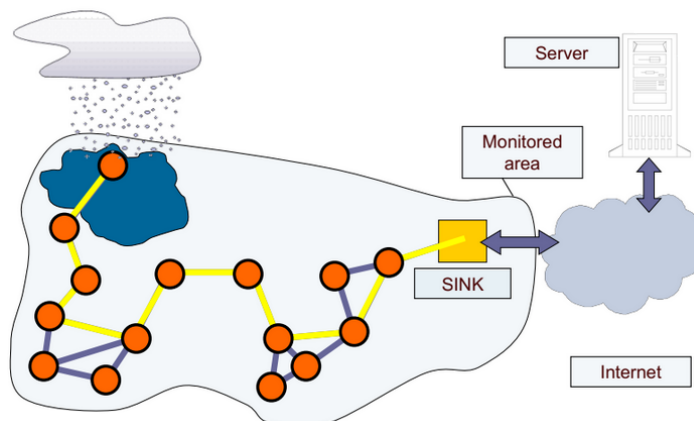


Figure 1.2: Sensor networks.

When we speak about IoT networks we are referring to heterogeneous collections of smart devices, sensors, RFID tags, Wi-Fi-enabled gadgets and gateways that form intelligent environments. Smart buildings and smart homes are canonical examples (see Figure 1.3).

- *Unmanned networks (dronet):* Drones are used for many applications: to capture aerial images, to inspect structures, to experiment with parcel delivery and to collect data in places that would be hard to reach otherwise. Many of those applications still rely on a human pilot: someone at a remote control or a command center who makes most of the decisions. Yet there is a complementary vision in which drones operate as fully unmanned systems — squads of aerial vehicles that fly without a person physically present or constantly steering them.

1.3 Autonomous means intelligent

A common question is: how do we make a squad of drones or an IoT network autonomous? A short answer is: by giving the system learning and decision-making capabilities. One of the most natural frameworks to introduce that kind of intelligence is reinforcement learning.

Reinforcement learning, or RL, addresses a simple but deep question: how can an intelligent agent learn to make a good sequence of decisions? That single sentence hides a lot.

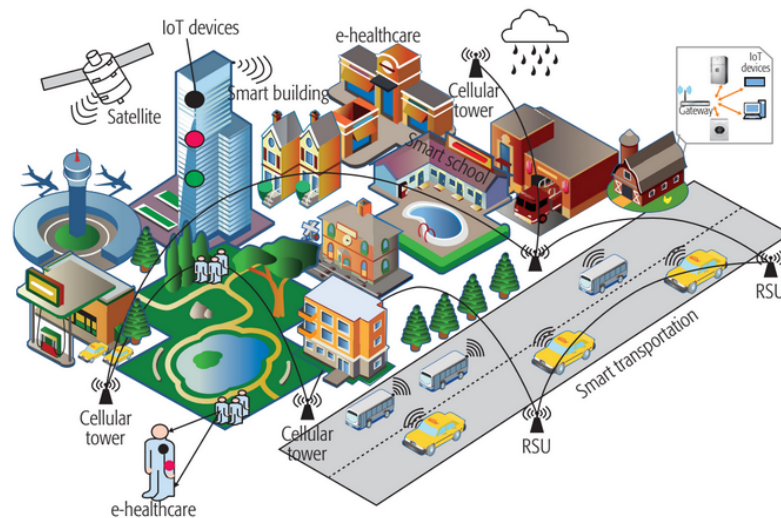


Figure 1.3: Smart city.

- First, we are not concerned with one-off choices; instead, the agent must consider sequences where current actions influence the future.
- Second, by saying a good sequence of decisions we implicitly mean there is some measure of utility or reward the agent seeks to maximize.
- Third, the agent typically starts without a model of how its actions will affect the world; it has to learn that relationship by interacting with its environment.

Put differently, in RL the agent performs actions, observes consequences and receives feedback in the form of rewards. Over time, the agent should learn which actions tend to produce better long-term returns.

There are a few central themes you will hear repeatedly in RL: optimization, delayed consequences, exploration and generalization. Each of these themes maps directly to networking problems.

- RL is an optimization problem: we want to find a policy — a way of making decisions — that yields the best outcome possible. In networks that objective could be throughput, latency, energy efficiency, fairness or some combination of metrics.
- A complicating factor is the temporal dimension: decisions taken now can ripple forward and affect rewards far in the future. Suppose an agent chooses to prioritize a low-latency flow now; that might make a future congestion episode worse. Planning and learning under such delayed consequences require us to identify correctly which actions were responsible for later successes or failures.
- Exploration is another practical challenge: agents learn about the world by trying things. In many real systems the learning data is censored — you only observe the outcome of decisions you actually took; you don't know what would have happened had you taken alternatives.
- Generalization is what allows an agent to cope with states it has never explicitly seen. A policy is, formally, a mapping from past experience to actions, so the agent must extract structure from past experience in order to act sensibly in new situations.

Equally important is robustness and risk sensitivity. Robustness refers to an agent's ability to perform well under different, possibly unseen, conditions. Risk sensitivity captures how an agent handles uncertainty — whether it prefers conservative policies or aggressive policies.

By the end of the course you will be able to take a networking problem, such as a MAC or routing challenge, and decide whether it is appropriate to formulate it as an RL problem. If the answer is yes, you will be able to define the problem formally: specify the state space, action space, environment dynamics and reward model. You should also be able to select an algorithm from those we study and justify why that algorithm fits the problem constraints and objectives.

Chapter 2

RFID Systems

In this chapter we explore the anatomy of an RFID system: its components, typical characteristics and common applications. We then dive into the Medium Access Control (MAC) problem that arises when many passive tags respond to a reader, review the families of solutions that were proposed.

Radio-Frequency Identification (RFID) is a technology that uses radio waves to automatically identify and track tags attached to objects. The system relies on electronic devices called RFID tags, which store data and transmit it wirelessly to a reader (or interrogator).

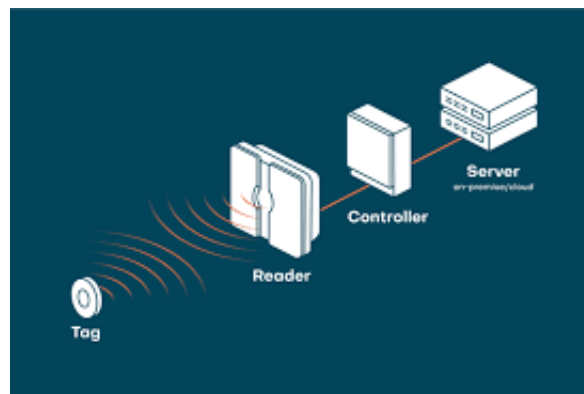


Figure 2.1: RFID system.

- *Tags* are small radio frequency labels that store a unique identifier — typically 96 bits in many standards, although other sizes up to 256 bits are possible. They consist of an antenna and a microchip and are attached to the objects we want to identify.
- *Readers* are more powerful devices. They transmit a continuous wave or carrier and interrogate tags in their coverage area; they also receive the weak signals backscattered by tags when they reply.
- The *server* is the backend that collects tag IDs, processes them according to application logic and stores the inventory data.

RFID systems are used wherever automated identification saves time, reduces errors or enables novel services. You will find them in inventory and logistics, access control, object tracking, library systems, airport baggage handling, assisted living and smart home applications, etc. These systems need to read unique IDs for many objects quickly and often without human intervention.

2.1 RFID communication

Communication in RFID systems is wireless, between reader and tag. There are two directions of interest: reader-to-tag and tag-to-reader.

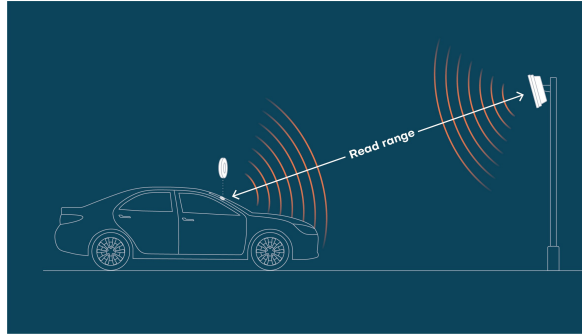


Figure 2.2: RFID application for reading ranges for automatic vehicle identification.

- The reader-to-tag link typically carries commands and power.
- Tag-to-reader is the way tags report their ID.

Tag-to-tag communication is not allowed because: many tags are passive (have no battery) and their replies are transmitted by *backscattering*¹.

A defining feature of many RFID tags is that they are battery-free. These passive tags harvest power from the reader's continuous high-power signal. To communicate, the tag does not transmit a fresh radio signal; instead it modifies how it reflects the incoming carrier. The reader senses tiny changes in the reflected wave and decodes the tag's response.

Because the tag does not generate its own carrier and has very small transmit power, the link is low-rate compared to typical Wi-Fi connections. Tag-to-reader data rates are often on the order of tens of kilobits per second.

The RFID radio channel has several properties that shape protocol design. It is wireless and shared among all tags in the reader's range. It is low bitrate and the tags' replies may overlap in time; when two or more tags reply simultaneously they create a collision from the reader's perspective. Crucially, passive tags typically cannot hear each other; they have no carrier-sense capability and no collision detection, so the reader must orchestrate which tags speak and when using a protocol to separate them efficiently.

When the reader initiates a reading session, it broadcasts commands to the tags in its field. Tags that match the command or query conditions will reply by backscattering their ID. The reader listens and attempts to decode the returned waveform.

If a single tag replies, the reader can decode its ID and acknowledge it; that tag can then be removed from the set of tags awaiting identification. If multiple tags reply at the same time, their signals mix and the reader is unable to decode individual IDs — we call this a collision. The reader detects a collision and triggers a collision-resolution mechanism, which is the role of the MAC protocol.

2.2 MAC protocols

Several characteristics make the MAC problem unique for RFID. First, there may be a very large number of tags to identify, far more than in typical wireless networks. Second, passive tags cannot initiate communication spontaneously; they wait for the reader's prompt. Third, tags respond via a backscattered signal that collides when simultaneous. Fourth, tags cannot listen to the channel — there is no carrier sense — and cannot detect collisions. Put together, these constraints force the design of reader-coordinated MAC protocols that aim to separate tag transmissions so that the reader receives one ID at a time.

Researchers have proposed many MAC protocols tailored to RFID. Broadly, we can group them into *sequential protocols* (tree based, Aloha based, etc.) and *concurrent protocols* that try to exploit collisions rather than avoid them. Here we focus only on sequential methods, specifically Tree based and Aloha based.

¹Backscatter (or backscattering) is the reflection of waves, particles, or signals back to the direction from which they came. Ambient backscatter technology uses existing signals to power and communicate with unpowered sensors.

- *Tree-based* methods aim to interactively split the set of tags into smaller subsets until each subset contains a single tag that can be identified. Binary Splitting is a classic example. Query Tree protocols are a related family that use prefix matching and tree traversal. There are improved variants of Query Tree that refine efficiency under different loads.
- *Aloha-based* methods borrow ideas from slotted Aloha: the reader defines frames and slots and tags pick a slot to reply. Framed Slotted Aloha is a common approach; some variants are standardized in the EPC Gen2 specification. Tree Slotted Aloha and other modified protocols (for example BSTSA) combine tree and slotted strategies to balance performance across different tag population sizes.

2.2.1 Binary Splitting

Binary Splitting (BS) is a recursive tree-based approach to singulating tags. The main idea is simple: whenever multiple tags collide, we split the set of colliding tags into two groups and recursively resolve each subgroup until all tags are identified. Let us walk through how a Binary Splitting session proceeds in practice.

1. Initially every tag participating in the reading has a local counter set to zero.
2. The reader issues a generic query that asks tags with counter equal to zero to reply.
3. If exactly one tag answers, the reader reads its ID and that tag is considered identified; the reader then broadcasts that fact and the process continues for remaining tags.
4. If more than one tag answers, the reader detects a collision. On collision, every replying tag generates a random binary digit — either 0 or 1 — and adds that digit to its local counter. The reader then queries again, asking tags with counter equal to zero to reply. This effectively partitions the original set into those tags whose random bit was zero (they keep counter zero) and those whose random bit was one (they increment their counter to one and stay silent for this query).
5. If the collision still occurs among the replying tags (the zeros), those tags each again flip a random bit and add it to their counters, further refining the partition.
6. Meanwhile, tags that are currently silent because their counter is greater than zero increment their counter each time the reader issues a query that they are not eligible to answer.
7. The process continues, recursively splitting the colliding group into smaller and smaller subgroups until single-tag responses are obtained. If none or one tag replies, all tags decrement their counter by 1.

An alternative way to view Binary Splitting is that tags generate a random binary tree where each collision makes the algorithm explore two children nodes corresponding to the two possible bit outcomes (see Figure 2.3). Tags follow a random path down this tree; when a leaf node contains exactly one tag that tag is identified and removed from the active set.

Example 2.2.1 – Eight tag binary splitting

Let's name our eight tags s_1 through s_8 . Initially every tag has counter $C = 0$. The reader issues a broad query for counter equals zero; all eight reply and we get a collision. Now each of the eight tags picks a random bit. Suppose s_1, s_2 and s_3 pick 0 and keep $C = 0$; s_4, s_5, s_6, s_7 and s_8 pick 1 and set $C = 1$. The next reader query asks only tags with $C = 0$ to reply, so s_1, s_2 and s_3 respond and collide. Because s_1 – s_3 collide, each of them now picks a fresh random bit and adds it to its counter. Suppose s_1 picks 0 (still $C = 0$), s_2 picks 1 (C becomes 1), s_3 picks 0 (C stays 0). The reader queries for $C = 0$ again and hears s_1 and s_3 ; if they collide again we continue splitting; if only one of them replies we identify it. Meanwhile the silent tags (s_4 – s_8 and eventually s_2) wait and increment their counters on each query. This recursive splitting continues until each group contains only one tag, and all IDs are collected.

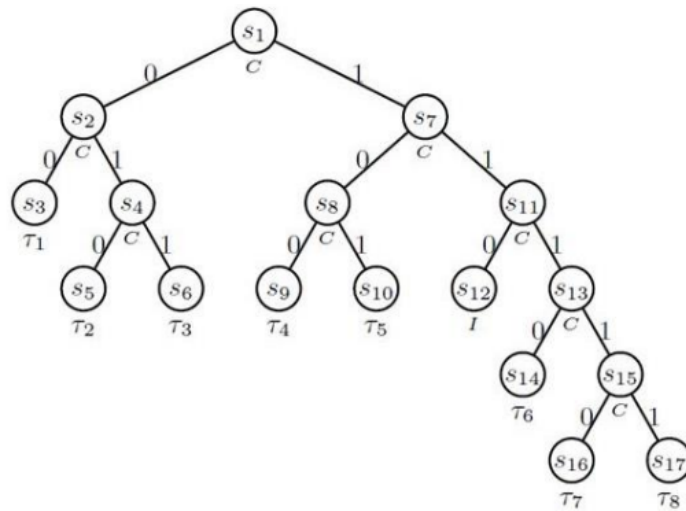


Figure 2.3: Tree representation of Binary Splitting.

2.2.2 Query Tree

Query Tree (QT) protocols use the binary structure of the tag's ID instead of randomized splitting. Each tag has a fixed binary ID. The reader issues queries that are simply binary strings (prefixes). Only tags whose ID begins with the transmitted prefix respond.

1. Start by asking the NULL prefix, meaning "everyone reply."
2. If a collision occurs, the reader extends the prefix by one bit and re-queries. For example the reader will try prefix "0" and prefix "1".
3. If prefix "0" still collides, the reader will try "00" and "01" and so on.

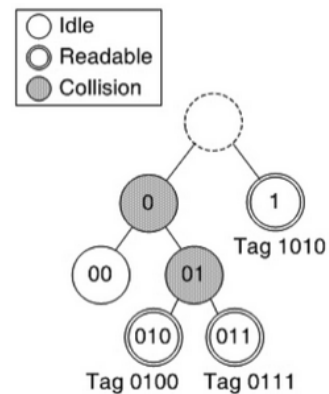
This procedure traverses an implicit binary tree of all possible IDs: leaves contain either a single matching tag (readable) or none (idle), while internal nodes correspond to collisions that must be split further.

This approach is deterministic in the sense that tags don't generate random bits; the split is guided by actual ID bits. It works well when IDs are uniformly distributed because the tree tends to balance itself, splitting the population roughly in half at each bit expansion. In that uniform-ID case the Query Tree and Binary Splitting produce very similar trees and comparable performance, which is why the literature often compares them directly.

Example 2.2.2 – Query tree

Suppose three tags have IDs 0100, 0111 and 1010.

The reader starts with the empty prefix and all three reply, causing a collision. The reader then queries prefix "0" and prefix "1". Prefix "1" matches only 1010, so 1010 replies alone and is readable; the reader reads it. Prefix "0" matches 0100 and 0111; they collide so the reader lengthens the prefix to "00" and "01". The prefix "00" is idle, so the reader tries "01," which causes a collision. Finally we come to the prefix "010" which only matches 0100 (readable) and the prefix "011" which only matches 0111 (readable).



2.2.3 Performance of tree protocols

A simple performance metric is system efficiency, defined as the ratio of successful single-tag responses to the total number of queries the reader sends. Denote by n the number of successful single responses (which equals the number of tags to identify) and by q the total number of queries sent during the identification. Then system efficiency is

$$SE = \frac{n}{q}$$

An ideal protocol would have $SE = 1$, meaning every query yields exactly one readable tag and the reader only sends n queries for n tags. In practice, RFIDs face collisions and idle queries, so SE is far below 1. We therefore care about minimizing q for a given n .

To estimate how many queries Binary Splitting requires on average to identify n tags, we can write a recursion for the expected total number of queries, which I will call $BS_{tot}(n)$.

- For $n \leq 1$, the protocol needs only one query: the reader asks the broad question and either hears nothing ($n = 0$) or hears a single readable tag ($n = 1$). So $BS_{tot}(0) = 1$ and $BS_{tot}(1) = 1$.
- For $n > 1$, after the first query we always get a collision and must split the set into two subsets. Each tag independently goes left or right with probability $1/2$, so the number of tags k on the left follows a binomial distribution. The expected total queries satisfy

$$BS_{tot}(n) = \begin{cases} 1, & n \leq 1, \\ 1 + \sum_{k=0}^n \binom{n}{k} \left(\frac{1}{2}\right)^k \left(1 - \frac{1}{2}\right)^{n-k} (BS_{tot}(k) + BS_{tot}(n-k)), & n > 1. \end{cases}$$

This equation reads naturally: we pay one query for the root node, then we add the expected cost of resolving the left and right subtrees, averaged over all possible splits (the binomial coefficients give the probability of each split). Because left and right are symmetric you can also collapse the sum to obtain that the fraction of queries that actually return a single readable tag converges to about 0.38, meaning only 38% of reader queries produce useful, single-tag responses. The remaining 62% are spent exploring internal tree nodes (collisions) or probing empty branches (idle queries).

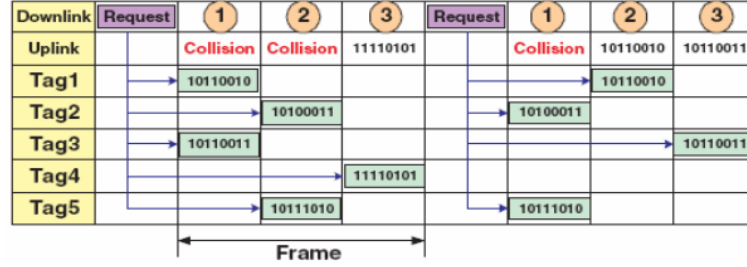
2.2.4 Framed Slotted Aloha

In *Framed Slotted Aloha (FSA)* the time is divided into slots and each slot is long enough for a tag to transmit its ID. A frame is just a group of consecutive slots.

1. When the reader starts a frame it tells tags how many slots the frame contains.
2. Each tag picks one slot at random and transmits in that slot.
3. If exactly one tag transmits in a slot the reader hears a readable ID.
4. If two or more pick the same slot their backscattered responses collide and the reader cannot decode them. Empty slots happen when no tag picks a given slot.
5. The process repeats itself until all tags are identified.

Example 2.2.3 – Framed Slotted Aloha

Suppose the reader uses 6 slots and, after tags choose, we end up with 3 slots that produced readable single-tag responses and 3 slots that were collision slots.



The simple system efficiency measured as

$$\frac{\#identifications}{\#slots} = \frac{3}{6} = 50\%$$

That toy example is optimistic; when you analyze expected behavior for large n the results change.

For large numbers of tags, average performance is maximized when the frame length N is tuned to match the number of active tags n . Intuitively, if you choose too many slots most slots are empty; choose too few and there are too many collisions. For large n the best average successful identifications is about 37%. Roughly 63% of slots are wasted (collisions or empties).

Transmission Time Model

EPC Gen2, the widely adopted commercial standard for passive RFID (Class 1 Gen2), is based on Framed Slotted Aloha. The standard adds practical machinery to improve performance: it defines explicit rules for how the reader adapts frame length based on observed collisions, successful reads and empty slots, and it specifies the low-level timing behavior necessary to compute how long each slot lasts.

Since Gen2 provides a transmission time model, we can translate slot-based metrics into absolute time costs, as illustrated in Figure 2.4. The timing model defines several relevant delays. There is a tag reaction time $R1$, which is the delay between the reader command and when the tag begins its reply. The reader has its own reaction time $R2$, and the reader also sets a threshold $RX_threshold$, the point in time by which it should observe the first bit of a tag transmission to conclude a tag is actually transmitting in this slot.

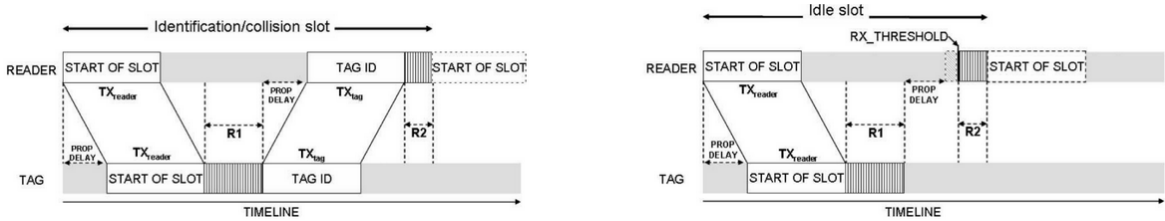


Figure 2.4: Transmission time model of EPC Class 1 Gen2.

That conversion reveals an important asymmetry: an idle slot — where no tag replies — typically lasts less than a slot with an actual transmission (successful or colliding). In an idle slot the reader detects absence quickly at $RX_threshold$ and moves on, while collision or successful slots consume the full transmission time.

Analytical Model

A classic mathematical abstraction for Framed Slotted Aloha is the balls-and-bins model. Think of N slots as N bins and n tags as n balls. Each tag independently chooses a bin with probability $1/N$. The number of tags that select a given slot follows a binomial distribution. If we are interested in the number of slots that contain exactly r tags, the expected count of such slots is

$$s(r) = N \times \binom{n}{r} \left(\frac{1}{N}\right)^r \left(1 - \frac{1}{N}\right)^{n-r}$$

Here $s(r)$ denotes the expected number of slots with exactly r tags, the binomial coefficient $\binom{n}{r}$ counts the ways r tags can occupy the same slot, $\left(\frac{1}{N}\right)^r$ is the probability those r specific tags chose that slot, and $\left(1 - \frac{1}{N}\right)^{n-r}$ is the probability all other $n - r$ tags avoided it. In particular, $s(0)$ is the expected number of idle slots, and $s(1)$ is the expected number of successful identifications within a frame.

From these expressions we can compute key quantities. The expected number of idle slots is

$$R_{idle} = N \times \left(1 - \frac{1}{N}\right)^n$$

and the expected number of successful slots (singletons) is

$$R_{ident} = n \times \left(1 - \frac{1}{N}\right)^{n-1}$$

The number of collision slots is simply what remains: $R_{coll} = N - R_{idle} - R_{ident}$.

When all slot types have equal duration, the system efficiency measured in slots is $R_{ident} / (R_{idle} + R_{ident} + R_{coll})$, and at the optimal point this equals about 0.36. If we account for different durations (say, idle slots last only a fraction β of the full transmission time), we can compute a time-based system efficiency by weighting each class by its duration:

$$Time_SE = \frac{R_{ident}}{\beta R_{idle} + R_{ident} + R_{coll}} = \frac{n \left(1 - \frac{1}{N}\right)^{n-1}}{(\beta - 1)N \left(1 - \frac{1}{N}\right)^n + N}$$

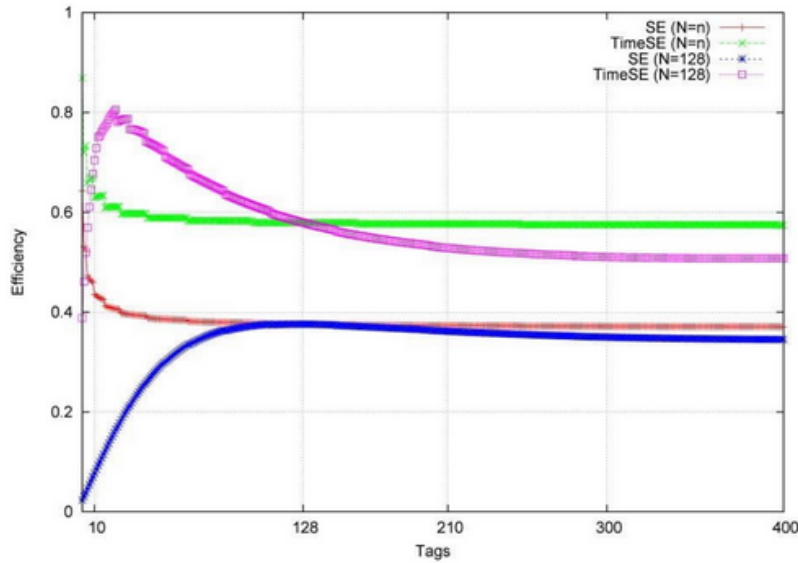


Figure 2.5: System efficiency and time system efficiency for FSA protocol.

2.2.5 Tree Slotted Aloha

Tree Slotted Aloha (TSA) tries to get the best of both worlds by adding a tree-structured progression of frames. Instead of using a single frame repeatedly, TSA executes frames following a tree traversal: when a slot collides, a new child frame is issued whose participants are only the tags that collided in that previous slot (see Figure 2.6). In other words, collision slots spawn sub-frames devoted solely to resolving that collision.

Picture the first frame as the root. Each slot in that frame corresponds to a child of the root. If a slot is idle it is a leaf with no tags. If it contains a single tag it is a leaf with a readable tag. If it contains more than one tag (collision), the algorithm issues a child frame that explores that slot's participants

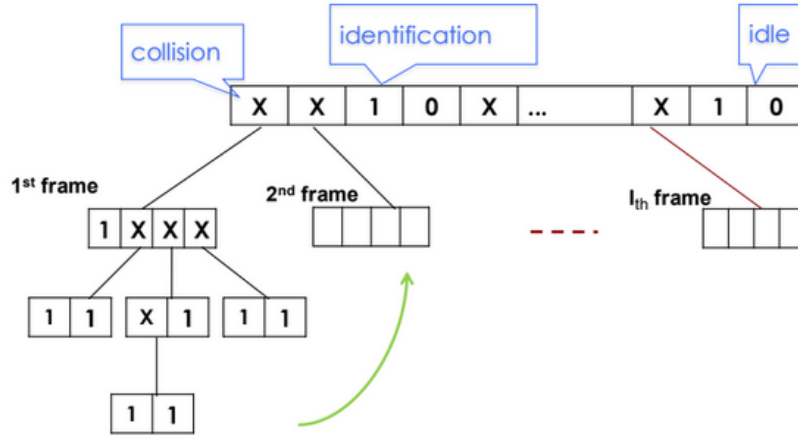


Figure 2.6: Tree Slotted Aloha.

through new slots, each of which becomes a child node in the TSA tree. This process continues until every collision subtree is resolved into readable leaves.

Counting slots in the TSA protocol is equivalent to counting nodes in this dynamic tree. Each node corresponds to a slot, and it is classified as idle, readable, or collision. The TSA total slot count $TSA_{tot}(n)$ for identifying n tags can be written recursively:

$$TSA_{tot}(n) = \begin{cases} 1, & n = 1, \\ n + n \sum_{k=2}^n \binom{n}{k} \left(\frac{1}{n}\right)^k \left(1 - \frac{1}{n}\right)^{n-k} (TSA_{tot}(k)), & n > 1. \end{cases}$$

When evaluated for large n , TSA measured in slots tends to an efficiency SE_{TSA} around 0.43, meaning about 43% of slot executions yield useful single-tag identifications.

2.2.6 Estimating tag population

Both FSA and TSA rely on choosing a frame length that balances collisions and idle slots. If the frame is too short many collisions occur; if too long many slots are idle.

The difficulty appears at two levels. First, the very first frame the reader issues needs a good initial size even though it has no prior observations. Conventionally people pick a reasonable default, like 128 slots, but that is often far from optimal when tag populations are very large or very small. Second, during identification, when the reader spawns new frames—especially in Tree Slotted Aloha (TSA)—it must estimate how many tags were involved in the collision that spawned the child frame. Bad estimates lead to inefficient frames, either too many empty slots or too many collisions. So estimating the tag population is central to good protocol performance.

A simple attempt at estimation uses the observed numbers of idle slots, identification slots and collision slots in the last completed frame. If you know those counts and you assume tags chose slots uniformly at random, you can invert the balls-and-bins formulas to obtain an estimate for n . The balls-and-bins model says that for a frame with N slots and n tags, the expected number of slots occupied by exactly r tags is

$$s(r) = N \times \binom{n}{r} \left(\frac{1}{N}\right)^r \left(1 - \frac{1}{N}\right)^{n-r}$$

In particular, the observed number of singleton slots should be close to the expected value $R_{ident} = n \times \left(1 - \frac{1}{N}\right)^{n-1}$, and the number of idle slots should be close to $R_{idle} = N \times \left(1 - \frac{1}{N}\right)^n$. If we observe triples of counts $\langle c_0, c_1, c_2 \rangle$ for idle, singleton and collision slots, we can try to find the value of n that minimizes some distance between the observed triple and the expected triple $\langle a_0(n), a_1(n), a_2(n) \rangle$. This is a minimum-distance estimator: search over possible n and pick the one that fits observed data best.

That sounds neat in theory, but there are practical problems. The variance can be huge; the minimum-distance search requires a reasonable range to search; and the upper bound for the search is tricky. A common yet crude bound is $2(c_1 + 2c_k)$ where c_1 is the number of singleton slots and c_k counts the collided slots. For some parameter settings this bound is disastrously small. Suppose there are $n = 5000$ tags, but the reader used an initial frame of $N = 128$. It is extremely likely that every non-empty slot will have multiple tags, so c_1 , the number of singleton slots, is zero. Plugging $c_1 = 0$ into the bound $2(c_1 + 2c_k)$ yields roughly 512 as the upper bound for n —far smaller than the real 5000 tags. The estimator then collapses and gives a meaningless result.

When N is small and n is large, the number of occupied slots concentrates on collisions: many tags crowd the same slots and singleton counts vanish. The estimator has almost no signal to distinguish between true $n = 5000$ and some much smaller n that produces very similar c_0 and c_1 values, so the estimated triples vary surprisingly little as n grows. Can we do better?

Instead of relying solely on frame observations to infer n , we can use randomized splitting techniques to actively partition the population until the groups are small enough to be estimated accurately or directly identified. This is the intuition behind two complementary solutions: a *dynamic TSA* (*Dy-TSA*) and the *Binary Splitting Tree Slotted Aloha* (*BSTSA*).

2.2.7 Binary Splitting Tree Slotted Aloha

Binary Splitting uses cheap randomness at the tags to split a large pool into two roughly equal halves; this works well because any large collection of items that randomly split tends to produce subgroups of similar size. TSA, on the other hand, is very good at identifying tags once the participating set size is moderate and a properly sized frame can be used.

BSTSA begins by applying randomized splits to break the initial massive crowd into smaller groups. Once a splitting branch reaches a small group — typically when the left child of a split contains a single tag — BSTSA switches to a Tree Slotted Aloha mode for the remaining sibling subgroups. In other words, use Binary Splitting for coarse partitioning and TSA for the final singulation and identification within each manageable subgroup.

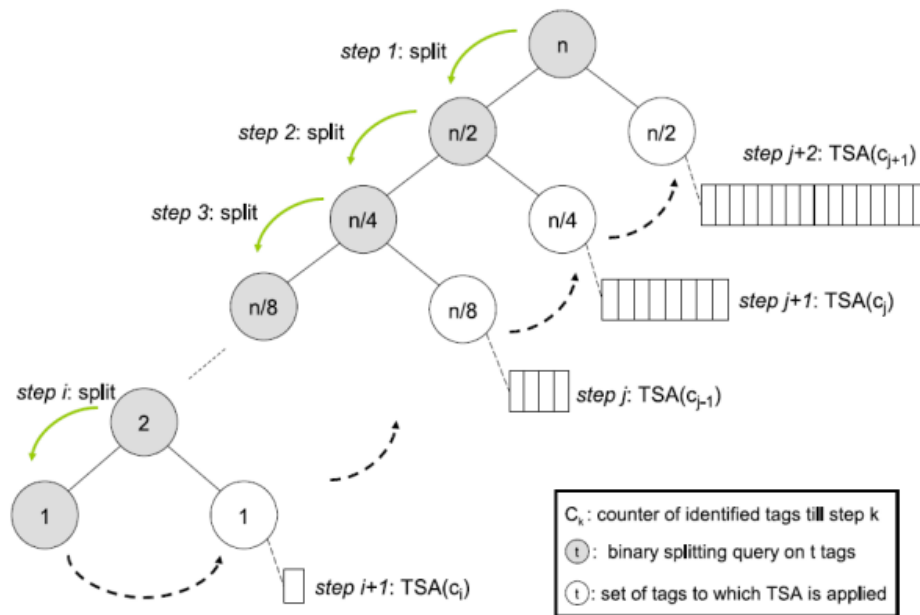


Figure 2.7: Binary Splitting Tree Slotted Aloha.

This hybrid offers a few practical advantages. First, splitting is cheap from the tag side and helps avoid the pathological case where an initial frame is tiny relative to the population. Second, after partitioning, TSA can adopt close-to-optimal frame sizes for each subgroup because the estimated group sizes are now in a range where balls-and-bins inversion is meaningful. The overall effect is improved time efficiency and reduced latency versus plain TSA or FSA with naive initialization.

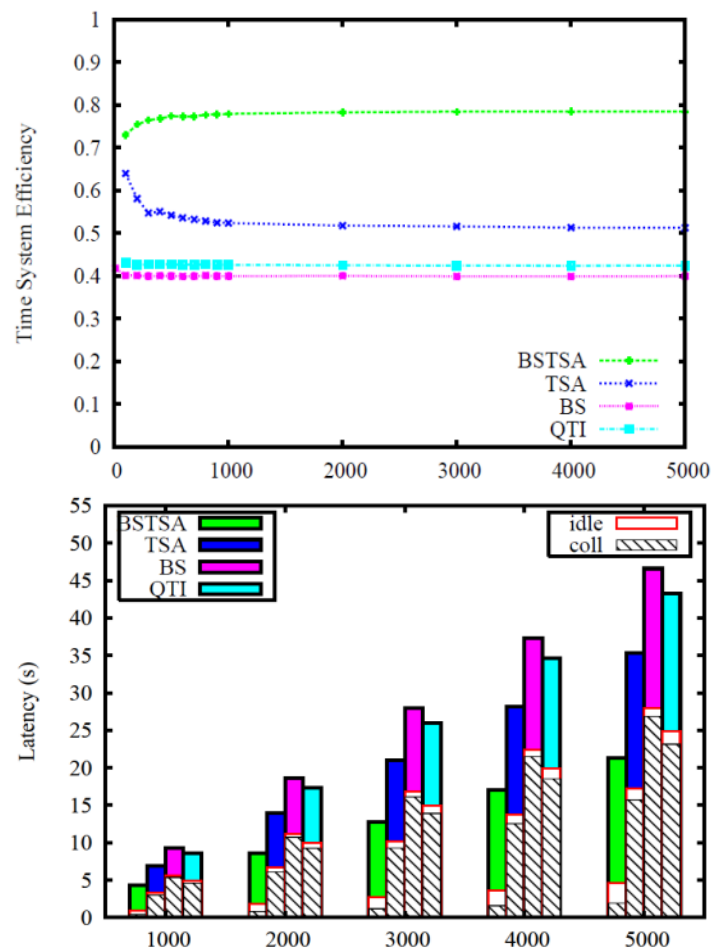


Figure 2.8: Time system efficiency and latency in Binary Splitting Tree Slotted Aloha.

Chapter 3

Sensor Networks

In this chapter we begin with an introduction to wireless sensor networks (WSNs), compare them briefly to RFID systems, and then move into applications and a practical MAC protocol used by many wireless systems (CSMA/CA).

An RFID system and a wireless sensor network might look similar at first glance because both involve small wireless devices dispersed in an area. But they have very different roles and capabilities.

- An RFID deployment is usually built around a reader that interrogates inexpensive passive or active tags. The tags typically send an identifier only. The communication topology is usually a star: the reader sits in the center and tags respond when interrogated.
- In sensor networks, a *sensor node* is more like a small computer with a radio, a microcontroller, some sensors, storage and a battery. Unlike passive RFID tags, sensor nodes can continuously sense environmental data, perform local computation, schedule transmissions, and run networking protocols (see Figure 3.1). Many sensor networks are multi-hop: data can hop from node to node until it reaches a sink or gateway that aggregates data and forwards it to servers on the Internet.

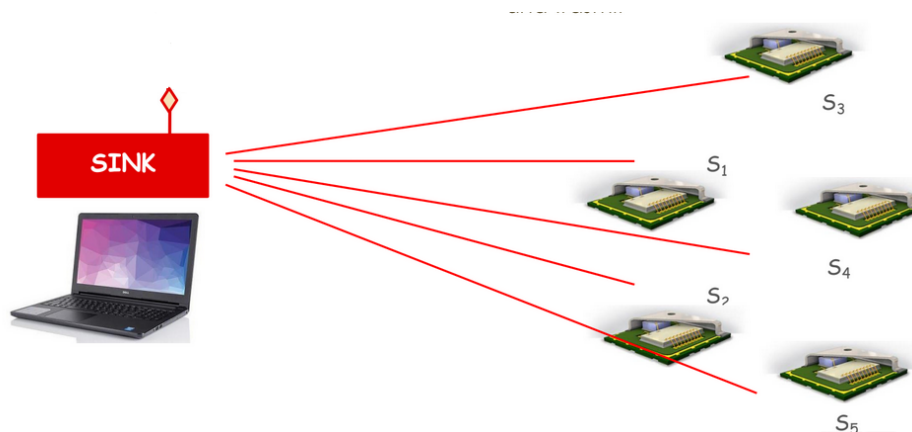


Figure 3.1: Sensor network.

3.1 Wireless Sensor Networks

A sensor node is composed of several components: a sensor to measure some physical quantity, an antenna and radio transceiver for wireless communication, a processor to run local code and coordinate sensing and communication, a battery that supplies power, and an optional flash memory for local logging.

A common architecture in WSN deployments goes like this. Each node senses the monitored area and periodically sends measurements to a sink node. The sink collects and may pre-process the data;

often it is connected to a server and the Internet so the data can be stored, analyzed or shown on user dashboards (see Figure 1.2).

There are three main benefits that explain why industries and researchers are so interested in WSNs.

- *Scale*: sensor networks let you cover large geographic areas, even in remote or inaccessible locations.
- *Autonomy*: once installed, they can run autonomously for long stretches, reducing the need for human intervention.
- *Timeliness*: because many applications require fast reaction, WSNs provide near real-time data and can trigger automated responses.

3.1.1 Applications of Wireless Sensor Networks

If a physical space needs monitoring or if a process would benefit from distributed sensing and feedback, a WSN can help. We'll now list the many application areas where WSNs are used.

- *Environmental monitoring and agriculture*. Sensor networks can measure temperature, humidity, light, rainfall, river levels and ground vibrations. A typical use case is using distributed sensors in a forest to detect early signs of wildfire, because local increases in temperature and changes in humidity can indicate conditions favorable to fire. In agriculture, sensors can measure soil moisture and temperature, allowing irrigation systems to water only where and when it is needed. This saves water and improves yield.
- *Structural Health Monitoring (SHM)*. SHM uses sensors to observe changes in a structure's material or geometry over time. The goal is to detect deterioration or damage early and avoid costly failures. SHM is relevant during construction, right after an impact or shock, and much later in a structure's life as it ages. Wireless sensor networks allow non-intrusive monitoring, often at a lower cost than wired solutions.
- *Health care and assisted living*. Sensors on or around people can track vital signs, movement, and activity to support rehabilitation programs, chronic disease management, or fall detection for elderly people living at home. Wearables like accelerometers and heart rate monitors can feed data to personal devices and cloud services to provide personalized wellness guidance.
- *Marine and underwater monitoring*. Placing sensors in the ocean gives us a way to map the seafloor, monitor ecosystems, or watch for pollution. Underwater wireless networks are their own subdiscipline because radio doesn't propagate well underwater — acoustic communications or special underwater optical links are often used instead.

3.1.2 Characteristics of WSN

In any WSN three logical roles appear repeatedly: sources, sinks, and actors.

- A *source* is a sensor node that measures environmental data and reports those measurements somewhere.
- A *sink* is the entity that wants to receive the data for storage, display, or analysis.
- An *actors* (also called actuators) react to data: they control actuators or devices when certain conditions are met, enabling closed loops between sensing and action.

How you place nodes depends on the scenario.

- When a site is inaccessible or when deployment must be quick, nodes can be dropped from an aircraft. This yields a roughly uniform random distribution across a finite area and is common in disaster response or environmental monitoring.
- When predictability and coverage guarantees matter, nodes are deployed in a planned, regular fashion. Regular placement is convenient in bridges, buildings, or industrial settings where you can attach sensors at pre-decided locations to monitor critical spots.

- Nodes can also be mobile. Mobility may be passive — a sensor drifting with water currents — or active — a robot or drone moving purposefully to improve coverage or gather data from interesting areas. Mobile nodes can help compensate for random deployment shortcomings, can reposition themselves in response to events, or can be used as collectors of data from stationary nodes.

At the hardware level a sensor node combines sensing, processing and networking on a small board (see Figure 3.2). The primary components are:

- An RF transceiver and antenna to send and receive wireless packets;
- A microcontroller or CPU to run software, perform preprocessing, and coordinate the node;
- A memory and flash storage to buffer and log data;
- A sensing unit composed of one or more transducers (temperature, humidity, accelerometer, etc.);
- A power source, usually a battery or energy harvester.

These pieces work together: the sensor reads analog values, a conditioning circuit cleans and scales the signal, an ADC converts it to digital, the CPU may compress or aggregate the reading, and the transceiver sends packets to neighbors or to a sink. Because power is precious, these functions are often duty-cycled: sensors and radios sleep much of the time and wake up briefly to sample or transmit.

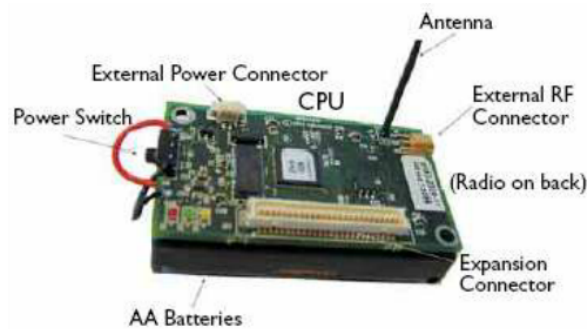


Figure 3.2: Sensor node components.

Embedded operating systems are common on sensor nodes to simplify programming and concurrency. TinyOS is a lightweight OS originally developed at UC Berkeley specifically for devices with strict resource constraints. TinyOS is written in and programmed using NesC, a dialect of C designed for component-based, event-driven programming.

A few practical constraints dominate WSN design.

- WSNs are expected to scale. A good design should support a large number of nodes without the system collapsing under its own signaling overhead. At the same time, the same design should work for small installations. So the protocols and algorithms used in WSNs must perform well across a wide range of node densities and deployment sizes.
- Nodes must be cheap, small, and energy-lean. Because they're usually battery powered, energy is the resource engineers worry about first.
- Memory and processing power are modest, radios are simple, and sometimes nodes don't even have a global address like an IP.
- Topologies can be mostly static, but there are growing deployments where node roles or positions change continuously — for example when mobile robots are used.
- Quality-of-service concepts familiar from classical networks need to be reinterpreted. Traditional metrics like throughput or latency still matter in some applications, but other concerns — energy efficiency, robustness to node failure, graceful degradation — are often more important.

- A sensor network should keep delivering the services it was designed for as long as possible; how “long” is defined by the application. Sometimes the goal is to maximize the lifetime of the whole network (e.g., keep the area monitored), while in other cases the lifetime of individual nodes matters less than preserving coverage or connectivity.
- Robustness is essential. Nodes can fail because they run out of battery, get damaged, or lose connectivity. The network should tolerate such failures and adapt: routes should reroute around dead nodes, and the system should be able to incorporate newly deployed nodes.
- Reprogramming nodes after deployment is often necessary — for bug fixes, configuration changes, or to change sensing behavior.
- Maintainability also includes the ability to self-monitor and adapt configuration at runtime. The network should be able to change duty cycles, adjust sensing schedules, or reassign roles to match changing conditions.
- A handful of mechanisms show up again and again because they address the key constraints of sensor networks. Multi-hop wireless communication is used to extend range and conserve energy by allowing nodes to forward packets for each other.
- Energy efficiency is pursued at every layer. Communication protocols minimize transmissions, computation is used to reduce transmitted data, and sensing schedules avoid wasteful sampling.
- Data-centric networking means designing around the information the network must provide rather than around node identifiers.
- Locality means do as much as possible locally — on a node or among its close neighbors — because local communication is cheaper than long hops.

Sensor networks exist to let us collect information from the physical world even in places that are hard to reach, to directly couple human decision-makers and automated systems to those measurements, and to provide information with precise spatial and temporal localization.

3.1.3 Wireless signal

Wireless communication is fundamentally different from wired because the radio signal weakens and distorts as it travels through space. Signal strength decreases with distance: energy is spread over a larger area as the wavefront expands, and in real environments obstacles cause additional loss. This attenuation is why range is limited and why placement and power settings matter.

When discussing ranges in wireless communications we commonly refer to three related concepts.

1. The *transmission range* is the distance over which the sender’s signal is strong enough that the intended receiver can decode it under typical conditions.
2. The *detection range* is the distance at which the receiver can detect that something is being transmitted but the signal may be too weak or noisy to decode correctly.
3. The *interference range* can be even larger: a transmitter far away may not be heard reliably, but its transmissions can still raise the noise floor at a receiver and disrupt reception of other nearby transmitters.

Real environments introduce multipath (see Figure 3.3): when radio waves encounters an obstacle, all or part of the wave is reflected, with a loss of power. A source signal can be reflected and multiple copies may arrive at the receiver at different times and phases. These multiple paths can add constructively or destructively, causing fading, delay spread, and intersymbol interference.

Interference arises both from multipath of the same sender and from other transmitters using the same frequency band. A receiver might receive several overlapping signals, and if they overlap in time and frequency they can corrupt each other. The signal-to-noise ratio (SNR) captures the ratio of good to bad signal. High SNR means the signal is stronger than the noise and the receiver has a good chance of decoding the data; low SNR means that the signal has been damaged by noise and errors are likely.

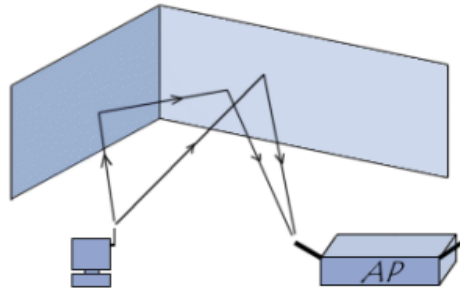


Figure 3.3: Multipath wireless signal.

Hidden and Exposed Terminal Problems

Two classic problems make medium access control in wireless networks interesting and often tricky: the hidden terminal problem and the exposed terminal problem.

Imagine three nodes labeled A, B, and C. A and C are out of range of each other, but both can reach B. If A wants to send to B and C also wants to send to B, A and C cannot sense each other's transmissions (they are "hidden" from each other), so they may transmit simultaneously. At B those concurrent transmissions collide and both packets are lost. The problem is "hidden" because A doesn't know about C and vice versa even though their simultaneous transmissions interfere at the receiver (see Figure 3.4).

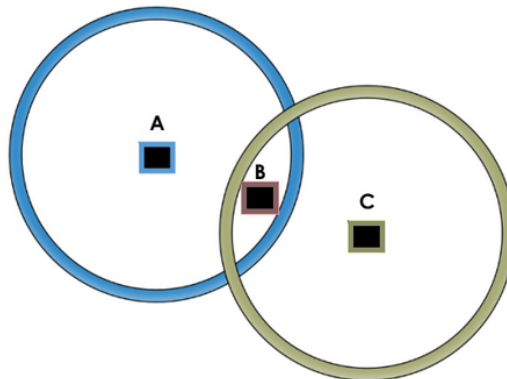


Figure 3.4: Hidden terminal problem.

Now imagine a different scenario. Suppose B is transmitting to A, and C is nearby the sender B but wants to transmit to D, which is out of range of B but within range of C (see Figure 3.5). If C senses B's transmission, it may refrain from sending because the medium appears busy. However, C's transmission to D might not interfere with B's reception at A, so the silence is unnecessary and hurts channel utilization. C is "exposed" to B's transmission and is being overly conservative.

Those two problems pull MAC design in opposite directions: you want to avoid collisions caused by hidden terminals, but you also don't want to unnecessarily silence nodes that could transmit safely.

3.2 Medium Access Control (MAC)

Medium Access Control — MAC for short — is the part of a wireless system that controls when a node should transmit and when it should listen. Those two operations — sending and listening — are arguably the most important operations for any wireless device, because they directly drive energy use and the user-visible performance of the system. Listening with the radio powered on, waiting idly for packets that may never arrive, can consume as much energy as a short transmission. So MAC design tries to:

- prevent collisions and therefore reduce retransmissions,

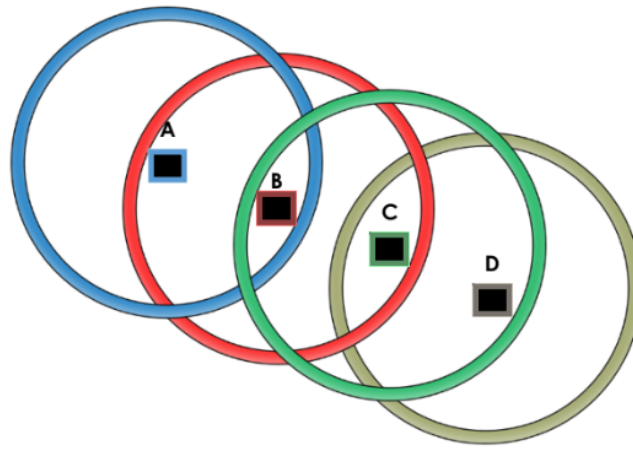


Figure 3.5: Exposed terminal problem.

- be energy efficient, especially by avoiding unnecessary idle listening,
- scale: a scheme that works for ten nodes but collapses at a hundred is not suitable for many WSN applications,
- meet latency requirements when they exist,
- be fair to nodes that need to transmit,
- use bandwidth efficiently,
- provide reasonable throughput.

In sensor networks, energy efficiency and collision avoidance often sit higher on the priority list than raw throughput.

Sensor nodes usually run from batteries. In many deployments swapping or recharging batteries is expensive, dangerous, or impossible. Because the battery capacity is finite, every design decision that affects how much time a node's radio or CPU spends active also affects the system's lifetime.

Nodes are often idle most of the time and wake only when something interesting happens, when they need to forward data, or on a periodic schedule. That sporadic activity pattern means the MAC must be tuned to avoid wasting energy during the long idle stretches while still providing timely delivery when activity happens. There are a handful of recurring ways energy is lost in WSNs:

- *Collisions* happen when two or more transmissions overlap at a receiver. When packets collide they are usually corrupted and must be retransmitted. Retransmissions cost both sender and receiver energy and increase network congestion.
- *Overhearing* happens when a node receives a packet intended for somebody else. Radios on nearby nodes will pick up transmissions they don't need, expend energy to receive and process headers, then discard them.
- *Control-packet overhead* includes the handshakes, beacons, periodic sync messages and other small packets that support the protocol. Too many control packets can offset the savings from reduced collisions or improved coordination.
- *Idle listening* is when the radio is powered and waiting for traffic that never arrives. Because listening power is non-negligible, long periods of idle listening quickly drain battery.
- *Overemitting* is wasted transmit energy when a sender transmits while the intended receiver is not ready — for example the receiver is asleep or out of range.

3.2.1 MAC protocols

Different applications induce different traffic patterns:

- One common pattern is broadcast or interest dissemination, where the sink or base station transmits a message intended for all nodes. This happens when a controller wants to query the network, push a configuration change, or distribute a code update.
- The other common pattern is convergecast or data gathering: many sensors send observations up to a single sink. This is asymmetric traffic (many-to-one) that can be used for collection of data from the environment or sensor network management and monitoring.

To design a good MAC protocol for wireless sensor networks, the following attributes must be considered:

- *Energy efficient.* It must conserve energy to extend network lifetime, yet adapt to changes in node density and topology.
- *Scalability and adaptability.* Protocols should behave sensibly from small testbeds to large-scale deployments.
- *Latency, throughput and bandwidth utilization* are important too, but many sensor applications can tolerate higher latency in exchange for longer node lifetimes.

At a high level MACs fall into two families: contention-based and schedule-based.

- *Contention-based* protocols allocate the channel on demand. Nodes that have data to send first sense the channel to check whether it's free; if it is, they transmit, often using randomized back-offs to reduce the chance that two nodes transmit at once. This approach is decentralized, scales without a central controller, and adapts dynamically to traffic, which makes it attractive in ad hoc and highly variable environments. But contention-based MACs suffer from idle listening, collisions, and interference; as node density increases, the probability of collision grows and energy costs can climb.
- *Schedule-based* protocols assign explicit times when each node may transmit. The schedule can be fixed or generated on demand, and when it is enforced collisions are essentially eliminated, which saves retransmissions and energy. They tend to be more energy-efficient in steady-state, because nodes know exactly when to wake and when to sleep. However, schedules require synchronization and some form of centralized coordination or distributed agreement to create the schedule, and that overhead can be costly.

Sensor networks often use hybrids: schedules for predictable periodic reporting, combined with contention-based access for bursty or event-driven traffic. Many modern MACs adaptively change behavior depending on load.

3.2.2 CSMA/CA

Contention-based MAC (Medium Access Control) protocols are the polite, distributed rules that let many wireless stations share the same radio channel without a brawl. The most widely used of these rules in Wi-Fi is called CSMA/CA — *Carrier Sense Multiple Access with Collision Avoidance*.

In wired Ethernet you can often detect a collision while you're transmitting: the voltage on the wire betrays that something else has transmitted at the same time, and you stop and retry. Wireless is not so forgiving. When a station transmits over the air it typically cannot listen at the same time and reliably measure whether its signal collided with someone else's. Because collisions are hard to detect, the philosophy shifts from collision detection to collision avoidance.

In CSMA/CA, when a station has a packet to send it

1. Listens to the channel.
2. If the channel is idle, the station waits a short fixed gap called an inter-frame space (IFS). If the channel remains idle for this IFS interval, the station may start transmitting immediately.
3. If the channel is busy when the station senses it, or it becomes busy during that IFS waiting period, the station defers transmission. It keeps monitoring the channel until the current transmission ends.

4. If the channel becomes idle again, the station does not necessarily transmit right away. Instead, to avoid many stations transmitting simultaneously after the channel frees up, each contending station enters a randomized backoff procedure: it chooses a random number of time slots from within a contention window and starts a backoff timer. That timer ticks down only while the channel remains idle. The first station whose backoff timer reaches zero takes the channel and transmits; others detect that transmission and freeze their timers, resuming them after the channel becomes idle again.

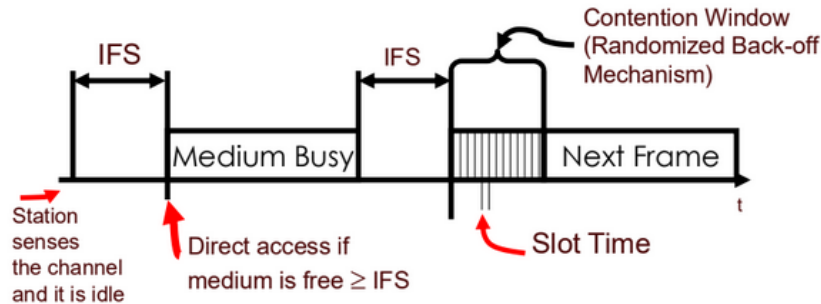


Figure 3.6: Basic CSMA/CA.

Each station chooses a backoff counter uniformly at random between $[0, CW - 1]$, where CW is the current contention window size. The backoff is measured in slot times; the slot length depends on the physical layer, but conceptually a slot is the basic unit of “wait” in the protocol. All stations decrement their counters one slot at a time while the medium remains idle.

When the channel becomes busy before the counter reaches zero, the station freezes the backoff clock and waits for the next opportunity. Once the current transmission finishes and the channel has been idle for the required IFS, the station resumes decrementing the remaining backoff.

If a station’s packet collides or otherwise is not acknowledged, the protocol increases the contention window. In plain terms, the station becomes more conservative: it picks its next random backoff from a larger range, so that subsequent retries are less likely to conflict with others. This increase is exponential: after each failed attempt the station doubles the effective contention window (up to some maximum). That doubling is called *binary exponential backoff*.

Not all transmissions are equal. Wi-Fi distinguishes priorities using different inter-frame spacing durations.

- The shortest is SIFS, the Short Inter-Frame Space. SIFS is used for highest priority, very short control responses such as acknowledgments (ACKs), CTS (clear to send) messages, and other immediate replies. Because SIFS is shorter than the other IFS values, the recipient can reply quickly before others try to contend for the medium.
- PIFS, the point coordination function IFS, is a slightly longer gap used in schemes with a centralized controller; it sits between SIFS and DIFS in priority.
- DIFS, the distributed coordination function IFS, is the longest of the three basic values used by the DCF. Stations waiting to start a new asynchronous data transmission must see the channel idle for a DIFS interval before beginning their contention procedure.

This ordering $SIFS < PIFS < DIFS$ ensures that short control replies (ACK, CTS) can be sent promptly and will not be preempted by ordinary data transmissions.

In the Distributed Coordination Function (DCF), an ordinary data transmission is followed by an acknowledgment from the receiver when the frame is correctly received (CRC check passes). The sender waits for DIFS before sending its data; the receiver, upon successful reception, responds after waiting only SIFS. Because SIFS is shorter than DIFS, the ACK from the receiver arrives before other stations opportunistically try to access the channel, preventing unnecessary retransmissions (see Figure 3.7).

If an ACK is not received in time, the sender assumes the frame was lost (either due to a collision at the receiver or a corrupted frame) and will schedule a retransmission following the backoff/retry rules.

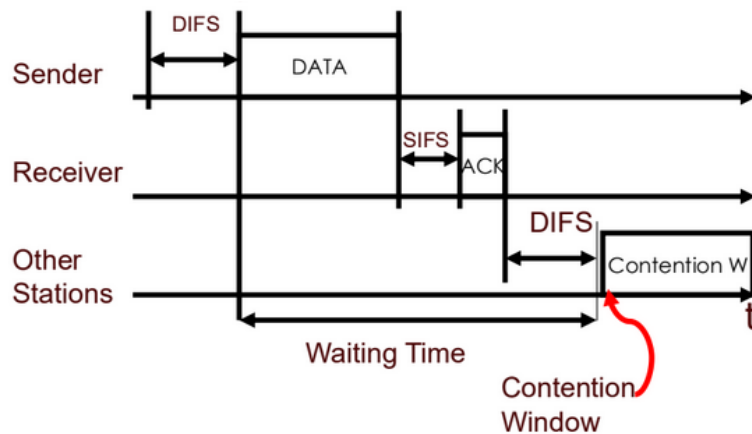


Figure 3.7: DCF CSMA/CA with ACK.

RTS/CTS

To address hidden and exposed terminals, discussed in Section 3.1.3, 802.11 provides an optional reservation mechanism: Request-to-Send (RTS) and Clear-to-Send (CTS). The sequence goes like this (see Figure 3.8).

1. A station that wants to send data first senses the channel and, if idle for DIFS, transmits an RTS control frame to the intended receiver. The RTS contains, among other things, a duration field that indicates how long the impending exchange (RTS, CTS, DATA, ACK) will take.
2. The receiver, if it hears and can accept the RTS, responds after SIFS with a CTS frame that echoes the duration.
3. Stations that hear either the RTS or the CTS set a timer called the Network Allocation Vector (NAV), which tells them how long the medium will be reserved. While the NAV is in effect those stations do not attempt transmissions, and so the sender can proceed to transmit the data packet without collision from those neighbors.

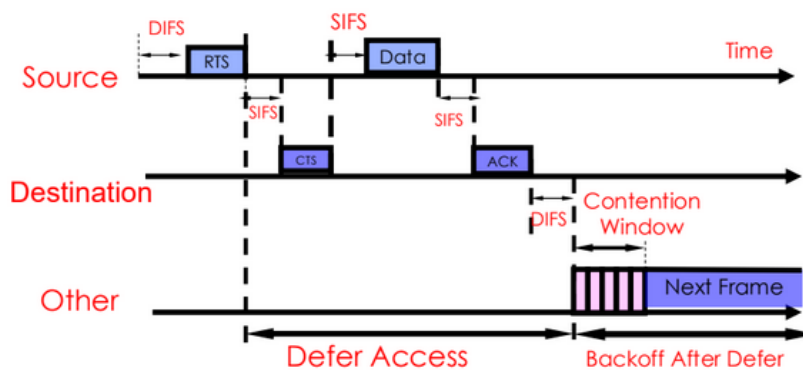


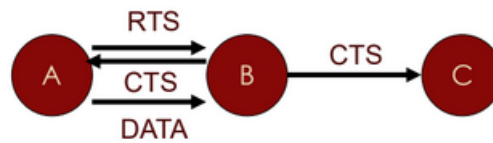
Figure 3.8: DCF CSMA/CA with RTS/CTS.

RTS/CTS reserves the channel in two helpful ways. First, it lets the receiver tell nearby nodes to be quiet (solving hidden terminals). Second, it helps exposed terminals recognize situations where they are incorrectly deferring: a node that hears an RTS but then hears a CTS from a station that it cannot hear will interpret the medium differently and may proceed to transmit to its own neighbor if the NAVs indicate no conflict.

Example 3.2.1 – How RTS/CTS solves the hidden terminal problems

Node A wants to send to B and sends an RTS. B responds with CTS. Node C, which could not hear A directly, will hear the CTS and thereby learn that B is about to receive from A. C sets its

NAV and refrains from transmitting, preventing a collision at B.



Example 3.2.2 – How RTS/CTS solves the exposed terminal problems

Suppose B is sending to A. Node C hears B's RTS to A and would normally defer. But if C examines the CTS and notices that it cannot interfere with the A-B exchange (for instance the NAVs and address fields indicate different receivers), C may be allowed to transmit to D.



RTS/CTS reduces many collisions but does not eliminate them entirely. RTS frames themselves can collide if two nodes attempt RTS at the same time. Fortunately, since RTS frames are short, such collisions are less costly than data frame collisions. Stations whose RTS or data attempts collide follow the same exponential backoff scheme and retry. In short, the protocol accepts that collisions are inevitable on a shared, distributed medium and uses randomized backoff and retries to recover gracefully.

Network Allocation Vector (NAV)

Virtual carrier sensing is implemented in 802.11 by something called the Network Allocation Vector, or NAV. The NAV is essentially a local timer that a station sets when it hears a frame that says "I'm going to use the channel for X more microseconds." That duration tells the listeners how long the sender expects to occupy the medium for this exchange, including any immediate replies (think CTS, DATA, ACK and the short SIFS gaps between them). When a station transmits, it includes in that duration field the time required for its whole planned exchange. Any station that hears the frame and is not the intended recipient converts that duration into its own NAV timer and begins counting down.

While the NAV timer is greater than zero the channel is virtually busy. That means the station will not attempt to access the medium. When the NAV reaches zero, the station then checks the physical condition of the channel — if the PHY reports the medium is idle for the required inter-frame spacing, the station can start its contention procedure.

To see NAV in action, picture the complete RTS/CTS/DATA/ACK exchange (see Figure 3.9).

1. The sender first listens until the channel has been idle for DIFS; then it sends an RTS (Request to Send). That RTS includes a duration value that covers the time from the end of the RTS through the planned reply and data exchange.
2. The receiver, after waiting only SIFS, replies with a CTS (Clear to Send). The CTS also contains a duration field: this is the remaining time of the planned exchange from the end of the CTS onward (the sender's original duration minus the brief interval used by the RTS and CTS themselves).
3. Every station that hears either the RTS or the CTS sets its NAV based on those duration fields. Those stations then remain silent for that NAV period.
4. Once CTS is received, the sender waits SIFS and then sends the DATA frame. The receiver, after SIFS, returns an ACK to confirm successful reception. When the ACK completes, the NAV reservations are effectively over and other stations may begin their own contention procedures.

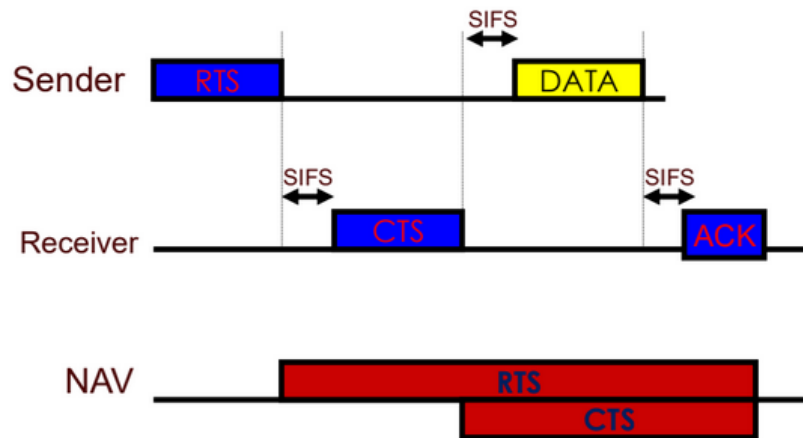


Figure 3.9: Network Allocation Vector (NAV).

3.2.3 S-MAC

This section walks through S-MAC, a contention-based MAC protocol designed for wireless sensor networks (WSNs).

Wireless sensor nodes are often battery powered and expected to run for long periods without human intervention. Measurements show that idle listening can consume significant amount of energy, somewhere between 50% and 100% of the energy required for receiving.

Because a naïve CSMA/CA node keeps the radio on and keeps sensing the channel, that idle time adds up and wastes battery. There are other energy drains too: collisions that force retransmissions, overhearing (nodes receiving packets that are not for them), and control-packet overhead (the extra messages used to coordinate access). S-MAC's core goal is to reduce those energy drains even if that comes at the cost of increased latency.

S-MAC asks nodes to be awake only when they need to be, and asleep for the rest. Nodes alternate between a listen period and a sleep period (see Figure 3.10). During sleep the radio is turned off completely and the node sets a timer to awake itself later, so it's not wasting power sensing the channel. When the node wakes up it listens for a while and either transmits, receives, or synchronizes with neighbors.



Figure 3.10: Sleep MAC.

Each node adopts a repeating cycle: listen for a fixed interval, then sleep for a fixed interval, then listen again, and so on. Two terms are useful here:

- *Listen interval*: how long the node keeps its radio on in each cycle.
- *Frame length (or cycle length)*: the listen interval plus the sleep interval.

The duty cycle is the fraction of time the node's radio is on. Mathematically,

$$\text{duty_cycle} = \frac{\text{listen_interval}}{\text{listen_interval} + \text{sleep_interval}}$$

Example 3.2.3 – Duty cycle

Reduce duty cycle to 10% (listen for 200 ms and sleep for 2 s).

Let's calculate that carefully.

$$\begin{aligned} \text{listen} &= 200\text{ms} & \text{sleep} &= 2\text{s} = 2000\text{ms} \\ \text{frame_length} &= 200\text{ms} + 2000\text{ms} = 2200\text{ms} \\ \text{duty_cycle} &= 200/2200 = 0.091 = 9.1\% \sim 10\% \end{aligned}$$

Example 3.2.4 – Duty cycle

If in each second a node sleeps for half a second and listens for the other half, duty cycle is 50%.

$$\begin{aligned} \text{listen} &= 0.5\text{s} & \text{sleep} &= 0.5\text{s} \\ \text{frame_length} &= 0.5 + 0.5 = 1.0\text{s} \\ \text{duty_cycle} &= 0.5/1.0 = 0.5 = 50\% \end{aligned}$$

If everyone chose an independent listen/sleep schedule, what if a node wants to talk to another node that is sleeping? To avoid that, S-MAC groups neighbors into virtual clusters that share the same schedule. Neighbors who share schedules wake up and sleep at the same times; this keeps control overhead low and simplifies communication inside the cluster.

To achieve and maintain this alignment, S-MAC uses periodic synchronization messages called SYNC packets. A SYNC packet is a short control message that tells to neighbors the sender's next sleep time. It typically contains:

- the sender's node ID (so neighbors know who sent it),
- the next sleep time or the next wake time (a timestamp or relative offset that lets receivers align their local timers),
- sometimes other small metadata about the schedule.

There are two synchronization phases:

1. *Initial sync*: when a node first joins the network, it listens during its active period for SYNCs from neighbors. If it hears one, it adopts that schedule and becomes a *Follower*. If it hears none after a reasonable interval, it picks a schedule of its own and becomes a *Synchronizer* that announces that schedule.
2. *Periodic sync*: after synchronization is established, nodes continue to periodically send out SYNC packets to maintain synchronization, compensating for any clock drift that might occur over time.

Each node keeps a small table of the schedules of its known neighbors (see Figure 3.11). Whenever a node receives a SYNC packet it records or updates the neighbor's entry: who the neighbor is and when that neighbor will next be sleeping or listening. That table allows a node to know which neighbors it can expect to reach during a given active period.

When a node decides to set a schedule:

1. First it listens to the medium for at least one synchronization period.
2. If it hears schedules from neighbors, it picks one of those schedules and becomes a *Follower*. *Followers* typically wait a random short delay and then re-broadcast the chosen schedule as a SYNC so neighbors know about it.
3. If the node hears nothing during that initial listen window, it picks its own schedule and immediately broadcasts a SYNC saying "this is my schedule." That node becomes a *Synchronizer* for nearby nodes.
4. The random delay used by *Followers* before re-broadcasting helps prevent a flood of SYNCs at once, reducing collisions during the control window.

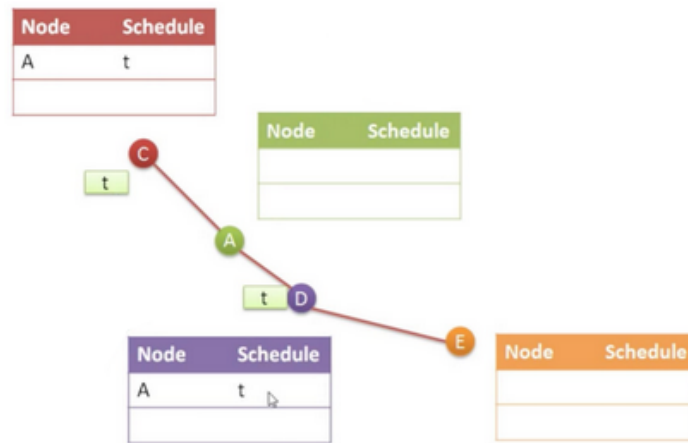


Figure 3.11: Schedule tables.

Because different parts of a network may adopt different schedules, you end up with virtual clusters of nodes sharing a schedule. A node on the border between two clusters will hear and store schedule information from both clusters. That border node has a tricky role: when it needs to broadcast a packet that should reach nodes in both clusters, it may need to send multiple times — once timed to the first schedule and once timed to the other.

S-MAC is a contention-based protocol, which means that multiple neighbors who want to send to the same node will all try during that node's listen window and must contend for the medium.

Before any transmission the node performs carrier sense: it listens to the channel to check whether someone else is transmitting. For unicast data, S-MAC follows the RTS/CTS/DATA/ACK handshake pattern (see Figure 3.8). The sender first sends a Request To Send (RTS). If the intended receiver hears it and the channel is free, it replies with Clear To Send (CTS). After that the sender sends the DATA packet and the receiver returns an ACK. Broadcast packets differ: because broadcast has no single receiving node to reply with a CTS and because ACKs are impractical for many receivers, broadcasts are sent without RTS/CTS. That leaves broadcasts vulnerable to collisions, but the trade-off is necessary for broadcast semantics.

Two kinds of carrier sense keep the medium state consistent. Physical carrier sense means simply listening to the channel and detecting ongoing transmissions. Virtual carrier sense comes from the NAV — the network allocation vector. Every transmitted packet includes a duration field that says how long this exchange will occupy the channel. Any node that overhears a packet records that duration in its NAV and sets a timer: during that interval the node treats the medium as busy. Combining physical and virtual carrier sense reduces accidental interference.

If a node tries to get the medium and fails (for example, it loses contention or senses the channel busy), S-MAC has an energy-aware approach: the node can go to sleep and wake up later when the receiver is expected to be free and listening again.

Finally, overhearing is reduced by a clever use of RTS/CTS. When nodes hear an RTS or CTS not addressed to them, they learn the duration of the upcoming exchange from the packet's duration field and go to sleep for that remaining time.

The process of sending SYNCs and coordinating them is often organized around minislots. Think of the active period as subdivided into many tiny minislots (see Figure 3.13). A node that wants to transmit a SYNC will pick one minislot to check the channel and to end its carrier sense. If the channel looks free, the node transmits the SYNC in the following minislot. This randomized minislot selection spreads SYNC transmissions across different small time offsets, reducing the chance that multiple nodes' SYNCs collide at the same instant.

S-MAC attacks the energy problem on several fronts:

- *Idle listening.* Turning the radio off during sleep windows dramatically cuts down this waste.
- *Collisions* are mitigated with RTS/CTS for unicast exchanges, which reduces the need for costly retransmissions.
- *Overhearing* is reduced because neighbors who hear an RTS or CTS not meant for them can sleep for the remaining duration.

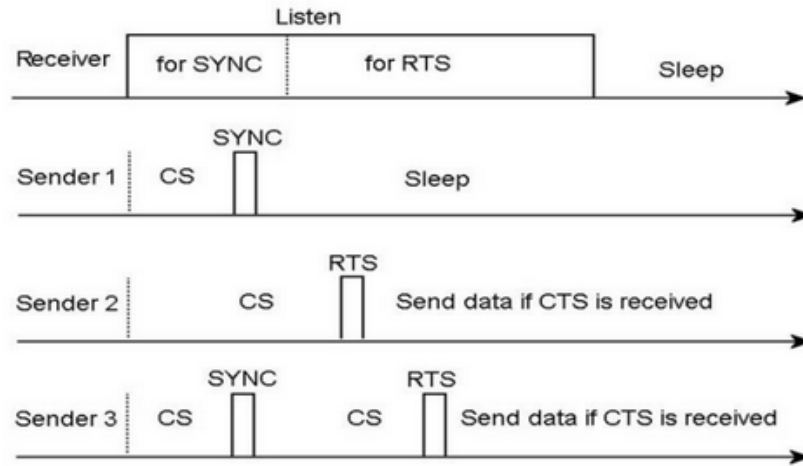


Figure 3.12: S-MAC synchronization.

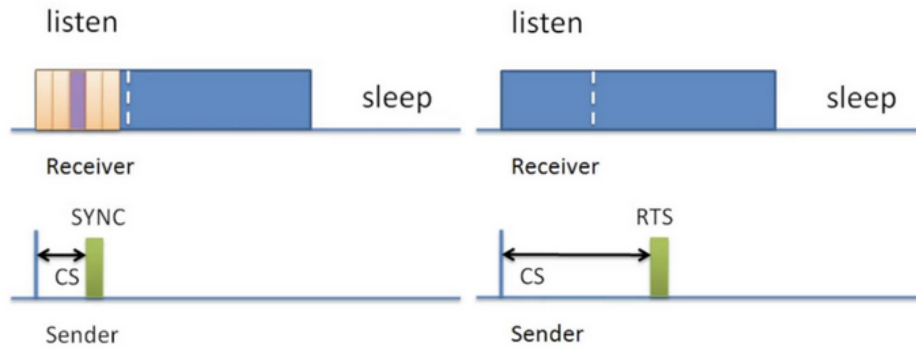


Figure 3.13: S-MAC minislots & data sending.

Consider a small two-hop topology with two sources and two sinks (see Figure 3.14). Each source periodically generates sensing messages that are split into fragments for transmission. The traffic load is varied by changing the inter-arrival period of messages: when the inter-arrival period is small (say 1 second) the traffic load is heavy because messages are generated frequently; when the inter-arrival period is large (say 10 seconds) the load is light.

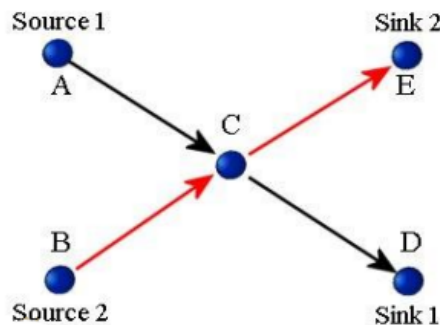


Figure 3.14: SMAC performance evaluation.

For each test the experimenters fixed the total amount of data to be sent so they could compare energy usage per node for delivering the same workload. Specifically, each source generated 10 messages in the test; each message was divided into 10 fragments, and each fragment was 40 bytes. If you multiply that out you get the total packet count: two sources $\times$ 10 messages $\times$ 10 fragments equals 200 fragments (or data packets) that need to be delivered from sources to sinks. The experiment then measures the total energy each node consumes while sending that fixed amount of data under different traffic patterns.

The energy curves were then plotted against the message inter-arrival period (the time between new messages), so the horizontal axis is the inter-arrival period (smaller values mean heavier traffic), and the vertical axis is energy consumption in millijoules (see Figure 3.15).

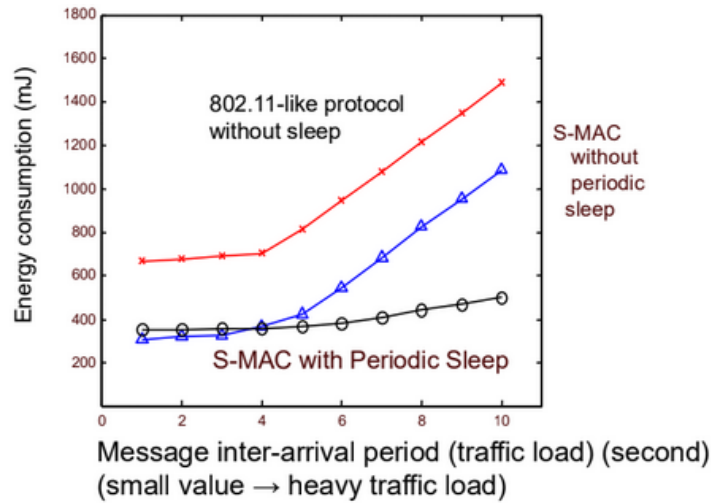


Figure 3.15: Average energy consumption in the source nodes A&B.

The plots compare three behaviors: an 802.11-like protocol without sleep, S-MAC without periodic sleep (which represents the S-MAC MAC behavior but with radios not sleeping), and S-MAC with periodic sleep. The key qualitative observations are:

- When the traffic load is heavy (inter-arrival period small), idle listening becomes less important because radios are busy transmitting and receiving most of the time. In this regime the energy savings that come from sleeping are limited — everyone is active frequently and contention and transmission dominate energy costs. Thus the three curves converge and the advantage of periodic sleeping shrinks.
- When the traffic load is light (inter-arrival period large), nodes spend much of their time idle if they don't sleep. That's exactly when periodic sleeping shines. S-MAC with periodic sleep uses far less energy than the always-awake 802.11-like baseline because the radios are off most of the time and the node avoids idle listening and overhearing. The measured energy per node drops substantially as inter-arrival increases.

S-MAC assumes a mainly static network where nodes don't move around much; schedules and neighbor tables are designed with that stability in mind. The protocol explicitly trades latency for energy: some transmissions must wait until receivers wake up, and border nodes may introduce extra delay because they juggle schedules.

Redundant data still flows, but it may flow with higher latency — S-MAC does not invent compression or deduplication; it optimizes when radios are on and off. Designers who need lower latency or higher throughput must either increase duty cycles or use other techniques (for example, reducing sleep intervals, using coordinated low-latency wakeups, or choosing a different MAC design).

3.3 Routing

Routing is what lets sensor nodes pass data along from where it was sensed to the base station that collects it. Most realistic deployments use multi-hop communication: a packet may traverse several intermediate nodes before it reaches its destination. Because sensor nodes are usually battery powered, the routing strategy you choose has a huge effect on how long the network will live. Good routing finds paths that use as little energy as possible and balances load so a few nodes do not die early.

At a minimum, routing must reliably deliver packets from sensors to sinks. Beyond that, the questions that guide design are practical and energy-centered. Do you want a routing scheme where

every node keeps a full picture of the network at all times, or one that only searches when it actually needs a route? Will node identities be arbitrary labels, or will you be able to use geographic position information? If nodes know their own locations, geographic routing becomes possible: you can pick the neighbor whose position is closest to the destination as the next hop, and that often reduces overhead because you don't need global route tables.

Another high-level design choice is how to view network topology itself. If nodes are symmetric and play similar roles, you get a flat topology. If nodes are organized into clusters with cluster-heads you get a hierarchical topology. When the network is flat, routing protocols for sensor networks fall into two major behavioral families, depending on when they operate.

- *Proactive or table-driven* protocols always tries to keep routes up to date. Each node maintains routing entries for reachable destinations at all times. If you need to send a packet, the route is already in the table so there is no discovery delay. The price you pay is continuous overhead: periodic updates must be sent to keep tables fresh, and that consumes bandwidth and energy. Those updates also risk being out of date because wireless links change, so protocols include mechanisms to mark stale routes and correct inconsistencies.
- *Reactive or source-initiated*, on-demand protocols only discovers a route on demand. Route discovery happens only when an application actually needs to send data to some destination. That avoids the constant overhead of proactive approaches but it introduces latency up front: sending the first packet requires time to find a route. Reactive protocols sometimes keep a route cache, so if a recent route is still valid you get the best of both worlds—low overhead and low latency for cached destinations.

Some protocols blend the two behaviors into hybrids, choosing proactive behavior for nearby destinations and on-demand lookups for faraway ones.

3.3.1 Destination Sequence Distance Vector (DSDV)

A classic example of a proactive adaptation for ad hoc networks is *Destination Sequence Distance Vector*, or DSDV. DSDV is a distance-vector protocol based on the distributed Bellman–Ford idea, but it adds sequence numbers and aging information to help prevent routing loops and to prefer fresh information.

DSDV periodically sends full routing updates. When topology changes, DSDV sends incremental updates that describe only the changes, reducing the volume of control traffic. If an update looks unstable the protocol may delay advertising the new route for a short time to avoid propagating transient, misleading information. That smoothing behavior reduces churn in the network state and lowers wasted transmissions.

Think of DSDV as everybody keeping a table that lists destinations, the next hop to reach each destination, and the cost (number of hops or other metric). In addition, each table entry carries a sequence number. A higher sequence number means the route information is more recent and therefore more trustworthy. When a node receives route updates from neighbors it prefers routes with larger sequence numbers. If two routes share the same sequence number, the one with the smaller cost (fewer hops) is chosen.

Sequence numbers solve a key problem of classic distance-vector algorithms: routing loops and count-to-infinity behaviors that can arise when nodes learn inconsistent information at different times. By tagging routes with sequence numbers and preferring fresh (higher-numbered) routes, DSDV converges toward loop-free behavior.

Let's walk through a small example. Picture four nodes arranged in a row: Node A connects to B, A to C, and C to D. Initially every node knows only about itself and its immediate neighbors (see Figure 3.16). Each routing-table entry stores a destination, next hop, hop count, and a sequence number that increases when the destination originates new information.

Start with each node advertising itself with sequence number zero (or some base value) and cost zero. Neighboring nodes learn reachable destinations via their neighbors and build entries. Suppose A learns about D via C with a hop count of two; B learns about D via A and D with hop count of tree, and so on (see Figure 3.16).

Now imagine the link between C and D goes down. C detects the link failure and wants to inform everyone that D is unreachable through it. C increments the sequence number for D and advertises the route to D with an infinite metric (or a special flag) and the incremented sequence number. When

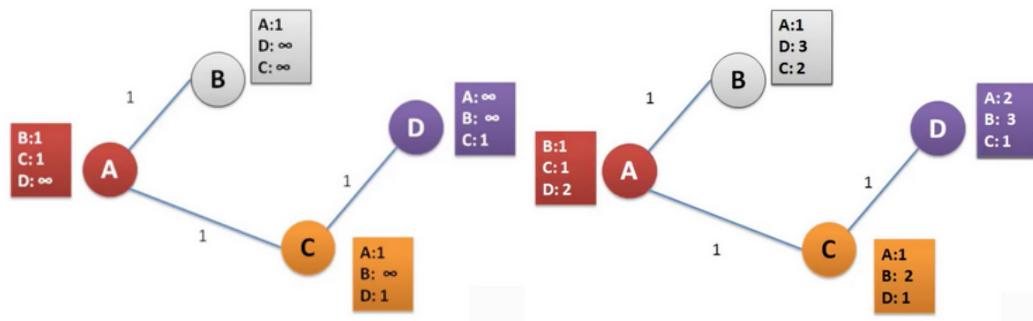


Figure 3.16: Destination Sequence Distance Vector example.

A and B receive that update, they will see the larger sequence number and will mark that route as broken. They will then search for alternative routes—if none exist, they will also mark D unreachable.

If later D becomes reachable again via a different route (say D moves close to B), then D or B will advertise a new route to D with an even larger sequence number and an appropriate hop count. Because sequence numbers only increase, nodes will prefer the freshly advertised route over the older one and converge toward a consistent view without forming loops.

3.3.2 Reactive Routing Protocols

Reactive, or source-initiated, routing for wireless sensor networks is an on-demand way to find paths: instead of keeping a big routing table all the time, nodes look for routes only when they need them.

Reactive routing waits until some source node actually wants to send to a destination before it tries to find a path. When that moment comes, typically the source floods a route-request message into the network. Nodes forward the request while recording reverse-path information so replies can take the forward path back. When the request reaches the destination or an intermediate node that knows a fresh route to that destination, that node sends a route-reply back. The source then caches the discovered path and sends its data.

The disadvantage is initial latency and the cost of flooding control messages when discovering routes. Route caches can compensate for the delay problem. If a source has a recent, still-valid route in its cache, it can send immediately without discovery.

Flooding and Gossiping

Two of the oldest reactive techniques are flooding and gossiping. They're conceptually straightforward and help illustrate why more sophisticated reactive protocols exist.

- *Flooding* is exactly what it sounds like. A node that wants to send a packet broadcasts it to all neighbors. Each node that receives the packet forwards copies to all of its neighbors except the one it heard the packet from. Flooding tries every possible route automatically, so it has two nice properties: if any route exists between source and destination, flooding will find it; and one copy of the packet will travel along the fastest possible route, because flooding explores all routes in parallel.

Those benefits come at a terrible cost. Flooding generates an enormous amount of redundant traffic. Nodes may receive many duplicate copies of the same message and forward them, producing implosion: neighbors repeatedly rebroadcast identical messages to the same nodes. If two sensors monitor the same event region, they might both send the same stimulus data at the same time and create overlap—redundant sensing that multiplies forwarding load. Worse, flooding is resource-blind: it doesn't care how much battery each node has. Every node in the broadcast domain receives and forwards the packet, draining energy and shortening network lifetime.

- *Gossiping* is a lower-chaos alternative. Instead of forwarding a packet to all neighbors, a node forwards it to a single randomly chosen neighbor (or to a small random subset). This avoids implosion because you typically keep just one copy of the message per node, but the tradeoff is slower spread: reaching every node in the network takes more hops and more rounds of

random forwarding. Gossiping reduces redundancy and conserves energy compared to blind flooding, but it's not ideal if you require fast, deterministic delivery to a specific destination.

Dynamic Source Routing (DSR)

Dynamic Source Routing (DSR) is a clean, elegant reactive protocol that uses source routing. In source routing the sender places the full ordered list of hops that the packet should follow inside the packet header. That header tells every forwarding node exactly where to send the packet next; no per-node routing table lookups are needed for forwarding.

DSR has two core mechanisms that work on demand.

- *Route discovery* is invoked when a node has a packet for some destination but no suitable route in its cache.
- *Route maintenance* watches the paths currently in use and notifies the sender if a link along a source route breaks.

Here's how route discovery works, step by step, in the classic RFC example. Imagine node A wants to reach node E but doesn't have a route cached. A broadcasts a Route Request (RREQ). The RREQ contains three critical pieces of information: the identity of the initiator (A), the identity of the target (E), and a unique request identifier. It also contains a route record that starts as an empty list (or as [A] if you consider A listed).

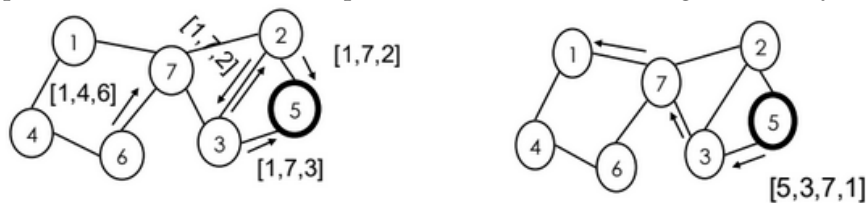
Each node that receives the RREQ appends its own address to the route record and rebroadcasts the request, unless it has already seen this RREQ identifier (in which case it discards the duplicate). Over time the route record accumulates the sequence of nodes the RREQ has traversed: first A, then A,B, then A,B,C, then A,B,C,D, and finally the request reaches E with the record [A, B, C, D].

When the target E receives the RREQ, it can construct a Route Reply (RREP) containing the complete reverse route. If source routing is used in the reply, the RREP will list the route E,D,C,B,A or simply use the accumulated record reversed. The initiator receives the RREP, caches the discovered source route, and starts sending data with that source route included in each packet header.

A few practical niceties follow from this mechanism. First, because the source route is in the packet header, any node that overhears packets can cache the route for future reuse. Second, a single route discovery may discover multiple distinct routes because different RREQ copies can reach the destination via different paths; the initiator can cache several alternatives to react quickly to failures. Third, when a link on a cached route breaks during data forwarding, the sender is informed (often by an explicit route-error message generated by the detecting node), and the sender can try another cached route before initiating a new global discovery.

Example 3.3.1 – Route discovery

To make this concrete, visualize a small network of nodes labeled 1 through 7. Node 1 broadcasts a route request for node 5. As the request spreads it may arrive at different neighbors with different accumulated route records: one copy might arrive at a node as [1,4], another as [1,7], another as [1,7,3], and so on, depending on which path the RREQ took. When node 5 finally receives one of these copies, it sends back a reply that includes the accumulated record—say it replies with [5,3,7,1], which reversed yields the forward path 1→7→3→5. That route is cached and immediately usable. Because multiple RREQs can reach 5 through different paths, node 1 can learn multiple distinct routes from a single discovery event.



Route maintenance detects when a link in a source route stops working. For example, if node X forwards a packet but does not hear the expected link-layer acknowledgment, X can generate a route-error and send it back to the source. The source then removes any cached routes that use the broken link and either tries another cached route or launches a fresh route discovery.

Because DSR supports caching multiple routes to a single destination, recovery is often quick: a failed route can be swapped out for a backup route without immediate global flooding.

Ad hoc On-Demand Distance Vector (AODV)

Ad hoc On Demand Distance Vector (AODV) takes much of the discovery spirit of DSR but avoids source routing and the large per-packet header overhead by using routing tables at intermediate nodes. AODV is widely used in ad hoc networking and is a natural reactive complement to DSR.

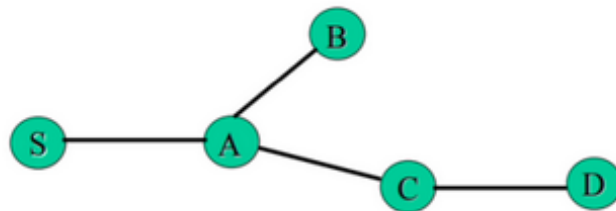
When a source needs a route and has none in its table, it broadcasts a Route Request (RREQ). Intermediate nodes forward the RREQ, and while doing so they record a reverse path toward the source: the first neighbor from which the RREQ came is recorded as the next hop toward the source. When the RREQ reaches the destination (or an intermediate node that knows a fresh route), that node unicasts a Route Reply (RREP) back along the reverse path. As the RREP is forwarded back to the source, intermediate nodes install forward route entries pointing to the neighbor from which the RREP was received. After the process completes, the source and the intermediate nodes along the path all have routing-table entries enabling future unicast forwarding without carrying the full path in packet headers.

AODV also uses sequence numbers to ensure freshness and avoid routing loops. Each route entry is tagged with a destination sequence number, and nodes only adopt routes with newer (larger) sequence numbers, similar in spirit to DSDV. Sequence numbers help nodes decide whether an intermediate node's cached route is fresh enough to be used when responding to a RREQ.

The learning of reverse and forward paths is a neat, efficient side effect: the RREQ sets up the reverse path, while the RREP establishes the forward path. If a link fails, AODV supports local repair or route error messages that notify affected upstream nodes, which then invalidate stale entries and may trigger new route discoveries.

Example 3.3.2 – AODV

Let's walk through an example discovery from the point of view of one source S wanting to talk to destination D.



Source S has a packet for D but no route in its table. S broadcasts a Route Request (RREQ). The RREQ contains the identities of S and D, a unique request identifier, and other control fields such as sequence numbers that help judge route freshness.

When a neighbor (call it A) receives S's RREQ it stores a reverse route entry toward S in its routing table: "To reach S, next hop is S." That reverse entry is used later to deliver replies back to the source. Then A rebroadcasts the RREQ to its neighbors.

The next node (call it C) that receives A's rebroadcast records a reverse route toward S as well. In C's table you now have "To reach S, next hop is A." C then rebroadcasts the RREQ onward. At some point the RREQ reaches a node that can answer. This might be the destination D itself or some intermediate node that already has a fresh route to D. Suppose node C is such a node and decides it can answer. C creates a Route Reply (RREP) and unicasts it back toward S using the reverse path already recorded.

As the RREP travels back along the reverse path (for example $C \rightarrow A \rightarrow S$), each intermediate node installs a forward route toward D. Concretely, when A receives the RREP from C it records "To reach D, next hop is C." Inserting these forward entries prepares intermediate nodes to forward subsequent data packets without requiring source routing.

Finally the RREP reaches the originator S. S installs a route entry "To reach D, next hop is A" (assuming A is the neighbor on the reverse path) and can immediately start sending data along the newly discovered path $S \rightarrow A \rightarrow C \rightarrow \dots \rightarrow D$.

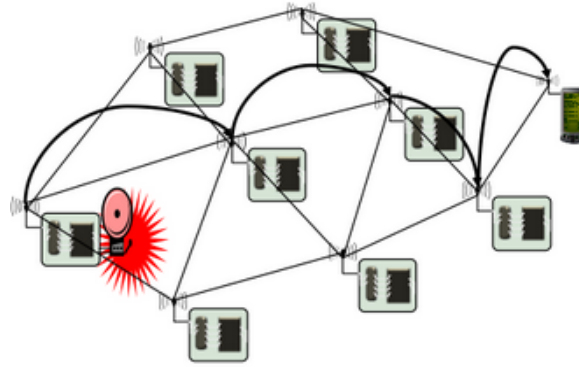


Figure 3.17: Position based routing: most forward within range r .

3.3.3 Geographic routing

Traditional routing expects each node to keep a table that maps destinations to next hops. An alternative, which is very natural for sensor networks, is to infer forwarding decisions implicitly from the physical placement of nodes. If every node knows its own position, knows the positions of its immediate neighbors, and knows the destination's location, the node can choose a neighbor that moves the packet closer to the destination.

This family of ideas is called *geographic routing* or position-based routing. It's especially appealing in large sensor deployments because you avoid the global state and periodic updates required by full routing tables. Two related concepts are worth naming: *geocasting*, which is "send a packet to any node in a given geographic area," and *position-assisted routing* more generally, where a location service may be used to map a node ID to a geographic coordinate when the node's exact location is needed.

The most common greedy strategy is called "*most forward within range r* ". Imagine you are node S and you want to forward a packet toward destination D . Look at all neighbors within your communication range (distance $\leq r$) and pick the neighbor that makes the most forward progress toward D . Concretely, that means pick the neighbor for which the remaining distance to D becomes smallest after the hop (see Figure 3.17).

One important clarification: "most forward progress" is not the same as "farthest from the sender." A neighbor that is far from S but sideways relative to the $S \rightarrow D$ line may not reduce the remaining distance to D at all. You want the neighbor that reduces the remaining distance to the destination most, not the one that simply maximizes hop length.

Greedy is the classic baseline, but designers often tweak it to meet other goals:

- choose the nearest neighbor that still makes forward progress. The idea here is to minimize transmit power: short hops consume less power per transmission even though they may increase hop count. If radios support power control, taking a close neighbor that still moves the packet forward can save energy.
- directional or angular routing. Instead of comparing distances, pick the neighbor whose bearing is closest to the bearing of the destination (angular closeness). Or pick the neighbor whose projection onto the straight line from S to D is closest (minimize perpendicular distance). These ideas try to keep the packet moving along a straight path, but they can accidentally form loops if the geometry and neighbor set interact poorly.

Greedy forwarding can fail. A packet can arrive at a node that has no neighbor closer to the destination than itself. That situation is called a *dead end* or local minimum (see Figure 3.18). When greedy forwarding hits a dead end, you need a fallback strategy.

When nodes know positions, routing can be dramatically simpler and more scalable. Position-based techniques avoid the overhead of maintaining global route tables and they scale nicely: each node only needs neighbor coordinates (local information) and possibly a lightweight location service when destination coordinates aren't known locally.

Which approach is best still depends on network characteristics: mobility, node capabilities (do they have GPS? can they measure signal strength?), application traffic patterns, energy budgets, and so on. For highly mobile networks (drones, vehicles), position-based methods can help because they

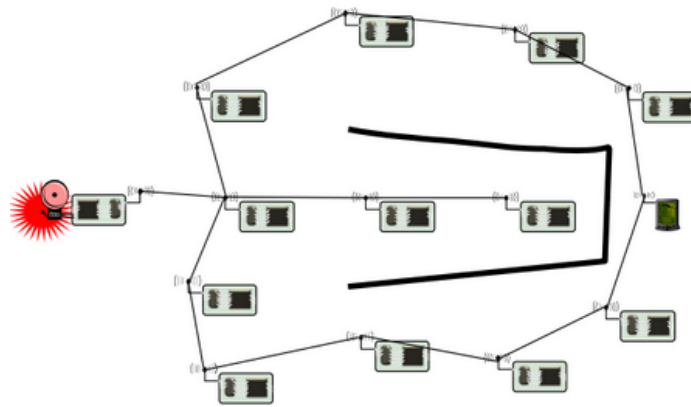


Figure 3.18: Dead ends.

avoid stale global tables and position-based routing uses local neighbor information that changes locally and can be updated cheaply.

Chapter 4

Networks of Drones

In this chapter we begin with what an unmanned aerial vehicle (UAV) is and the degrees of autonomy drones can have. Then we walk through the networks of drones — how multiple UAVs cooperate, what a “dronet” is, and the practical problems that researchers and engineers care about.

4.1 Unmanned Aerial Vehicle (UAV)

A *UAV* is simply an aircraft that flies without a human pilot on board (see Figure 4.1). This vehicle may operate with various degrees of autonomy: at one extreme a human on the ground directly pilots every motion via remote control; at the other extreme the aircraft makes all decisions onboard (like sensing, planning and acting) without human intervention.



Figure 4.1: Types of UAV.

When we say “autonomy” we mean the ability of the drone to sense its environment and take decisions without continuous human input. This can be only partial (e.g., automatic takeoff/landing or waypoint following) or full mission autonomy (find, track and act on targets by itself). Drones vary enormously in size, capability and endurance, and those differences shape how they are used. To give a sense of the extremes: tiny research prototypes can weigh less than a gram, while very large high-altitude military drones weigh many tonnes. Speed also spans a wide range: hobby and commercial quadcopters typically cruise at tens to a few hundreds kilometers per hour, whereas some experimental unmanned hypersonic vehicles reached many thousands of kilometers per hour. Propulsion and energy are fundamental constraints. Fuelled propulsion (gasoline, methane, hydrogen) gives long endurance and higher speeds but adds complexity, emissions and maintenance. Electric propulsion dominates small and medium commercial drones because batteries (Ni-Cd, Ni-MH, Li-Po, and emerging Li-S) are light, quiet and easy to manage; the trade-off is flight time. Solar cells are sometimes used to extend endurance for high-altitude, long-duration platforms.

Finally, drones come in different mechanical forms: single-rotor helicopters, multi-rotor (quad, hexa, octo), fixed-wing airplanes and lighter-than-air platforms like airships. Each form has trade-offs: multi-rotors hover and offer high maneuverability at the cost of energy efficiency, while fixed-wing designs are efficient for covering long distances but need forward speed to stay aloft.

Drones bring a few advantages that explain why they are being widely adopted. First, they can be deployed quickly and reach areas that are otherwise inaccessible, which is invaluable in disaster response. They can carry small but critical payloads — medical supplies, sensors or communications equipment — to places where roads are blocked or dangerous. They reduce human exposure to hazardous environments, whether that’s a collapsed building, wildfire perimeter or polluted site.

Beyond emergencies, drones are useful for everyday tasks such as traffic monitoring, wind estimation for meteorology and remote sensing for agriculture or infrastructure inspection. Their flexibility and relatively low cost make them appealing for many sensing and monitoring roles.

4.2 Dronet

When multiple UAVs operate together in the same area and coordinate, we call that a *dronet* or UAV network (see Figure 4.2). The point of a dronet is cooperation: drones share sensor data, plan their movements to avoid overlap, forward mission-critical information, and sometimes act as flying communications infrastructure.

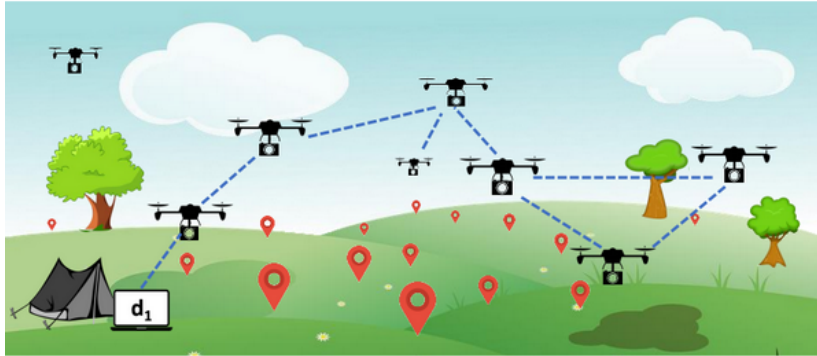


Figure 4.2: Dronet.

Imagine an area of interest where several drones are flying. Some might be focused on sensing while others form a communication backbone to move the sensed data back to a control station. Often there is also a depot or control center on the ground that owns the mission and may provide higher level commands.

In many scenarios, successful operation requires drones to exchange substantial amounts of data: high-resolution images, video streams, telemetry, position and status updates, maps and mission plans. Designing networks that move this data efficiently and reliably is one of the main engineering challenges.

There are four central issues in dronets:

1. How to provide communication in emergency scenarios where there is no network infrastructure available? After storms, earthquakes or in very remote areas, cellular towers and wired backhaul may be down or non-existent.
2. How to optimize task assignment and trajectory planning for UAVs operating in real-world scenarios? Each drone has limited battery, limited payload and physical motion constraints. The planner must assign which drone does which job and compute paths that respect these limits while minimizing time, energy or mission risk.
3. How to perform continuous data offloading when deployed in large and harsh areas? Continuous offloading is when drones stream data as they collect it.
4. How to perform periodic data offloading when deployed in large and harsh areas? Periodic offloading is when drones store data locally for some time and only upload when they rendezvous with a depot or come into contact with a stronger link.

4.2.1 How do we provide communication in emergency situations where there is no network infrastructure?

Drones can act as flying base stations, temporarily providing wireless access to users on the ground. They can act as relay nodes that bridge line-of-sight gaps, forming a mobile mesh that routes traffic from isolated teams to the internet or a control hub. They can act as backhaul nodes: a high-altitude drone can link a cluster of lower drones to a satellite or a fixed ground station.

Always remember the constraints: limited battery, payload, regulatory restrictions, weather sensitivity, and the cost/complexity of operation. The higher the autonomy and the more integrated the networking capability, the more complex the onboard software becomes and the heavier the energy burden. That's why mission design usually involves trade-offs: fewer sensors and more flight time, or richer sensing at the cost of frequent returns to the depot.

As case study the DANGER system from the academic literature: "DANGER: a Drone-Aided Network for Guiding Emergency and Rescue operations," authored by Andrea Coletta, Gaia Maselli, Mauro Piva and Domenicomichele Silvestri, is an example of research that tackles the problem of using drones to provide on-the-spot communications for emergency response.

4.2.2 How to optimize task assignment and trajectory planning for UAVs operating in real-world scenarios?

That short description hides a lot. Let's translate it into what you actually need to think about.

1. *"Multi-depot"* means drones don't all start or charge from one fixed point. In an emergency, some drones might be based at a vehicle, others at a rooftop depot, and yet others at a temporary field base. That affects how you route drones between missions and where they can refill energy or offload data.
2. *Squads and heterogeneity* means that some drones carry thermal cameras and must hover to acquire useful frames; others are relay drones with larger batteries but limited sensing; some can carry extra batteries or replacement radios. Assign the right drone to the right trajectory and the right task.
3. The *"set of trajectories"* formulation is important because in many deployments you predefine mission patterns — sweep lines, circular loitering trajectories, or orbiting paths around a site — and select which of those predefined patterns each drone will fly.
4. The *"required target hovering time"* means each target imposes a minimum dwell time before the drone can be considered to have completed its sensing or communications goal.

4.2.3 How to perform continuous data offloading when deployed in large and harsh areas?

When drones carry sensors they produce data that must reach the depot or operators. Continuous offloading means streaming data as it is collected, such as a video feed streamed from a search drone to a command vehicle. Continuous offload requires low latency and sustained bandwidth. This typically implies either line-of-sight wideband links (e.g., Wi-Fi at short range, directional links) or relayed multi-hop connections where drones forward data along a chain to a gateway drone with a backhaul (satellite or ground link).

Dronets (UAVNETs) look similar to wireless sensor networks or MANETs, but they change the game in several ways. Mobility in drone networks is medium to high, and it can be three-dimensional. Topologies are fluid; drones move, hover, leave for recharging and reappear. Node failure is common either because of batteries, hardware faults or collisions. Energy is constrained and often the single most important resource. Crucially, moving a drone typically costs far more energy than sending a modest radio packet. These characteristics affect every networking layer.

MAC

Drones commonly carry multiple radio modules depending on mission needs. Wi-Fi is a common choice because it's widely supported by user devices and offers wide bandwidth at short ranges. Cellular can provide robust backhaul and mobility support, but in many emergency or remote scenarios cellular connectivity is not available. Low Power Wide Area Networks like LoRa give long range and low energy consumption, but they offer tiny bandwidth and are inappropriate for high-rate video.

Routing

Routing in dronets is driven by the need to stream wideband content from moving drones back to a depot. Large-area monitoring may require multihop connections because a single wideband link seldom covers the whole area. The question is “which routing protocol?”

Comparing classical Wireless Sensor Networks to dronets makes the differences clear. WSN nodes are usually static or mildly mobile, energy harvesting might be available, and the topology is relatively stable. Dronets are highly mobile in 3D, topologies are fluid, nodes fail more often, and the mission contexts include rescue and surveillance with strict real-time demands.

Because of these differences, routing goals change. In dronets we care about keeping overhead low, guaranteeing loop freedom, ensuring scalability as the number of drones grows, conserving energy, reducing delay especially for high-priority traffic, and increasing delivery ratio under mobility.

- *Proactive protocols* keep tables of routes to every other node and periodically refresh them. The advantage is that routes are immediately available when you need them. The downside is control traffic overhead and slow convergence under very high mobility; keeping route tables consistent across fast-moving nodes is expensive and can cause stale information. For these reasons, plain proactive protocols often struggle in high-mobility dronets unless carefully tuned.

Examples to be aware of are OLSR (Optimized Link State Routing), DSDV (Destination Sequenced Distance Vector), and B.A.T.M.A.N. OLSR runs periodic hello and topology control messages and uses multipoint relays to reduce flooding. DSDV is a distance-vector protocol with sequence numbers to prevent loops. B.A.T.M.A.N. is also categorized as proactive but it actually uses a different philosophy: it distributes link quality information and only stores next-hop choices, reducing the need for global knowledge.

- *Reactive protocols* discover routes on demand, so they avoid periodic control traffic at rest but incur discovery latency when communication starts. In highly dynamic scenarios, frequent discoveries can themselves overwhelm the network. Typical reactive protocols are DSR and AODV.
- *Hybrid protocols* try to get the best of both worlds by making route discovery proactive inside a local zone and reactive beyond it. This reduces discovery delays locally while limiting global overhead. ZRP and TORA are classic hybrids, but hybrid protocols can be hard to implement and tune in practice.
- *Geographic protocols* take a different approach: they route using node positions rather than full route tables. If nodes know their positions and the destination’s approximate location, a node can forward to a neighbor closer to the destination. Drones typically have GPS and position information, and it naturally adapts to mobility. However, it requires location knowledge and suffers around voids or obstacles in the network where greedy forwarding fails; fallback strategies are needed.

B.A.T.M.A.N. B.A.T.M.A.N., short for *Better Approach To Mobile Ad-hoc Networking*, is a lightweight, decentralized mesh routing protocol. Unlike classical link-state protocols that try to build a global view, B.A.T.M.A.N. keeps things local: each node learns only which neighbor is the best next hop for reaching a certain originator. That simple decision reduces the amount of global state each node keeps.

1. Each node periodically broadcasts an *Originator Message* or OGM that identifies itself. Neighbors forward OGMs so they propagate through the network.
2. OGMs contain a sequence number so that nodes can determine which OGMs are fresh and avoid routing loops or replayed old information.
3. Nodes evaluate how often they receive OGMs from a neighbor as a proxy for link quality: if you receive OGMs frequently and with low loss, that neighbor likely offers a good path to that originator.
4. Based on these observations, each node records the best next hop toward each originator in a small table.

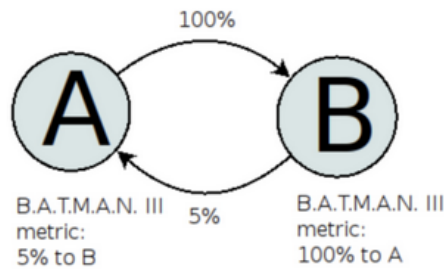


Figure 4.3: Asymmetric link.

Let's break down the contents of a typical OGM packet: The originator address is simply the identifier of the node that sent the OGM; the sequence number timestamps and orders OGMs to avoid mistaken use of old reports; the Time-to-Live (TTL) limits hops so OGMs don't propagate indefinitely; the Link Quality indicator (LQ) may be explicit or derived from how many OGMs are received and how recent they are, it gives a measure of the stability and reliability of the link between the neighbor and the originator; the hop count is useful for path length estimation and sometimes used for route selection.

Asymmetric links Imagine two drones, A and B (see Figure 4.3). A shouts its presence by sending periodic Originator Messages (OGMs). B hears A clearly and receives every OGM. But when B talks, A receives almost nothing. The radio behavior is asymmetric: packets reliably flow from A to B, but not from B to A.

This asymmetry is a real headache for routing algorithms that infer link quality by counting received control packets. If you simply count how many OGMs you get from a neighbor, B will conclude the link from $A \rightarrow B$ is near perfect (it receives lots of OGMs). But that measurement says nothing about the opposite direction, $B \rightarrow A$. If B uses only that count to advertise "I have a great link to A", it's lying — from A's point of view, the link is poor. So the system needs a way to detect and correct for asymmetry: not just "how many OGMs do I receive from you?" but also "how many of my OGMs do you forward (or otherwise reflect) back to me?" *B.A.T.M.A.N. IV* solves this using two related but distinct counters: *Receive Quality* and *Echo Quality*, and combines them into a *Transmit Quality* metric.

- If node A looks at how many OGMs from B arrived in the last N seconds, that count (often normalized into a 0–1 fraction) is A's estimate of $B \rightarrow A$ link reliability (see Figure 4.4).

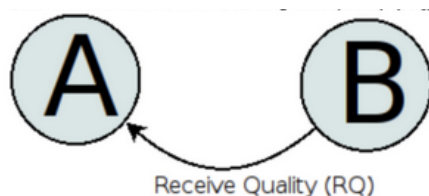


Figure 4.4: Receive quality.

We often write this as:

$$RQ = \text{number of OGMs received from neighbor (within sliding window)}$$

So if B sent 100 OGMs in the window and A received 90, A's RQ for B is 0.90. That tells A how good the neighbor $\rightarrow$ me direction has been, but it doesn't tell A whether its own transmissions reach the neighbor.

- *Echo Quality* is the clever bit that helps detect asymmetry. The intuition is simple: if I broadcast OGMs, do my neighbors rebroadcast them in such a way that I can observe those rebroadcasts? If yes, that indicates the neighbor actually heard my OGMs and is willing (and able) to forward them — i.e., my transmissions reach them.

Concretely, EQ can be implemented by counting how many times the neighbor rebroadcasts OGMs that originated from you. You detect that because OGMs include the originator address

and sequence numbers. When you later hear a neighbor forwarding an OGM that lists you as originator, that's an "echo" of your own packet (see Figure 4.5).

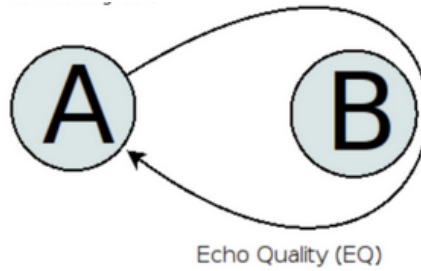


Figure 4.5: Echo quality.

So EQ is effectively:

$$EQ = \text{number of times neighbor rebroadcasts your OGMs (within sliding window)}$$

If many of your OGMs are echoed back by the neighbor, your transmission toward that neighbor is likely successful.

- To produce a usable bidirectional link estimate, B.A.T.M.A.N. IV computes *Transmit Quality* as the ratio of echo counts to receive counts. One practical formula is:

$$TQ = EQ/RQ$$

RQ tells you how many OGMs you heard from the neighbor (their transmissions toward you). EQ tells you how many of your OGMs the neighbor admitted to rebroadcasting (an indirect signal that your transmissions reached them). If RQ is large but EQ is small, you are in the asymmetric case where you receive their OGMs but they don't reliably receive yours — TQ becomes small and that link is demoted. If both EQ and RQ are high, TQ approaches 1 and the link is symmetric and good.

Suppose in the last sliding window:

- Node B counted $RQ_B(A) = 80$ OGMs received from A (out of an expected 100), so $RQ \approx 0.80$.
- Node B also observed that A's OGMs were rebroadcast by B (and heard back) only 40 times, so $EQ \approx 0.40$.

Then B would compute $TQ \approx EQ/RQ = 0.40 / 0.80 = 0.5$. That TQ of 0.5 tells B that while it seems to hear A often, A likely doesn't hear B reliably — the effective two-way usability of the link is moderate at best.

B.A.T.M.A.N. also uses a clever propagation mechanism so that nodes can estimate the end-to-end transmit quality to an originator through multiple hops. The idea is multiplicative: when an OGM is forwarded, each forwarding node multiplies the incoming TQ by its own local link quality and writes the result into the packet before rebroadcast.

Start by having the originator broadcast with a maximum TQ field (think of this as 1.0). A direct neighbor that receives it measures its local TQ to the originator (based on EQ/RQ or a local LQ estimate) and multiplies:

$$TQ_outgoing = TQ_incoming \times TQ_local$$

So if node A sets $TQ_incoming = 1.0$ when sending, node B receives and computes $TQ_local(B \longleftrightarrow A) = 0.8$; B rebroadcasts the OGM carrying $TQ = 1.0 \times 0.8 = 0.8$. Node C receives that rebroadcast, measures its own $TQ_local(C \longleftrightarrow B) = 0.9$, and rebroadcasts with $TQ = 0.8 \times 0.9 = 0.72$. The TQ that C carries for originator A (0.72) now reflects the cumulative product of local link qualities across the path $A \rightarrow B \rightarrow C$. Any node seeing an OGM learns not only that A exists, but also a scalar estimate of how well A's transmissions are likely to arrive at that node (see Figure 4.6).

This multiplicative scheme has a strong intuitive property: long weak chains produce small TQ numbers, and short strong links produce larger TQs. It gives nodes a compact, end-to-end metric without requiring full path knowledge.

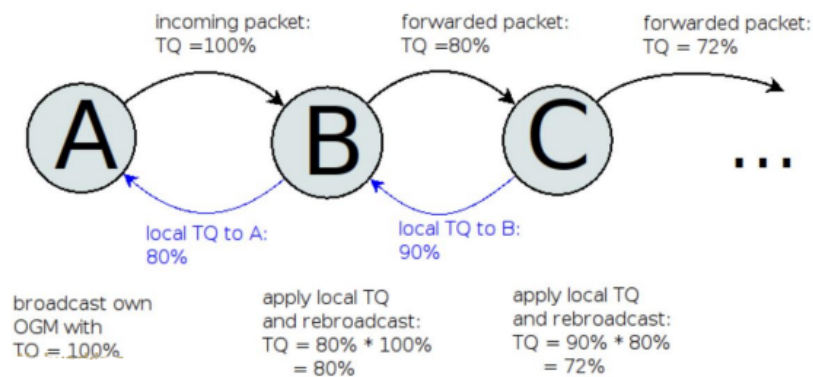


Figure 4.6: Transmit quality propagation.

Geographic routing Every drone knows its own position — usually via GPS — and periodically exchanges position beacons with neighbors. When a node has a packet to forward to a destination, it looks up the destination’s coordinates and forwards the packet to the neighbor whose position is closest to the destination. That greedy step repeats until the packet reaches the destination.

Geographic routing has two big advantages for dronets. First, it uses only local information (your neighbors' positions) so it scales well and adapts to mobility. Second, it avoids route discovery overhead: there is no table of whole-path routes to maintain.

Drones operate in three dimensions, so the routing decision should use 3-D Euclidean distance. *GDSTR-3D* (Geographic Distance-based STRategy in 3D) picks the neighbor with minimal 3-D distance to the destination. This approach is simple, but it can fail. A packet can arrive at a node that has no neighbor closer to the destination than itself. That node is in a *dead end* or local minimum (see Figure 3.18). In 2-D sensor networks researchers devised techniques to route around obstacles by walking the edges of a planar subgraph. With drones, the situation is easier because drones can move. If a node is at a dead end, you might reposition one or more drones for a few seconds to break the dead end.

In geographic routing there are three simple strategy to handle the forwarding decisions:

- *Greedy forwarding.* When a node must forward a packet, it hands the packet to the neighbor whose position is closest (in Euclidean distance) to the destination (see Figure 4.7). Then that neighbor repeats the rule. It's cheap, needs no route discovery and stores no big tables, but it suffers from the local-optimum (dead-end) problem: you can get stuck at a node that is the closest so far yet has no neighbor closer to the destination.

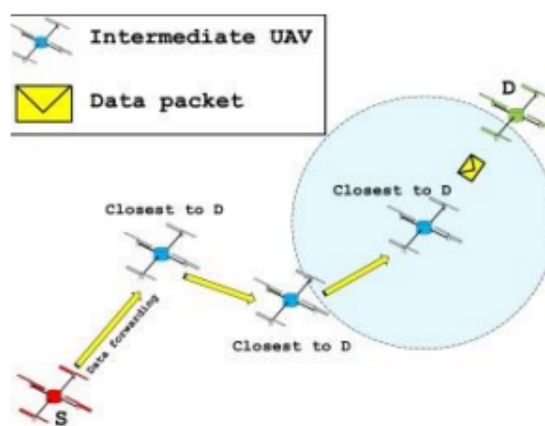


Figure 4.7: Greedy forwarding.

- *Store-carry-and-forward.* If connectivity is not continuous, a node simply keeps the packet, moves (or continues its mission), and forwards the data later when it meets a suitable relay or gets within range of the depot (see Figure 4.8). This is the delay-tolerant networking mindset and it works well when latency requirements are loose and mobility is predictable or favorable.

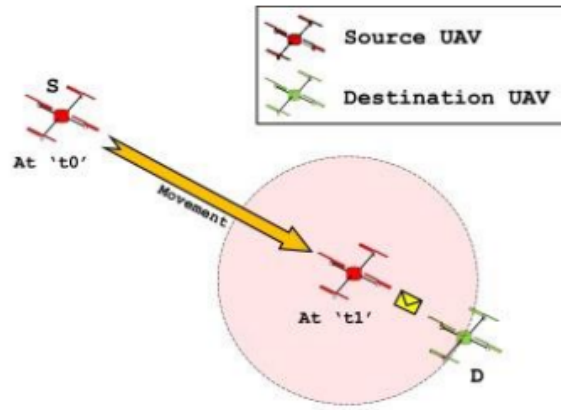


Figure 4.8: Store-carry and forward.

- *Prediction.* Here you use position plus heading and speed to forecast future locations of neighbors (see Figure 4.9). Forwarding decisions favor nodes that not only are close now but are likely to remain useful (or become even better) shortly. Prediction reduces pointless handovers and helps avoid fleeting contacts that won't actually progress the packet.

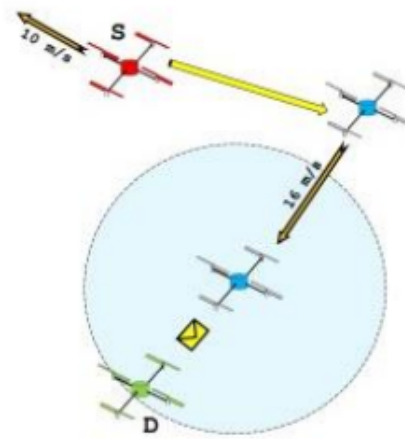


Figure 4.9: Prediction forwarding.

4.2.4 How to perform periodic data offloading when UAVs are deployed in large and harsh areas?

You have a squad of UAVs performing sensing over a large area and a depot where collected data must be offloaded. Offloading uses a short-range, high-data-rate channel like Wi-Fi, so UAVs must get physically near the depot (or a connected UAV) to dump large payloads. Each UAV has localization (GPS), and the area may be too large for constant connectivity.

The design goal is to periodically build a connected topology so data can be offloaded while also: minimizing the extra distance UAVs must travel to form that topology, minimizing the time needed to form the connected topology, and maximizing the time UAVs spend on their sensing mission (see Figure 4.10).

Stop & Route

The *Stop & Route* solution (Novella Bartolini et al., WONS 2023) tackles the periodic offloading problem with a distributed protocol that avoids global planning. The idea is simple and pragmatic: discretize the area, have UAVs exchange lightweight heartbeats and hello beacons, and use a Follow-Leader mechanism so that a connected chain of UAVs forms on demand between remote nodes and



Figure 4.10: Periodic connectivity problem.

the depot. Some UAVs become stationary relays (they STOP) while others follow leaders to fill connectivity gaps.

Important assumptions for the distributed version (see Figure 4.11): the operational area is discretized into tiles or rendezvous points, high-data-rate offloading uses short-range Wi-Fi only, UAVs are lazy-synchronized, the depot periodically transmits heartbeat packets so nearby UAVs, UAVs periodically send hello messages to advertise position and state, and a Follow-Leader mechanism lets UAVs form connected chains by following a closer-to-depot UAV that acts as a leader.

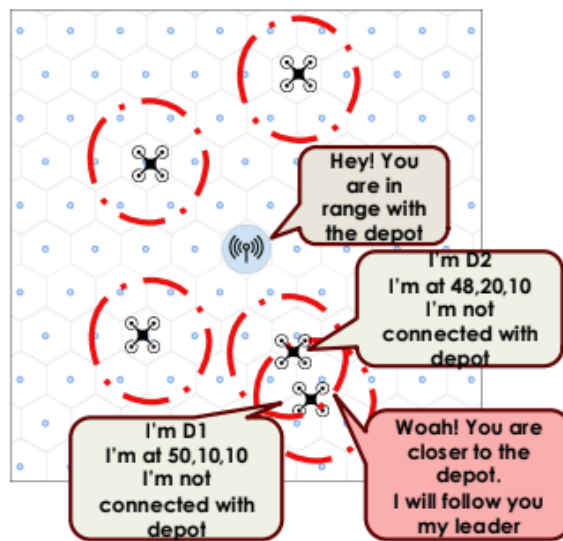


Figure 4.11: Stop & route distributed solution.

This lightweight signaling is enough for UAVs to locally decide whether to stop and act as part of the offload chain, follow another UAV, or keep sweeping their sensing area.

DGA is the core local rule set used by Stop & Route during the “connected formation building” phase. Each UAV repeatedly evaluates three conditions (see Figure 4.12):

- If it is in direct communication range of the depot, or if it is connected to another UAV that itself can reach the depot, then it stops — it will act as a stationary relay or offload node. Stopping means holding position on a tile and serving as part of the connected topology.
- If it sees a non-connected UAV that is closer to the depot than itself, it adopts a follow-leader role: it sets that neighbor as its leader and moves to follow it, effectively marching the chain toward the depot.
- If it is isolated — no connected neighbor and no closer node in range — it keeps moving along its mission toward the rendezvous point that is the next tile closer to the depot.

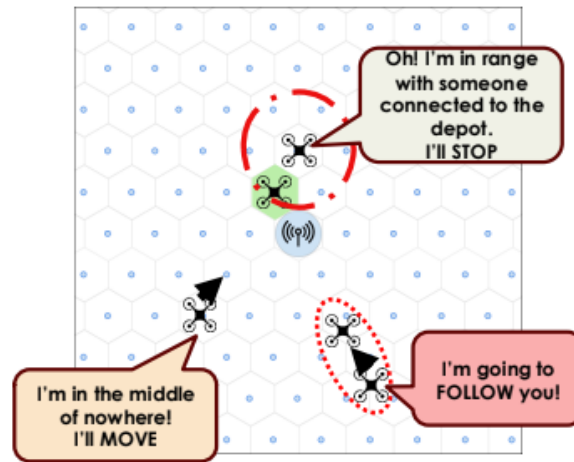
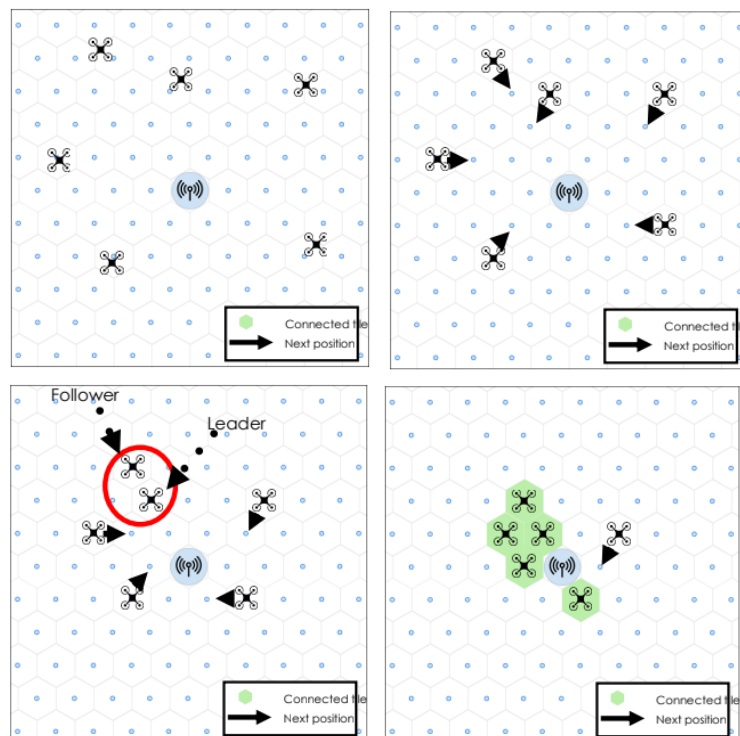


Figure 4.12: Stop & route distributed gathering algorithm (DGA).

Example 4.2.1 – Stop & Route – DGA

Think of the area as a grid of tiles. The depot sits at some tile near one edge. Initially most UAVs keep moving along their own sensing trajectories. When a UAV (call it D1) comes into Wi-Fi range of the depot, it stops and becomes a connected tile. Nearby UAVs that hear D1 and are not yet connected check whether they are closer to the depot than the UAVs they hear; if so one becomes a follower and moves to the tile right behind D1, then stops itself, creating a two-hop chain. Meanwhile other UAVs that are farther keep advancing toward the next rendezvous tile. Step by step the chain grows, but only as large as necessary to bridge the gap.



Stop & Route's strength is its low-communication, fully distributed nature and the fact that it minimizes unnecessary movement by only having the minimal set of UAVs stop and form the chain.

Chapter 5

IoT & Backscattering

In this chapter we begin with the Internet of Things — what it really means, what “things” are, and why this topic matters now. Then we move to a focused and increasingly practical question: can smart objects work without batteries? The answer opens a fascinating area: battery-free IoT and backscattering techniques.

5.1 What is the Internet of Things (IoT)?

When people say “*Internet of Things*” they usually mean one or more of three related things.

- First, it can mean the global network formed when everyday objects become able to sense, compute, and communicate — not just phones and laptops, but tiny devices embedded in the physical world.
- Second, IoT refers to the collection of technologies that make that interconnection possible: addressing and naming schemes, wireless access technologies, low-power electronics, data formats and management systems, and so on.
- Third, IoT describes the set of applications and services that run on top of those networks and technologies — for example, smart home energy management, fleet tracking, or automated inventory in a warehouse.

Think of IoT as both an infrastructure idea (lots of tiny devices connected) and an ecosystem (technology stacks plus services and business models built on top).

When we ask “what kind of things,” the straightforward answer is: everything that becomes “smart.” A smart object is an ordinary physical thing that has electronics embedded so it can compute and exchange information. Common examples are obvious: smartwatches, phones, TVs. But the IoT idea reaches far beyond consumer gadgets. When cars, buildings, streetlights, or entire city systems can sense and communicate, they become part of the IoT.

A helpful way to think about IoT is to ask whether a smart object can do three things.

1. It must be *identifiable*. If anything in the environment can be addressed, discovered, and tied to an identity, then systems and services can reason about it.
2. It has to *communicate*. That means at least minimal networking capabilities — the ability to receive messages or commands and to send responses.
3. Third, it must *interact*. Interaction can be between devices — building ad hoc, local networks — or between a device and a human.

A single smart object usually combines five basic aspects.

1. It has a physical body: size, shape, casing, connectors, and placement matter a lot because they affect the environment the device senses and how it communicates.
2. It offers basic communication functionality: discovery, accepting messages, and replying.

3. It has at least one name and one address so that other entities can refer to it and route messages to it.
4. It has some compute ability; even very limited processors can match incoming messages to fingerprints, apply simple filters, or run control logic.
5. It has means to observe the physical world (sensors for temperature, light, motion) or to act on it (actuators that turn a relay on, move a motor, or switch a valve). That sensing/acting capability is what differentiates smart objects from routers or servers in the classical Internet.

Many devices in IoT come from two well-established areas: RFID and wireless sensor networks (WSNs). RFID focuses on identification and very low-power tags, while WSNs focus on distributed sensing and multi-hop communication. IoT includes both but also devices that may have only the most basic communication and compute stacks — sometimes not a full TCP/IP stack. The unifying view of IoT is about objects acting as providers or consumers of data that describe the physical world.

Putting millions — or billions — of small, smart objects into the world creates a long list of system requirements. Let me walk you through the most important ones.

- *Devices heterogeneity*: devices differ widely in CPU power, memory, radio capabilities, and energy sources. The network and protocols must accommodate that diversity gracefully.
- *Scalability*: naming, addressing, routing, data management and service discovery must scale to huge numbers of devices distributed over large physical areas.
- *Ubiquitous local data exchange*: many use cases need local wireless proximity networking (for example, a smart meter talking to a gateway), and that requires efficient use of radio spectrum and reliable short-range communication.
- *Energy optimization*: many devices run on small batteries or on harvested energy. Energy is often the most scarce resource, motivating careful duty cycling, low-power radios, and sometimes entirely battery-free designs.
- *Localization and tracking*: numerous applications require knowing where a device is — asset tracking, logistics, and animal tagging among them — so position estimation and tracking methods are necessary.
- *Self-organization*: devices may come and go, so networks must form ad hoc, adapt to topology changes, and recover from partitions with minimal central control.
- *Semantic interoperability and data management*: the data coming from millions of heterogeneous sensors must be interpretable. That means common formats, metadata, and possibly ontologies to make data shareable and meaningful across applications.
- *Security and privacy*: without strong, built-in mechanisms for authentication, confidentiality, and data protection, users and organizations won't accept IoT systems.

To turn the conceptual pillars into reality, devices need a set of enabling capabilities: identification, communication, computation, and direct interaction with the physical environment. Two of the key building technology families for these are wireless sensor networks and RFID. WSNs bring networking theory, routing, and energy-aware MAC protocols; RFID contributes ultra-low-power identification and the idea of passive tags.

5.2 Battery-free smart objects

Can we redesign smart objects so they work without batteries? The answer is yes — at least for a class of low-power devices — and the enabling trick is called backscattering.

Backscattering is a simple physical idea: instead of generating a radio signal actively, a device modifies how much of an incoming radio wave it reflects (see Figure 5.1). By toggling that reflection in controlled ways, the device encodes information into the reflected wave. Since the device does not need to create its own RF carrier, the energy cost can be orders of magnitude lower.

Backscatter systems can sometimes harvest the energy they need from the illuminating radio waves themselves, which eliminates the need for an on-board battery. That's why they're often called battery-free or passive devices.

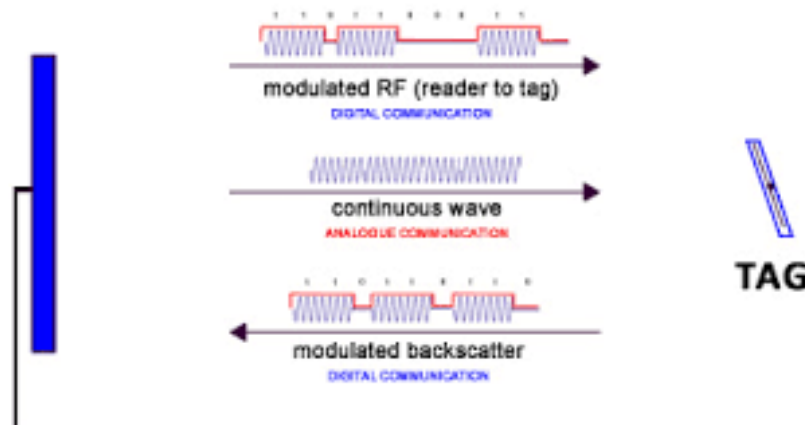


Figure 5.1: Backscattering.

Consider a radio reader or a Wi-Fi transmitter shining electromagnetic energy across a space. A passive tag sits in that field and uses an antenna and a tiny rectifier to pull out a little bit of energy. The tag doesn't amplify and transmit; it instead changes its antenna impedance to reflect more or less of the incoming wave. That controlled change in reflection appears as variations in the signal the reader receives. The reader senses those variations and demodulates the data the tag encoded.

Two common backscattering flavors are *ambient backscatter* and *RFID backscatter*.

- Ambient backscatter devices don't rely on a dedicated reader. Instead, they harvest energy from existing RF transmissions in the environment — TV broadcasts, cellular towers, or nearby Wi-Fi access points. Since those signals are already present, an ambient backscatter tag can reflect them to send information without requiring any new infrastructure.

Ambient backscattering tends to have very low data rates, typically below 1 kilobit per second. This makes it suitable for applications that only need to send tiny bursts of information occasionally — for instance, a passive sensor that reports whether an item is present or not. It's not suitable for real-time streaming or frequent updates.

Another limitation is signal availability. Even though some signals (like TV towers) broadcast continuously, you can't assume uniform coverage everywhere. In practice, signals can be weak indoors and may drop off sharply with distance or be blocked by buildings. If the ambient signal is too weak, the device can't harvest enough energy to power even its minimal electronics.

- In RFID systems the reader is part of the design. The reader emits a continuous RF signal (the carrier), and passive RFID tags harvest energy from that signal to run a tiny circuit. To communicate, the tag modulates the reflection of the reader's signal. Because the reader is present and controllable, RFID systems typically have better performance and predictable availability compared to ambient backscatter.

This predictability is the main advantage: when a reader is present, it supplies both the energy and the receiver that decodes the tag's reflection. That enables longer ranges, better data reliability, and higher effective throughput in many deployments.

RFID systems are heavily used in logistics, retail, and access control because of this dependable reader-tag model. RFID tags originally were used primarily for identification: a tiny ID number the reader interrogated. But the technology scales: you can add sensors to tags so they measure motion (accelerometers), occupancy or motion (PIR sensors), or even small cameras in very constrained setups. These are called sensor-augmented RFID tags.

The idea is powerful: combine passive energy harvesting with embedded sensing so a tag can both say "I am item X" and also report a sensed value like temperature or whether a package has been opened. Because the tag can harvest energy from the reader, it can sometimes power these sensors for short periods.

However, these tags face fairly strict constraints. They have limited power because they only harvest what the reader transmits and what the antenna/rectifier can capture. Their operating range is short compared to active radios — the reader typically has to be close. The time they can operate continuously is limited (they may need to accumulate charge between transmissions), and their

available storage is small so they can't buffer much data. Data rates tend to be low compared with smartphones or Wi-Fi devices. Yet, despite these limits, they can run small sensors and provide valuable sensing data at very low cost and with near-zero maintenance (no battery to replace).

The research group built real, playable devices to prove the concept. One example is *SapyJoy*, a battery-free video game controller. The team mounted an analog joystick and two buttons onto a Moo tag and wired them through a small PCB. Because the Moo tag also contains an accelerometer, the controller supports richer, complex inputs. When the reader energizes the tag, the tag harvests power and quickly encodes joystick positions and button presses into backscattered payloads that the server translates into game inputs. Because the microcontroller on the tag can measure the analog joystick and encode that into the tag's payload, the controller can be used for real games though responsiveness depends on how frequently the reader polls tags and how many other devices share the channel.

A similar idea maps to a battery-free mouse: the x and y axes are read by the tag and translated into pointer motion on the host. This requires precise timing and small latencies so the cursor movement feels smooth.

The group also built a battery-free light switch by mounting a button onto a Moo tag. Pressing the tag button triggers an event when the reader energizes the tag; the server receives the sensor payload and toggles the target LED or light fixture. Multiple buttons can be exposed on a single tag to control different lights; the logical mapping between the tag and the actuator is implemented in the server so the tag remains simple.

These demos show that almost every small human interface can be reimaged as a passive RFID device, but they also expose scalability and timing limitations.

The tags in this work use *EPC Gen2* — the ubiquitous standard for passive UHF RFID — but with a twist. The standard defines memory banks primarily for storing tag IDs. In practical deployments a full ID field (commonly 96 bits) isn't always necessary, so researchers repurpose spare bits.

The particular experiment limited the data field such that one byte is reserved for a compact tag identifier while six bytes are used for sensor data (this payload includes the CRC bits used for error checking — in the team's case, four bits of CRC are included in the payload accounting). The reason for keeping the payload short is robustness: shorter payloads decrease packet error rates under the weak-signal, noisy conditions typical of backscatter links. The only device that would require more payload than this architecture supports is a camera; even small image data would have to be fragmented into many backscatter frames.

A few practical details worth understanding. The tiny tag still needs to compute a CRC (cyclic redundancy check) for its payload to help the reader detect bit errors in reception; CRC computations can be implemented in small, energy-efficient logic. Because backscatter links have constrained throughput and intermittent availability, the entire cycle of harvest → sense → compute → modulate → sleep must be designed to fit the tag's energy budget and the application's latency requirements.

In the experiments the team used a USRP (a software-defined radio) as the RFID reader. A USRP allows flexible control of the carrier and reception chain.

Two antenna ports per room (one for Tx and one for Rx) let the reader separate the forward carrier from the weak backscattered signal. On the signal-processing side, a matched filter was used in the *SapyJoy* demo. A matched filter is a classic detector that maximizes signal-to-noise ratio (SNR) when you know the waveform you expect; by correlating the incoming signal with the known pattern (or expected symbol shape), the receiver becomes more resilient to noise. In practice, matched filtering means the reader can better decode the tiny, reflected modulations coming from the tag and so reduce packet errors or improve range.

For interactive and environmental applications the reaction time is the key metric. Reaction time is defined as the time interval between the generation of a new sensor reading and the time an application reacts to it. For the joystick, reaction time measures the time from when the user moves or presses a control to when the game reacts. For a presence sensor connected to a camera, reaction time measures the time between the physical event (someone entering a room) and a camera starting to record or an alarm being raised.

Testing one device at a time hides the biggest challenge: simultaneous operation. When multiple passive devices share a reader, they must be polled, scheduled, or otherwise coordinated to avoid collisions and to make sure each tag can harvest enough energy to operate.

The experiments used a time-division multiple access (TDMA) style approach: the reader cycles through timeslots, polling different tags in a repetitive frame. If a device is polled every slot, it gets low reaction time; if it is polled once every many slots, its reaction time increases. In a three-device



Figure 5.2: JoyPaper.

experiment (two environmental sensors and the joystick) the reaction time for the joystick increased from about 93 ms when alone to about 200 ms when sharing the channel. That doubling shows the impact of multiplexing: more devices implies longer wait times between opportunities to harvest and transmit.

With more devices (40 devices in simulation), naive equal allocation leads to unacceptable delays for interactive devices. That's where smarter MAC strategies are required.

The research identifies a clear open problem: equal allocation of channel resources doesn't satisfy the widely varying needs of heterogeneous devices. A joystick demands low latency under high activity, while a temperature sensor is content with sporadic updates. Static slotting is inefficient and leads to long delays for interactive devices as the number of devices grows.

Adaptive protocols like APT-MAC show a path forward, but the general problem remains: what is the best tradeoff between fairness, responsiveness, and energy distribution when devices cannot generate their own energy and must wait for reader energization? This question touches MAC design, power control, scheduler fairness, application priorities and even economic or user policy choices (who gets priority: safety sensors or entertainment devices?).

Imagine you could draw a button on a napkin, stick a tiny sticker-sized RFID chip on it, and turn that doodle into a real wireless input. *JoyPaper* does exactly that: it uses commercially available ultra-compact UHF RFID integrated circuits in sticker form, conductive-ink pens, and plain paper to make interactive, battery-free tags you can draw by hand (see Figure 5.2).

The basic materials are simple. You start with a small loop IC packaged as a sticker; these are the off-the-shelf RFID chips that normally sit on labels. Then you draw conductive traces on paper using a pen filled with conductive ink. The sticker is placed over the traces so that when the printed antenna path is complete, the loop IC has a functioning antenna and behaves like a normal passive RFID tag.

What's clever about JoyPaper is that the conductive traces can be designed so that touching or pressing a particular spot restores antenna continuity. That touch turns the tag into an input sensor. In other words, the act of pressing a paper button physically closes (or opens) a conductive path which switches the tag from "silent" to "active." When the tag is active and a reader is querying the environment, it harvests power from the reader's carrier and sends its ID. The reader — or a server attached to it — then interprets that ID as a button press or other input event.

5.3 Infrastructure-based wireless networks

This is the model you use every time when your phone connects to a cell tower (see Figure 5.3). You have base stations (GSM, UMTS, LTE, 5G) or Wi-Fi access points that are connected to a wired backbone. Mobile devices speak to base stations; the base stations forward traffic over the wired network; handovers let users move from one base station to another while maintaining connections. Routers, gateways and servers in the backbone carry and manage traffic, authentication, billing, and

other administrative tasks.

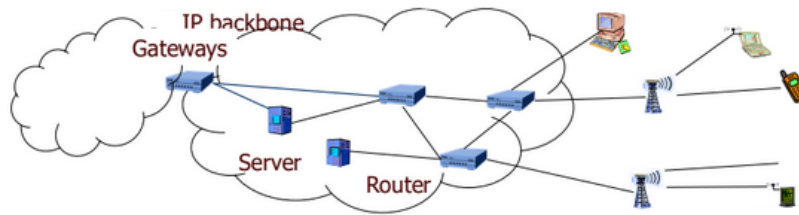


Figure 5.3: Infrastructure-based Wireless Networks.

There are many situations where the necessary infrastructure is missing or impractical. Maybe you're in a disaster zone where the towers and wired links are down. Maybe you're on a remote construction site or in a rural area where no operator provides coverage. Sometimes deployment time is tight — in military or emergency operations you can't wait to build a fixed network. Even a house or a temporary event might not be connected to a backbone in the way you want. Those scenarios motivate infrastructure-less networking.

5.4 Infrastructure-less networks and MANETs

If you can't rely on base stations or backbone routers, you build networks with the devices you have. That's the idea behind ad hoc networks and, in particular, *Mobile Ad Hoc Networks (MANETs)*. In a MANET each node — whether it's a laptop, a phone, a vehicle or an embedded device — participates in building the network. Nodes forward packets for others, discover neighbors, and coordinate access to the shared wireless medium all without a central controller.

Picture a set of laptops in a conference room sharing slides directly with each other—no Wi-Fi access point necessary. Or imagine first responders at a disaster site all wearing devices that must communicate even though the usual infrastructure is offline. Car-to-car communication for cooperative driving is another natural fit: vehicles exchange information directly about speed or hazards. A search-and-rescue team in an avalanche can use an ad hoc network to find victims and coordinate routes without relying on a distant server. Even everyday services — finding empty parking spaces by local broadcast, or a soldier platoon coordinating movement — can be enabled by MANETs. Smaller scale personal area networks (smartwatch, glasses, medical sensors) are another form of infrastructureless networking, often with tight energy and latency constraints.

Building a functioning ad hoc network requires solving a cluster of problems that are mostly hidden by infrastructure in cellular or Wi-Fi networks. The main challenges are the absence of a central organizing entity, limited radio range, node mobility, and constrained energy supplies.

Nodes have to discover each other, decide who should transmit when, and figure out paths to forward traffic. All of that happens in a distributed, self-organizing way. There's no base station to assign resources or to route packets, so protocols must do neighbor discovery, medium access control (MAC), and routing cooperatively.

MAC in infrastructure networks is often simple because the base station can schedule or coordinate transmissions. In ad hoc networks everyone shares the medium. The MAC must therefore solve classic distributed problems like avoiding collisions, dealing with the hidden terminal problem, and allowing fair or prioritized access. Common approaches include CSMA/CA (listen before talk), distributed TDMA variants, or hybrid approaches, but each has tradeoffs in overhead, energy use, and fairness.

Because radio range is short and obstacles can block signals, direct communication to a distant node is often impossible. The solution is multi-hop routing: nodes forward packets through intermediate neighbors. But routing without infrastructure is tricky. As nodes move, routes break and must be discovered or repaired. This requires constant or on-demand control traffic, which consumes bandwidth and energy. Scalability is also an issue: route maintenance becomes more burdensome as networks grow.

Mobility changes the neighborhood graph continuously, so routes and local state must be updated fast enough to remain useful. Some routing schemes proactively maintain routes, costing continuous overhead, while others discover routes on demand, which introduces delay when a path is first needed. Balancing responsiveness and control overhead is central to MANET protocol design.

Multi-hop routing can sometimes save energy because transmitting over many short hops can be cheaper (in energy per bit) than a single long transmission that requires many more watts. But that's not always true: relaying packets costs energy for the forwarder, and nodes that forward a lot may die early, hurting overall network lifetime. Designing energy-aware routing means balancing per-link energy costs, fairness, and the network's longevity.

Multi-hop networks also complicate latency and reliability. Each hop introduces delay and a chance of packet loss, so protocols must mitigate errors (retransmissions, forward error correction) and manage queuing at intermediate nodes.

Most devices in ad hoc networks are battery powered. For many applications, maximizing the lifetime of an individual device and the network as a whole is a primary design goal. That leads to energy-efficient strategies: duty cycling radios so they sleep most of the time, choosing routes that minimize energy per delivered bit, using wake-on-event sensors, or employing cooperative techniques where nodes alternate forwarding responsibilities to avoid draining any single node.

But energy optimization is a multi-objective problem. You might optimize for latency, or reliability, or fairness, and those goals often conflict with minimizing energy consumption. Designers must choose what to prioritize: for example, in a life-saving application you accept higher energy use for lower latency; in a long-term environmental monitoring deployment you favor energy savings at the cost of higher delay.

Despite intense research into ad hoc networking for decades, a single, mass market "killer app" that requires pure infrastructureless, large-scale MANETs has been elusive. Many specialized applications succeeded or found niches, like tactical military networks or sensor networks, but the broad consumer market adopted infrastructureed solutions because they are easier to monetize and control.

Sensor networks — a close relative of ad hoc networks — achieved greater practical success because their topology and traffic patterns are usually more predictable, they focus on energy efficiency and multi-hop strategies optimized for many low-rate sensors, and the application domains (environmental monitoring, industrial sensing) map well to their constraints.

Chapter 6

Introduction to Reinforcement Learning

When I say “autonomous network,” I don’t just mean “it runs without me pushing buttons.” I mean a communication system that behaves like an independent organism: it senses what’s happening, it reasons about what to do, it acts, and it improves over time.

An autonomous network requires **intelligence**: the ability to learn from the local environment and to change behavior when conditions change. That could be a squad of drones routing messages among themselves, or a home full of tiny devices that wake up and speak only when they have something new to say.

We’re going to focus on autonomy at the networking level — that is, how nodes communicate and route information. The central question is: how do we make a distributed system of devices behave intelligently without constant human configuration?

Autonomy in networking shows up in many wireless domains. Think about sensor networks and IoT, backscatter/RFID systems, and drone networks. These examples share a few traits: many small devices, limited energy budgets, event-driven data, and a wireless medium that is noisy and shared. Each environment raises specific constraints and opportunities for autonomy.

A particularly revealing example is the battery-free smart home. That setting is useful because it stretches a designer’s constraints: devices cannot carry batteries, so they must harvest energy; sensors are heterogeneous (cameras, temperature probes, presence detectors, remotes, switches); and data arrival is highly irregular — sometimes bursty, sometimes sparse, sometimes continuous. The peculiarities of battery-free devices change how we design the network:

- First, devices produce data with very different timing characteristics. A joystick or remote produces bursts only when a user interacts. A temperature sensor usually updates slowly, but during a fire it will produce urgent, rapid updates. Presence sensors are silent until someone walks by. A camera, by contrast, produces continuous high data volume.
- Second, these devices cannot transmit whenever they like. In backscatter systems, a reader must query a tag to provide the energy and the “opportunity” to transmit. That means the reader controls when devices can speak.
- Third, we’d like two things at once: ask a tag only when it actually has new data (avoid wasted queries), and make sure we don’t miss data that was produced between queries (avoid data loss).

A natural approach is TDMA: give each tag a fixed timeslot and cycle through them. Another is random access (e.g., ALOHA-style), where tags reply opportunistically. Both have problems in battery-free backscatter environments. With random access, simultaneous replies collide because many tags share the same short window to backscatter. Collisions mean wasted energy and lost packets — bad when the reader’s energy budget matters. With TDMA fixed assignment, a tag may be polled repeatedly even when it has nothing new to report. That wastes the reader’s time and the tiny energy budget available to tags (they must harvest enough energy during that timeslot to sense and reply).

Worse, if a device produces new data twice between its assigned slots, the older sample might get overwritten or lost before it's ever reported.

Our aim is a MAC protocol that adapts autonomously with two practical goals:

1. Query devices only when they are likely to have new data, minimizing wasted queries.
2. Avoid losing data due to infrequent polling, so fresh samples are delivered within acceptable delay.

So how can a single reader learn which devices to poll and when? The answer in this context is to learn from experience: observe which polling choices lead to useful data and which don't, and adjust the policy accordingly. That learning-by-interaction approach is exactly what reinforcement learning (RL) provides.

Reinforcement learning is about learning what to do through trial and error. An RL agent doesn't get told the correct action; instead it takes actions and receives feedback in the form of rewards or penalties. Over time the agent learns which actions tend to lead to high cumulative reward.

Crucially, RL handles sequential decision making: your current action affects not only the immediate reward but the future state and future rewards.

Three core pieces make up an RL setup: *sensation* (what the agent observes about the world), *action* (what the agent can do), and *goal* (the reward it is trying to maximize).

There are a few things that separate RL from other machine learning approaches, and those differences make RL attractive here.

1. First, there is no supervisor giving the "right" action. You don't have labeled data saying "this tag should have been polled now." You only have a signal: the poll either returned fresh data or it didn't.
2. Second, feedback can be delayed. Choosing to poll tag A now might prevent polling tag B until later, causing B's next sample to be lost — a bad outcome that's only apparent later. RL handles these delayed consequences naturally because it optimizes for cumulative reward, not immediate gains.
3. Third, actions change the data stream the agent receives. If the reader polls a device frequently, it will see a steady stream of data from that device; if it ignores it, the device's samples may pile up or be dropped, changing future observations.
4. Finally, RL naturally supports online learning. We don't want to train the reader offline and freeze it: device populations and human habits change.

As an example, think of an infant waving its arms. There is no supervisor telling the baby what each motion will cause, but the baby quickly learns — some motions cause attention, others cause objects to fall. Through many small trials and the consequences that follow, the infant builds a sense of cause and effect. The reader similarly must "experiment." It will poll devices it suspects have new data, learn which polling patterns work, and gradually improve. Early on there will be mistakes: wasted polls and maybe missed samples. A robust RL formulation minimizes the long-term cost of those mistakes and keeps exploring enough to adapt to changes.

6.1 Applications of Reinforcement Learning

Reinforcement learning powers real systems you've heard of and many you haven't.

- Think about self-driving cars: beyond end-to-end perception, RL can be used to optimize trajectories, plan motions, choose dynamic lanes on a crowded highway, tune low-level controllers, or learn policies for specific highway scenarios where rules and interactions are complicated.
- In factories and industrial automation RL drives robots that do repetitive, heavy, or hazardous work more efficiently than people and sometimes in ways people can't (or shouldn't) do themselves.
- In healthcare, clinical decisions and treatments often have delayed outcomes: the effect of a medication may be visible only days or months later. RL can learn policies from past patient trajectories — without requiring a perfect mathematical model of human biology — and, when

carefully validated, propose treatment schedules that maximize long-term patient outcomes rather than immediate, short-term metrics.

- Facebook (now Meta) developed Horizon, an open-source RL platform, and has applied RL to personalize suggestions, choose which notifications matter, and even improve video streaming quality by learning policies at scale.
- News recommendation and ad bidding are natural RL playgrounds because user preferences change quickly; an RL system can adapt recommendations by observing how users return or engage after each suggestion.

And beyond these examples, RL plays roles in finance and trading, natural language processing (e.g., optimizing dialogue policies), gaming and simulation, robotics, real-time bidding, and, importantly for our course, computer networking: routers, schedulers, and MAC protocols can all learn better policies from interaction.

6.2 Rewards

At the heart of RL is the reward. The *reward* at a single time step, call it R_t , is just a scalar number the environment hands back to the agent after it takes an action. That number is the agent's only direct signal about how well it's doing at that moment. The agent's objective is to maximize cumulative reward over time, not just instant reward.

There's also the reward hypothesis: the informal claim that any goal can be represented as maximizing expected cumulative reward. That hypothesis is useful as a unifying idea — if you can design a reward function that captures what you want, RL will try to optimize it — but it's also a caution. Examples help make reward design concrete.

- Suppose you want a humanoid robot to walk forward. The reward design can be simple: give positive reward for forward displacement and a steep negative penalty if the robot falls. The positive reward encourages movement in the desired direction; the negative penalty prevents the agent from exploiting the positive reward by adopting unstable gaits that produce forward motion only briefly and then crash.
- Now imagine a helicopter learning stunt maneuvers. The helicopter should be rewarded for tracking a desired trajectory (following the planned path) and heavily penalized for crashing.
- Translate the same idea to communication in battery-free environments. A sensible reward might give a positive value when a query returns new sensor data and a negative value when a query finds nothing or when a sample is lost due to long polling intervals.

RL is a framework for sequential decision making. The action you pick now can influence what states and rewards you see later, sometimes dramatically. That's why RL optimizes expected cumulative reward rather than immediate payoff. Sometimes you should sacrifice a small immediate reward to unlock a large reward later.

6.3 The exploration–exploitation dilemma

A central conundrum in RL is the *exploration–exploitation* trade-off. To collect reward you want to pick actions that you already know to be good — *exploit*. But to discover possibly better actions you need to try things you haven't tried yet — *explore*. If you exploit too much you may get stuck in a locally optimal but globally subpar strategy. If you explore too much you waste opportunities that you could have turned into reward.

As a simple life example: your favorite restaurant sits around the corner. If you eat there every day you get consistent meals you like — low risk, reliable reward. But you'll never learn whether an even better place exists. If, instead, you try a new restaurant every lunch you might uncover better food, but you'll also suffer more bad meals along the way. The best long-term strategy typically mixes both behaviors: exploit often but keep trying new places occasionally to discover surprises.

In a stochastic environment, the same action can give different outcomes each time; to estimate an action's true value you must try it many times. That requirement is costly when trials are expensive

(physical robots with wear and tear), slow (medical trials), or risky (a policy that could cause a crash or safety violation).

The exploration–exploitation problem has been studied for decades and remains a central challenge because there is no single perfect strategy that works in every domain.

6.4 General RL framework

The general RL framework picture a simple loop: there’s an agent and there’s an environment (see Figure 6.1). At each step the agent chooses an action and the environment responds by handing back two things — an observation and a scalar reward. We usually write the action at time t as A_t , the observation as O_t , and the reward as R_t . Time progresses in discrete environment steps, and every step repeats that same handshake: agent acts, environment replies.

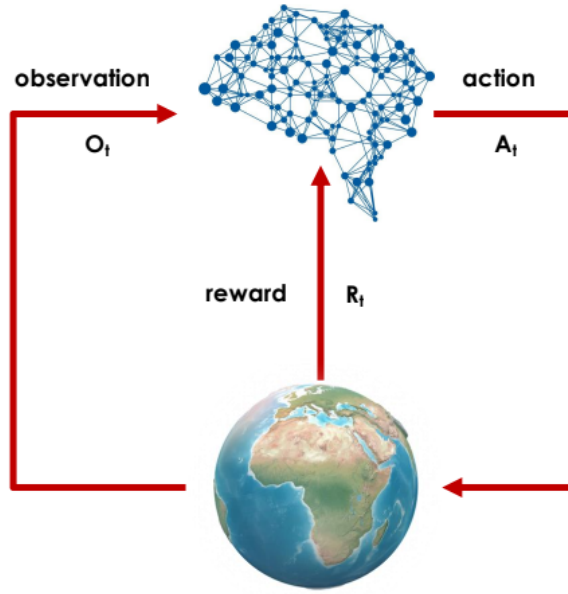


Figure 6.1: Agent and environment.

That handshake is the foundation of reinforcement learning. The agent never sees the “internal wiring” of the environment; it only sees the observations and rewards that the environment emits and then must act on those signals.

Everything the agent has ever observed, done, and received up to time t can be collected into a single object called the *history*. Formally the history at time t is

$$H_t = A_1, O_1, R_1, \dots, A_t, O_t, R_t,$$

which is simply the full stream of action taken, observation perceived, and reward obtained. That sequence is important because what happens next generally depends on the entire history. In practice, however, the agent doesn’t need to reason over the whole raw history every moment. Instead we define a *state* as any function of the history that contains the information needed to predict what comes next. More compactly,

$$S_t = f(H_t),$$

where f is some mapping that extracts the relevant information from the history. That mapping can be as simple as the last observation, a short window of recent observations, or a complicated belief distribution over hidden environment variables.

Two complementary notions of state matter. The environment has its own private state, call it S_t^e , which the environment uses internally to determine the next observation and the reward. The agent has an internal representation — its agent state, call it S_t^a — which is the information the agent actually uses when picking its next action. The environment’s state S_t^e is generally not directly visible to the agent; the agent must infer or represent what it thinks the environment state might be from

its history. The agent internal representation S_t^a is the information used by reinforcement learning algorithms. It can be any function of history:

$$S_t^a = f(H_t)$$

If the agent state equals the environment state, we say the problem is *fully observable*: whatever the environment uses to decide the next outcome is available to the agent. In that ideal world you can, in principle, make optimal decisions because you can see exactly what matters.

Reality is usually messier. In many practical tasks the agent only sees part of the environment. A robot with a camera doesn't receive the robot's absolute coordinates; a trading agent only sees current market prices and not hidden trader intentions; sensors in a house only report local measurements but not the whole home state. When the agent only has indirect information, the problem is *partially observable*. In these cases the agent must summarize the history into a representation (beliefs, recency statistics, or other features) that helps it to predict future observations and rewards.

An RL agent typically contains three conceptual components that work together.

1. *Policy*. The policy is the agent's behavior function: given the agent's internal state, the policy decides what action to take. It is the active part of the agent — it produces behavior.

When we say “policy,” we mean a mapping from the agent's state to an action. If the mapping is deterministic, the policy returns a single action for each state. If it's stochastic, the policy returns probabilities for each action — the agent then samples an action according to those probabilities.

The reward signal is the agent's only direct evaluation of behavior: when a chosen action is followed by low reward, the policy should update to favor different actions in similar states. The design of how the policy updates depends on the algorithm (policy gradients, Q-learning, actor-critic, etc.), but the conceptual link is the same: reward guides change.

2. *Value function*. Rewards tell you what's good immediately, but value predicts what's good in the long run. A value function answers the question “starting from this state, how much cumulative reward can I expect to get if I follow policy X?” That predicted future reward is what lets the agent prefer actions that may pay off later rather than only chasing immediate gains.

A value function is a prediction. For a policy π , the state-value function $v_\pi(s)$ is the expected return starting from state s and following π thereafter. If we use the discounted return

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots,$$

then the value is

$$v_\pi(s) = E_\pi[G_t | S_t = s].$$

Intuitively, $v_\pi(s)$ tells you how “promising” that state is in terms of future cumulative reward. Even a state with low immediate reward can have high value if it reliably leads to better states later. Value functions are central to many RL algorithms because they reduce long-term optimization to estimating expected sums of rewards.

3. *Model*. A model predicts consequences: given a state and an action, what next observation or next environment state is likely to follow, and what reward can we expect? Models let agents plan by simulating hypothetical futures.

A model predicts the environment's reaction to actions. It can give the resultant next state, the expected immediate reward, or both. Model-based agents can use such predictions to plan: they simulate sequences of actions ahead of time to choose those that look best, which can accelerate learning when models are accurate.

Model-free methods skip building an explicit model and instead directly learn policy or value estimates from transitions observed in the real environment.

Chapter 7

Bandit Algorithm

Imagine you're an agent that repeatedly chooses among several actions. After each choice you get a numeric reward that tells you how well that choice worked. The tricky part is that the reward you see depends on the action you chose. If you always pick the action that looks best now, you might miss an action that's actually better but hasn't been tried enough.

That tension — should I try something new to learn more about it (explore), or should I stick with what already seems best to get reward right now (exploit)? — is the exploration–exploitation trade-off.

7.1 K-armed bandit

Think of a row of k slot machines (k “arms”) sitting at a casino (see Figure 7.1). Each arm, when pulled, yields a reward drawn from an unknown but fixed probability distribution associated with that arm. The distributions are stationary (they don't change over time), but you don't know their means.

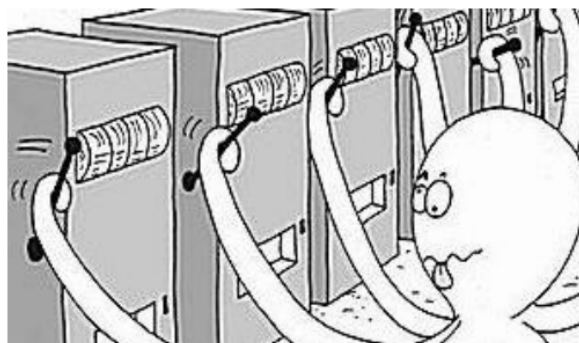


Figure 7.1: K-armed bandit.

At each time step t you choose one arm A_t . The environment returns a reward R_t drawn from that arm's distribution. Your objective is to maximize the total (or expected total) reward accumulated over a number of steps — for example over 1000 plays.

This problem isolates the exploration/exploitation question: you must decide which arms to sample to learn their quality, while trying to collect as much reward as possible.

To decide which action is best we need a way to rank actions. For each action a there is a true action-value $q_*(a)$: the expected reward when action a is selected. In symbols,

$$q_*(a) = E[R_t | A_t = a],$$

that is, the mean of the reward distribution for arm a . If you knew all $q_*(a)$, the best action is simply the one with the largest $q_*(a)$. But you don't know those means, so you must estimate them from samples.

7.2 Estimating action values

A simple and commonly used estimator for an action's value is the sample average of the rewards you got from that action so far.

Formally, define an indicator $1_{A_i=a}$ which equals 1 when the action at time i was a , and 0 otherwise. Then the sample-average estimate at time t is

$$Q_t(a) = \frac{\sum_{i=1}^{t-1} R_i 1_{A_i=a}}{\sum_{i=1}^{t-1} 1_{A_i=a}}.$$

This reads: add up all rewards received when you took action a prior to time t , and divide by the number of times you took a . If you have never taken action a yet (the denominator is zero), a common convention is to set $Q_t(a) = 0$ or some other default value.

Intuitively, by the law of large numbers, as the number of times you sample an action grows, its sample average $Q_t(a)$ will converge to the true mean $q_*(a)$, assuming stationarity.

A computationally convenient incremental form of the same update is useful in practice. If an action a has been selected n times so far and its current estimate is Q_n , then after seeing a new reward R for that action you can update:

$$Q_{n+1} = Q_n + \frac{1}{n}(R - Q_n).$$

This formula updates the average without keeping the whole history.

An arm's rewards could follow many distributions: Bernoulli (0/1 success), binomial, uniform, Gaussian, etc. Whatever the distribution, $q_*(a)$ is its mean. The particular distribution matters for the spread and variance of observed rewards, but the central object for decision-making is always the mean.

7.3 How to pick an action

Given current estimates $Q_t(a)$, how should the agent select an action at time t ? There are three simple, conceptually different approaches we'll discuss: random selection, greedy selection, and ϵ -greedy selection.

7.3.1 Random selection

Choose an action uniformly at random. This guarantees exploration but ignores your knowledge; it's usually not a good long-term strategy since it doesn't exploit promising actions.

7.3.2 Greedy selection

Pick the action with the highest estimated value:

$$A_t = \operatorname{argmax}_a Q_t(a).$$

Greedy selection exploits current knowledge fully: it always picks the currently best-estimated action. The trouble is it may get stuck on a suboptimal action because it never tries others once it thinks one is best.

When there is a tie for highest estimate, ties are typically broken arbitrarily (often uniformly at random among the maximizers). That means when all Q values are equal (e.g., at the very start), a "greedy" choice can look random because the maximizers include all actions.

t

7.3.3 ϵ -greedy selection

This simple method mixes greedy exploitation with random exploration. With probability $1 - \epsilon$ pick the greedy action (the action with the current largest Q_t); with probability ϵ pick an action uniformly at random among all k actions. This guarantees occasional exploration but usually exploits.

$$A_t = \begin{cases} \operatorname{argmax}_a Q_t(a) & 1 - \epsilon \\ a \sim \operatorname{Uniform}(\{a_1 \dots a_k\}) & \epsilon \end{cases}$$

As the number of steps grows and each action is sampled enough, the sample averages $Q_t(a)$ converge to $q_*(a)$ (for stationary problems). But choosing a sensible exploration policy while learning still matters a lot in finite time.

Exercise 7.3.1 – ϵ -greedy selection

In ϵ -greedy action selection, for the case of two actions and $\epsilon = 0.5$, what is the probability that the greedy action is selected?

With probability $1 - \epsilon$ we exploit, so the greedy action is chosen then. That is $1 - \epsilon = 1 - 0.5 = 0.5$.

With probability ϵ we explore, i.e., we choose uniformly among the two actions. Given exploration, the probability of selecting the greedy action is $\frac{1}{2}$.

So the probability of selecting the greedy action during exploration is $\epsilon \times \frac{1}{2} = 0.5 \times 0.5 = 0.25$.

Add exploitation and exploration contributions: $0.5 + 0.25 = 0.75$.

So the greedy action is selected with probability 0.75.

Exercise 7.3.2 – ϵ -greedy selection

Consider a k -armed bandit problem with $k = 4$ actions, denoted 1, 2, 3, and 4. Consider applying to this problem a bandit algorithm using ϵ -greedy action selection, sample-average action-value estimates, and initial estimates of $Q_1(a) = 0$, for all a . Suppose the initial sequence of actions and rewards is

$$\begin{array}{ll} A_1 = 1 & R_1 = 1 \\ A_2 = 2 & R_2 = 1 \\ A_3 = 2 & R_3 = 2 \\ A_4 = 2 & R_4 = 2 \\ A_5 = 3 & R_5 = 0 \end{array}$$

On some of these time steps the ϵ case may have occurred, causing an action to be selected at random. On which time steps did this definitely occur? On which time steps could this possibly have occurred?

- First, compute the Q estimates after each step. Start with all $Q = 0$.
- After $A_1 = 1, R_1 = 1$ action 1 has been taken once and yielded reward 1, so its sample average is $1/1 = 1.0$. Actions 2, 3, 4 are still 0. So Q becomes: $Q(1) = 1, Q(2) = 0, Q(3) = 0, Q(4) = 0$.
- After $A_2 = 2, R_2 = 1$ action 2 now has one sample of reward 1, so $Q(2) = 1/1 = 1.0$. The others remain as before: $Q(1) = 1, Q(2) = 1, Q(3) = 0, Q(4) = 0$.
- After $A_3 = 2, R_3 = 2$ action 2 now has two samples, rewards 1 and 2. So $Q(2) = \frac{1+2}{2} = \frac{3}{2} = 1.5$. Thus estimates: $Q(1) = 1.0, Q(2) = 1.5, Q(3) = 0, Q(4) = 0$.
- After $A_4 = 2, R_4 = 2$ action 2 now has three rewards: 1, 2, 2. So $Q(2) = \frac{5}{3} \approx 1.666$. Estimates: $Q(1) = 1.0, Q(2) = 1.666, Q(3) = 0, Q(4) = 0$.
- After $A_5 = 3, R_5 = 0$ action 3 now has one sample of 0. So $Q(3) = 0/1 = 0$. Final estimates after the five steps are: $Q(1) = 1.0, Q(2) = 1.666, Q(3) = 0, Q(4) = 0$.

Now reason about which steps must have been exploration. A time step must have been an ϵ -exploration step if the chosen action at that time was not a greedy action given the estimates immediately before that selection.

- Time step 1 ($A_1 = 1$): all Q values were equal (all zero). The greedy set is all actions (tie). Selecting action 1 could be the greedy choice (tie-break among all). So A_1 could be greedy or could be exploration — we cannot say for sure.
- Time step 2 ($A_2 = 2$): the estimates were $Q(1) = 1.0$ and $Q(2) = Q(3) = Q(4) = 0$. The greedy action was action 1 (unique maximizer). Yet the actual choice at time 2 was action 2. Choosing a non-greedy action when there is a unique greedy action can only happen during exploration. Therefore, step 2 definitely was an ϵ -exploration step.
- Time step 3 ($A_3 = 2$): estimates were $Q(1) = 1.0$, $Q(2) = 1.0$, $Q(3) = Q(4) = 0$. Now the greedy set contains actions 1 and 2 (a tie). Choosing action 2 could have been the exploitation choice (resolving the tie in favor of action 2), or it could have been exploration. So step 3 could have been exploration but does not have to be.
- Time step 4 ($A_4 = 2$): estimates were $Q(1) = 1.0$, $Q(2) = 1.5$, others 0. Here action 2 is uniquely greedy. Choosing action 2 could be exploitation or could still happen during exploration (exploration can happen and by chance pick the greedy action). So we cannot say step 4 was definitely exploration; it could have been exploration but need not be.
- Time step 5 ($A_5 = 3$): estimates were $Q(1) = 1.0$, $Q(2) = 1.666$, $Q(3) = 0$, $Q(4) = 0$. The unique greedy action was action 2, yet the action taken was action 3. That can only happen if the algorithm was exploring. So step 5 definitely was an ϵ -exploration step.

So the definite exploration steps are steps 2 and 5. Steps 1, 3, and 4 could have been exploration but might also have been greedy choices depending on tie-breaking or chance during exploration.

7.4 How to estimate action values

7.4.1 Incremental trick

Suppose you concentrate on one particular action and you want to track its value. Let R_i be the reward you get the i -th time you pick this action. Let Q_n be your estimate of that action's value right before the n -th time you pick it. (That means Q_n is based on the first $n - 1$ observed rewards for that action.)

The straightforward sample average after observing n rewards is

$$Q_{n+1} = \frac{1}{n} \sum_{i=1}^n R_i.$$

But you don't want to recompute the whole sum every time. There's an elegant incremental form that updates the average using just the previous estimate and the new reward. Start from the two averages:

- the old average: $Q_n = \frac{1}{n-1} \sum_{i=1}^{n-1} R_i$ (remember this is before seeing R_n),
- the new average after seeing R_n : $Q_{n+1} = \frac{1}{n} \sum_{i=1}^n R_i$.

Now write the new sum as the old sum plus the new reward:

$$\sum_{i=1}^n R_i = \sum_{i=1}^{n-1} R_i + R_n.$$

Divide both sides by n :

$$Q_{n+1} = \frac{1}{n} \left(\sum_{i=1}^{n-1} R_i + R_n \right).$$

Replace the old sum by $(n-1)Q_n$, because $\sum_{i=1}^{n-1} R_i = (n-1)Q_n$. That gives

$$Q_{n+1} = \frac{1}{n}((n-1)Q_n + R_n).$$

Distribute $1/n$ and regroup:

$$Q_{n+1} = \frac{n-1}{n}Q_n + \frac{1}{n}R_n = Q_n + \frac{1}{n}(R_n - Q_n).$$

This last form is the incremental update. It says: take the old estimate, add a fraction $1/n$ of the prediction error $R_n - Q_n$. Intuitively, the error tells you how surprised you were by the new reward and the fraction $1/n$ shrinks as you have more samples so newer observations change the average less.

Note that this formula is valid even for $n = 1$. If Q_1 (the estimate before the first selection) is defined as 0 or some initial value, then after seeing R_1 you get $Q_2 = Q_1 + (1/1)(R_1 - Q_1) = R_1$, which is exactly the average of the one observed reward.

Here's a simple recipe you can implement in code. It uses the incremental update above to maintain action-value estimates and ϵ -greedy selection to balance exploration and exploitation.

```

1 Initialize, for all action a = 1 to k:
2   Q[a] <- 0
3   N[a] <- 0
4
5 For each time step t = 1, 2, ...:
6   A <- with probability 1-ε:
7     choose action A_t = argmax_a Q[a]
8   with probability ε:
9     choose action A_t uniformly at random
10
11   R <- bandit(A)
12   N(A) <- N(A) + 1
13   Q(A) <- Q(A) + 1/N(A) * [R - Q(A)]

```

A few practical notes about this code. The choice of ϵ controls how often you force exploration; a small ϵ means mostly exploit. $N[a]$ is the count of times action a has been selected; that count defines the step size $1/n$ in the sample-average update. Ties in argmax are typically resolved uniformly, which can be important at the start when many Q 's are equal.

7.4.2 Stationary vs nonstationary problems

The sample-average method and the $1/n$ step size are ideal when the underlying reward distributions are stationary — that is, the arms' distributions do not change over time. Under stationarity, the sample average converges to the true mean by the law of large numbers.

But often the world is not stationary. Imagine the doctor example. Maybe treatment B works better in winter because of seasonal symptoms, while treatment A works better in summer. If those underlying success probabilities change with time, averaging every reward equally (so that old data forever has the same weight as recent data) makes the agent slow to adapt.

To handle changing conditions you want more weight on recent observations. One easy way is to use a constant step size in the incremental update instead of the diminishing $1/n$.

Replace the $1/n$ term by a constant $\alpha \in [0, 1]$. After taking an action and seeing reward R_n , update as

$$Q_{n+1} = Q_n + \alpha[R_n - Q_n].$$

This is the same structure as before, but now the learning rate α does not shrink over time. The effect is that recent rewards keep shifting the estimate a fixed fraction of the current error; older rewards get exponentially discounted.

You can expand the update to see the effective weights on past rewards. Start from

$$Q_{n+1} = (1 - \alpha)Q_n + \alpha R_n$$

and unroll back:

$$Q_{n+1} = (1 - \alpha)^n Q_1 + \alpha \sum_{i=1}^n (1 - \alpha)^{n-i} R_i.$$

So the weight on reward R_i is $\alpha(1 - \alpha)^{n - i}$, an exponentially decaying factor depending on how old that reward is. The older the reward, the smaller its influence. That's exactly what you want when the world may have shifted and recent evidence is more relevant.

Choose α based on how quickly you want to adapt: large α means fast adaptation (but more variance), small α means slower but smoother updates.

7.4.3 Optimistic initial values

This is a neat trick that encourages exploration even if you always choose greedily. Start with artificially high initial estimates for all actions; because observed rewards will typically be lower than the initial optimistic values, the agent becomes “disappointed” and tries other actions until their estimates come down.

Instead of starting every $Q[a]$ at zero, set them all to a high number. The idea is that early on, every action looks great, so a greedy policy will try different actions to see which ones really deliver. When an action disappoints (observed reward lower than the optimistic initial value), its estimate falls and the policy tries others.

Suppose there are three treatments, rewards are either 1 for success or 0 for failure (Bernoulli), and we pick a constant step size $\alpha = 0.5$ for simplicity. Set each initial estimate to $Q_1(a) = 2$ (very optimistic, since actual rewards are only 0 or 1).

Pick some sequence of actions and rewards and watch the updates. If you pick action A first and observe $R_1 = 1$, then $Q_2(A) = Q_1(A) + \alpha(R_1 - Q_1(A)) = 2 + 0.5(1 - 2) = 2 - 0.5 = 1.5$. So after one success the estimate dropped from 2 to 1.5. If you then pick a different action B and it yields $R_2 = 0$, that action's estimate goes from 2 to $Q_2(B) = 2 + 0.5(0 - 2) = 2 - 1 = 1$.

Because all actions started very high, the greedy policy (which always picks the current argmax) will cycle through actions until the initial optimism is corrected by actual rewards. That produces natural exploration even without explicit ϵ randomness.

Optimistic initial values are especially attractive when rewards are deterministic or low-variance and you want to encourage early exploration: the agent behaves greedily but still samples each action because each one initially looks promising.

Chapter 8

Performance of methods for action selection

When an agent faces several actions whose rewards are unknown, it must balance two goals: try actions that look good right now (exploit) and try other actions to learn if they might be even better (explore). Different strategies — from pure randomness to careful optimism — handle that tradeoff in different ways. Comparing these strategies systematically is how we decide which are useful in practice.

There are four families of simple strategies we use as baselines or building blocks.

- *Random action selection.* This is the simplest: pick any action with equal probability.
- *Greedy.* Always pick the action with the highest current estimated value.
- *ϵ -greedy.* Mostly act greedily, but with small probability ϵ pick a random action.
- *Optimistic initial values.* Initialize the estimated values of all actions to a high number so that, even if you act greedily according to those estimates, every action will be tried at least a few times (because the optimistic estimates will be reduced once you observe reality).

8.1 The 10-armed testbed

To compare action-selection methods fairly and repeatably we use a canonical simulated environment called the 10-armed bandit testbed. It's small (10 actions) but random enough to be a useful benchmark.

For each independent bandit problem, each action a is assigned a true value $q_*(a)$. Those true values are drawn independently from a normal distribution with mean 0 and variance 1. When the agent selects action a , the observed reward is a noisy sample centered on $q_*(a)$. Specifically, the actual reward R is drawn from a normal distribution with mean $q_*(a)$ and variance 1. So each run is a realization of a random problem: first the vector $q_*(1 \dots 10)$ is drawn; then, as the agent acts, individual rewards are drawn around the selected action's mean (see Figure 8.1).

There are three separate sources of randomness that matter:

- The true action values q_* are random from one run to the next. Every independent run uses a different underlying bandit problem.
- Given q_* , the rewards the agent observes are random because of the Gaussian reward noise.
- If the action-selection policy performs exploration (e.g., ϵ -greedy or random selection), those exploration steps are themselves random.

To obtain a stable picture of performance we perform many independent runs. The classic setup used here is 2000 independent runs of a 10-armed bandit, each run lasting 1000 time steps. For each time step t we average the reward received, using the sample-average method, across the 2000 runs. This averaged curve smooths out per-run noise and reveals the learner's average behavior over problems (see Figure 8.6).

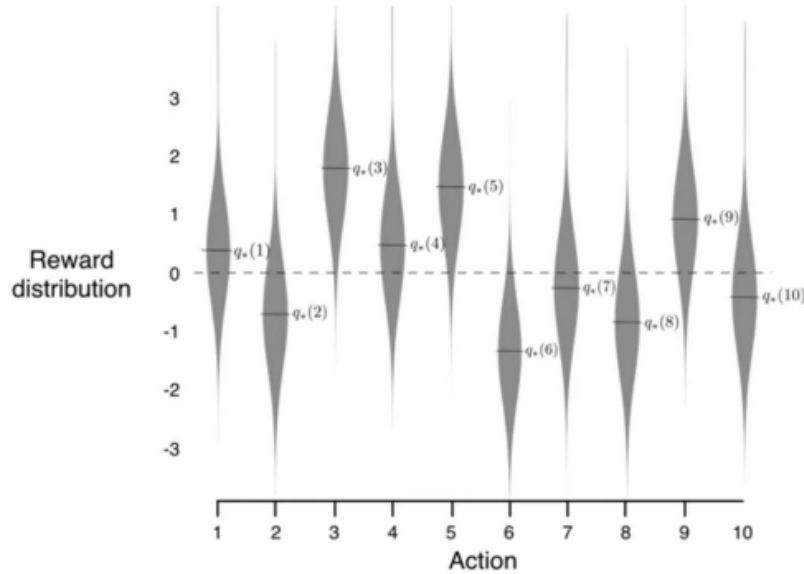
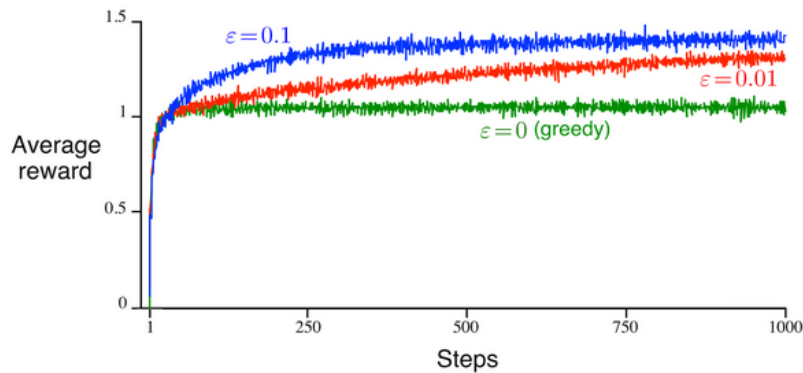


Figure 8.1: 10-arms testbed.

When we average over 2000 runs, two phases usually appear in the reward-per-step curve. First, an early learning period where reward increases sharply as the agent discovers better actions. After that, the curve levels off toward a steady value that represents the long-run average reward for that algorithm in this testbed.

Across experiments, a consistent pattern emerges: the purely greedy algorithm ($\epsilon = 0$) tends to perform poorly in the long run because it can get stuck on suboptimal actions discovered early. By contrast, small amounts of exploration ($\epsilon = 0.01$ or $\epsilon = 0.1$) allow continual sampling of other actions and better long-term performance (see Figure 8.2). On the particular testbed used in these experiments, the greedy method achieved an average reward per step of about 1.0, whereas the best possible average reward for that testbed is around 1.55. Adding exploration with $\epsilon = 0.1$ moves the agent closer to that optimum.

Figure 8.2: Performance comparison between greedy and ϵ -greedy.

Let's compare greedy ($\epsilon = 0$ with optimistic initialization $Q_1(a) = +5$ and $\epsilon = 0.1$ (regular ϵ -greedy) to see how optimistic initialization changes behavior (see Figure 8.3).

$\epsilon = 0.1$ usually performs much better long run because it keeps exploring 10% of the time. Optimistic initial values with greedy action selection can approximate the exploration of an ϵ -greedy agent during the initial time steps: agents try many actions early because of the initial optimism. However, after that initial exploration period the agent behaves greedily and will not continue to explore, so long-term performance depends on whether that initial exploration found the best action.

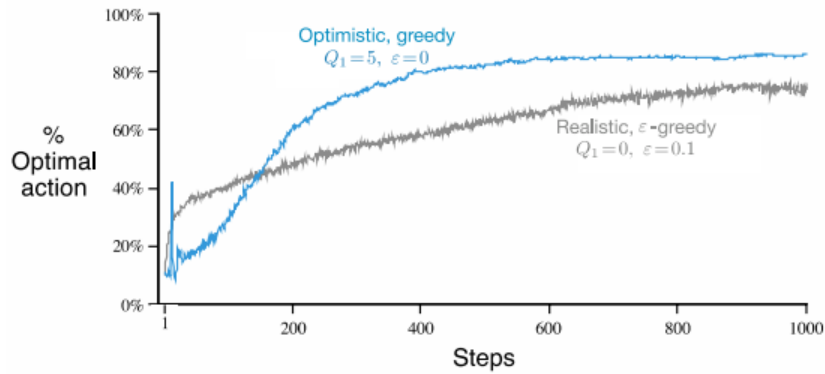


Figure 8.3: Performance comparison between ϵ -greedy and optimistic initial value.

8.2 Optimism in the Face of Uncertainty

There are several simple ways to push an agent away from pure greed and make it explore. A more subtle and powerful principle is “*optimism in the face of uncertainty*.” The idea is not merely to prefer actions whose current estimated means are high, nor to prefer actions you haven’t tried at all. Instead, prefer actions that have the greatest potential to be the best, i.e., those that have high upper bounds on their plausible true value because we’re still uncertain about them.

Think about two simple toy problems. First, a very easy one: two arms, one always good and one always bad. Try both once and you’re done; you’re certain which is best. Now a harder one: two arms where one is slightly better than the other, but rewards are noisy. Here it may take a very long sequence of samples to confidently tell which mean is higher. Hard problems are those where arms have similar true means and the observation noise is large.

We can picture uncertainty as a confidence interval around our current estimate $Q(a)$ (see Figure 8.4). If that interval is tiny, we’re pretty sure $q_*(a)$, the true mean, is near $Q(a)$. If it’s large, all we know is that the true mean might still be much higher (or lower) than our estimate. The width of that interval shrinks with more samples.

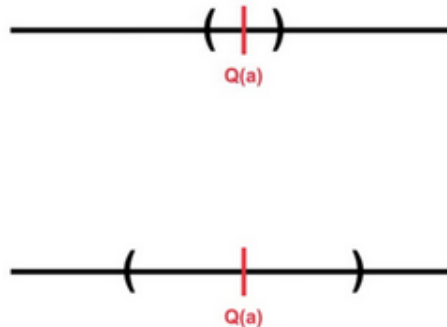


Figure 8.4: Confidence interval.

If an action’s estimated mean is modest but its uncertainty interval stretches well above the other actions’ current values, that action has the potential to be the best. Optimism in the face of uncertainty says: treat that potential as real until we’ve shown otherwise. In practice this leads us to pick actions not simply by their point estimates but by optimistic upper estimates — the highest plausible value consistent with the data so far.

Imagine you’re looking at three arms (see Figure 8.5). One has a high estimated mean but a tight confidence interval (green line); the second has a slightly lower estimate but a wide interval (blue line); the third is poorly sampled and its interval reaches the highest possible values (red line). Choosing the arm with the highest upper bound either gives you a good reward immediately if the optimism was justified, or it teaches you the most about uncertain arms if the optimism turns out to be wrong. Both outcomes are valuable.

Upper Confidence Bound action selection makes the optimism concrete. At time step t pick the

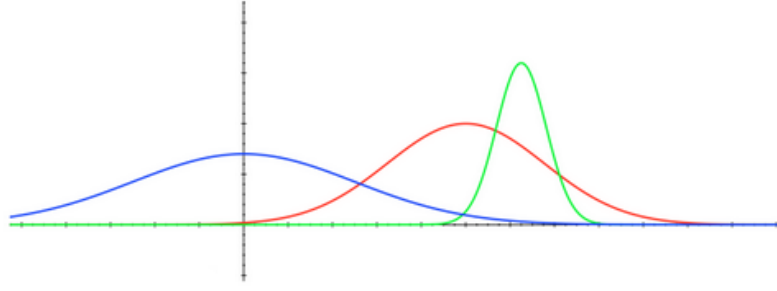


Figure 8.5: Optimism in the face of uncertainty plot example.

action a that maximizes

$$A_t = \operatorname{argmax}_a \left[Q_t(a) + c \sqrt{\frac{\ln t}{N_t(a)}} \right]$$

where: $Q_t(a)$ is your current estimate (sample average, for instance) of action a 's mean reward at time t . $N_t(a)$ is the number of times action a has been selected so far (at time t). $\ln t$ is the natural logarithm of the current global time step; it grows slowly with time. c is a positive parameter you choose that scales how optimistic you are — larger c means more exploration. The second term is the “exploration bonus” or “upper-confidence term.” It decreases as $N_t(a)$ grows, roughly like $1/N_t(a)$, so the more you try an action, the less extra credit you give it for uncertainty. The log factor $\ln t$ makes the bonus shrink slowly with overall time so that arms that were neglected a long time ago get a slowly increasing chance relative to others (but not too rapidly).

Example 8.2.1 – Exploration term

Suppose we are at global time step $t = 10,000$. For an action a that has been chosen $N_t(a) = 5,000$, the uncertainty term becomes $\sqrt{\frac{\ln 10000}{5000}} \approx 0.043$. If another action has been chosen only 100 times, the term is $\sqrt{\frac{\ln 10000}{100}} \approx 0.303$.

That shows how the UCB bonus strongly favors less-sampled actions and how the magnitude depends on the counts. So if $c = 1$, the less-sampled action would have a bonus of about 0.3 while the well-sampled action gets 0.043. With a larger c you amplify the preference for uncertainty.

ϵ -greedy will forever pick a random action at rate ϵ (for example 10% of the time if $\epsilon = 0.1$), while UCB tends to front-load exploration: it explores a lot initially to reduce uncertainty about all arms and then gradually reduces exploration as confidence grows. UCB's exploration decreases with time because the upper-confidence bonuses shrink as sample counts increase. This leads to strong performance on stationary problems where the action means don't change.

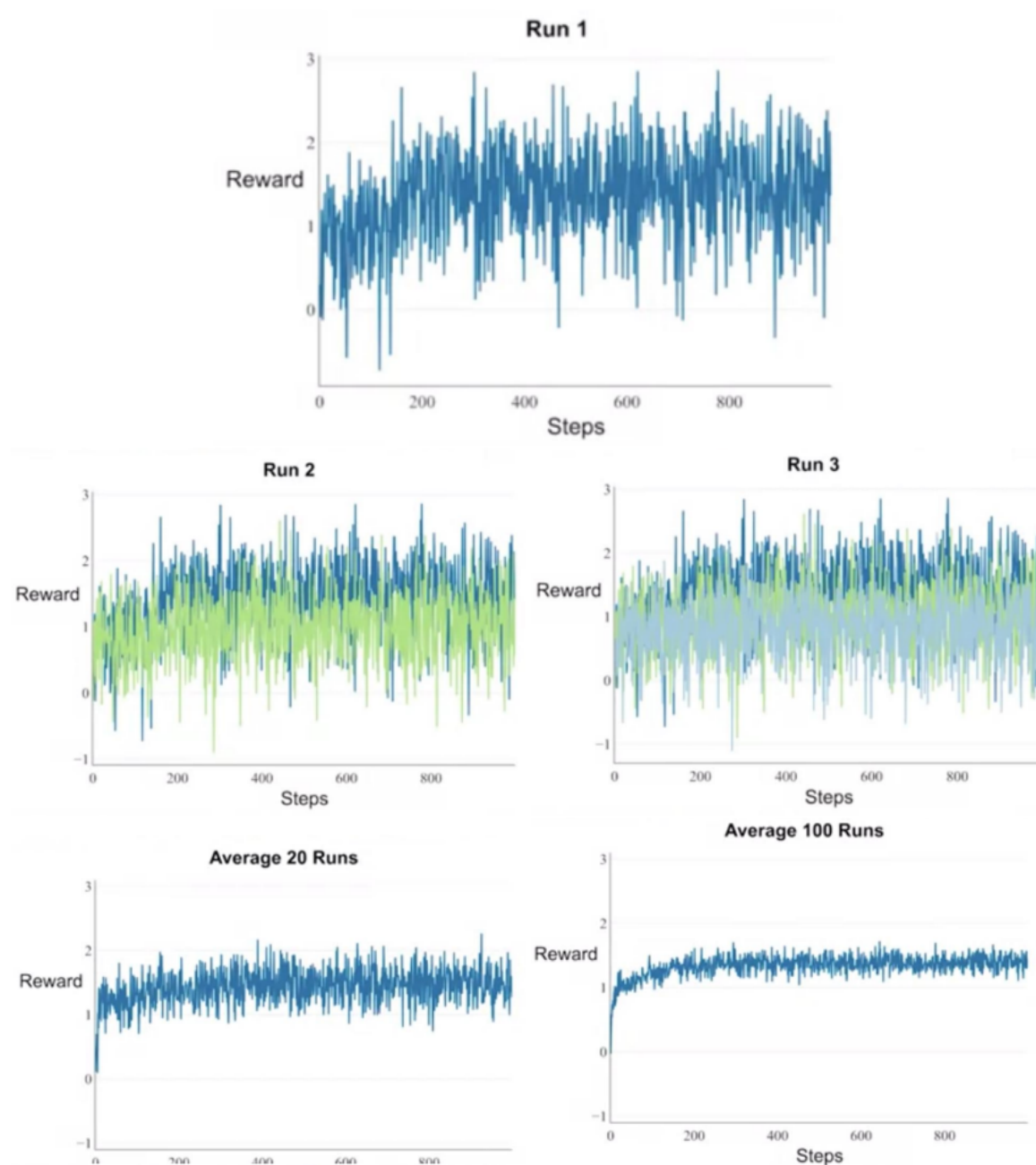


Figure 8.6: Multiple runs in the 10-arms testbed.

Chapter 9

Formalization of sequential decision making

In this chapter we will dive into Markov Decision Process (MDP). First we'll remind ourselves what it means for a process to be Markovian. Next we'll add numerical rewards to that process and ask about total accumulated reward; that gives us a Markov reward process (MRP). Finally we allow an agent to choose actions that influence the transitions and the rewards; that gives us a Markov decision process (MDP). Formulating a learning problem as an MDP is the first step in applying reinforcement learning: once you have the MDP, you can ask "what policy maximizes expected return?" and start designing algorithms to learn that policy.

Many decision problems have delayed consequences: the action you take now affects not only the reward you receive immediately but also which states you'll see later and therefore the future rewards you can obtain. The MDP framework forces you to model this sequential dependence explicitly and to think about trading off immediate against delayed reward.

Imagine an agent interacting with an environment at discrete time steps $t = 0, 1, 2, \dots$. At time t the agent observes a state S_t (a representation of the environment), chooses an action A_t , and then, one time step later, receives a numerical reward R_{t+1} and lands in a new state S_{t+1} .

9.1 Introduction to Markov Decision Process

A state S_t is called Markov if knowing S_t makes everything from the full history irrelevant for predicting the future. Formally, a state sequence has the Markov property if

$$P(S_{t+1}|S_t) = P(S_{t+1}|S_1, S_2, \dots, S_t).$$

In words: once you know the current state, the probability distribution of the next state does not depend on how the process arrived at the current state. The present is a sufficient statistic of the future. That is why we can "forget" the history and reason only about the current state and the transition probabilities from it.

If the state space S is finite we can arrange all transition probabilities into a matrix. For two states s (current) and s' (next), the one-step transition probability is

$$P_{ss'} = P(S_{t+1} = s' | S_t = s).$$

The matrix P with entries $P_{ss'}$ collects these probabilities for all pairs s, s' . Each row of P is a probability distribution over next states given the current state, so each row sums to 1.

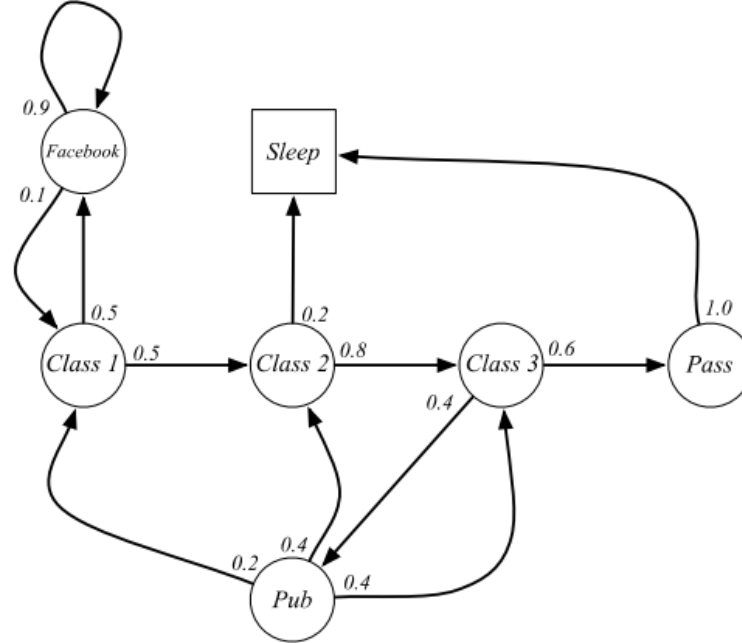
A Markov process (also called a Markov chain) can thus be compactly described as the pair $\langle S, P \rangle$: the set of states and the transition matrix.

Example 9.1.1 – Student Markov Chain

Imagine a simple model of a student moving through states that describe the student's academic day.

Let's assume the student was taking Class1 of the Autonomous Networking course. From

this state, he can reach Class2 with a probability of 0.5, and if bored by the lesson, he starts scrolling Facebook with a probability of 0.5. Here, the social network algorithm keeps him glued to the screen with a probability of 0.9, and he returns to the lesson only 10% of the time. Obviously, the student's ultimate goal is to reach the Pass state to pass the exam. To do this, he will have to attend all three classes, avoiding to falling asleep during Class2, which is also the most difficult. In the meantime, he might consider taking a break from studying and going to the Pub. Here, he shouldn't overdrink, otherwise he'll forget the lessons learned in the classes and have to start over again.



Each state has probabilities of transitioning to certain successor states. From the transition matrix you can sample sequences such as

$$C_1 \rightarrow C_2 \rightarrow C_3 \rightarrow \text{Pass} \rightarrow \text{Sleep},$$

or

$$C_1 \rightarrow \text{FB} \rightarrow \text{FB} \rightarrow C_1 \rightarrow C_2 \rightarrow \text{Sleep},$$

or longer weird sequences like

$$C_1 \rightarrow \text{FB} \rightarrow \text{FB} \rightarrow C_1 \rightarrow C_2 \rightarrow C_3 \rightarrow \text{Pub} \rightarrow C_1 \rightarrow \text{FB} \rightarrow \text{FB} \rightarrow \text{FB} \rightarrow C_1 \rightarrow C_2 \rightarrow C_3 \rightarrow \text{Pub} \rightarrow C_2 \rightarrow \text{Sleep}.$$

Those sequences are just realizations of the Markov chain according to the probabilities in the transition matrix. When you simulate, the actual sequence depends on the random draws at each step but the distribution over sequences is completely defined by P .

A representation of the transition matrix P of the student Markov chain example can be:

$$P = \begin{bmatrix} 0 & 0.5 & 0 & 0 & 0 & 0.5 & 0 \\ 0 & 0 & 0.8 & 0 & 0 & 0 & 0.2 \\ 0 & 0 & 0 & 0.6 & 0.4 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1.0 \\ 0.2 & 0.4 & 0.4 & 0 & 0 & 0 & 0 \\ 0.1 & 0 & 0 & 0 & 0 & 0.9 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1.0 \end{bmatrix}$$

with row-labels: $C_1, C_2, C_3, \text{Pass}, \text{Pub}, \text{FB}, \text{Sleep}$ and same order for column-labels.

9.2 Markov Reward Process

A Markov reward process is simply a Markov chain with a numerical reward assigned to states (or transitions) as illustrated in Figure 9.1. We write an MRP as the tuple $\langle S, P, R, \gamma \rangle$. Here S and P are as before, R is a reward function and γ is a discount factor.

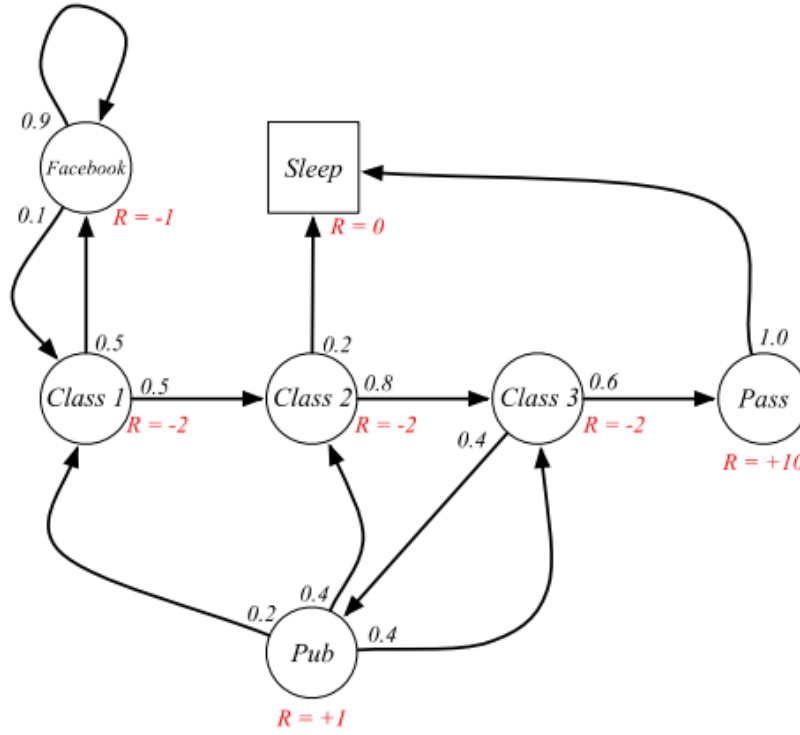


Figure 9.1: Markov Reward Process in the student example.

The reward function R is commonly given as the expected immediate reward when in state s :

$$R_s = E[R_{t+1} | S_t = s].$$

Being in state s at time t leads to receiving reward R_{t+1} on the transition to $t + 1$.

The discount factor γ is a number in the interval $[0, 1]$ which defines how future rewards are valued relative to immediate rewards. We'll explain its role in a moment.

When we evaluate what happens from time t onward we care about the sequence of future rewards. The return G_t is defined as the total discounted reward from time t :

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}.$$

Discounting multiplies a reward received k steps in the future by γ^k . If $\gamma = 0$ the agent is completely myopic and only considers the immediate reward. If γ is near 1 the agent is far-sighted and cares about long-term consequences.

Why do we include a discount factor? Several reasons. Discounting is mathematically convenient because it ensures the infinite sum is finite in many cyclic or continuing processes. Practically it expresses a preference for rewards sooner rather than later, models uncertainty about long-term outcomes, and sometimes corresponds to economic notions of time preference. In episodic tasks (where sequences terminate) it is sometimes appropriate to set $\gamma = 1$, but for continuing tasks the discount is usually strictly less than 1.

The state value function $v(s)$ gives the expected return when starting in state s . Formally,

$$v(s) = E[G_t | S_t = s].$$

Because G_t is a sum of discounted future rewards, $v(s)$ captures the long-run goodness of being in state s , taking into account future transitions and rewards.

If $\gamma = 0$ the value function ignores future rewards entirely and reduces to the immediate reward. Whatever reward you get immediately in state s is the value $v(s)$. If $\gamma = 0.9$ (a moderately high discount), the value $v(s)$ combines the immediate reward and substantial weight on rewards a few steps ahead. The value of states that are likely to lead to Pass or other high-reward states will increase, while states likely to lead to FB or Sleep will have lower value. If $\gamma = 1$ (no discounting, appropriate only when episodes terminate or when the sum is known finite), the value becomes the expected undiscounted sum of future rewards. This makes long term prospects weigh exactly the same as near-term ones, so states that eventually lead to a lot of reward are very valuable even if the reward is delayed.

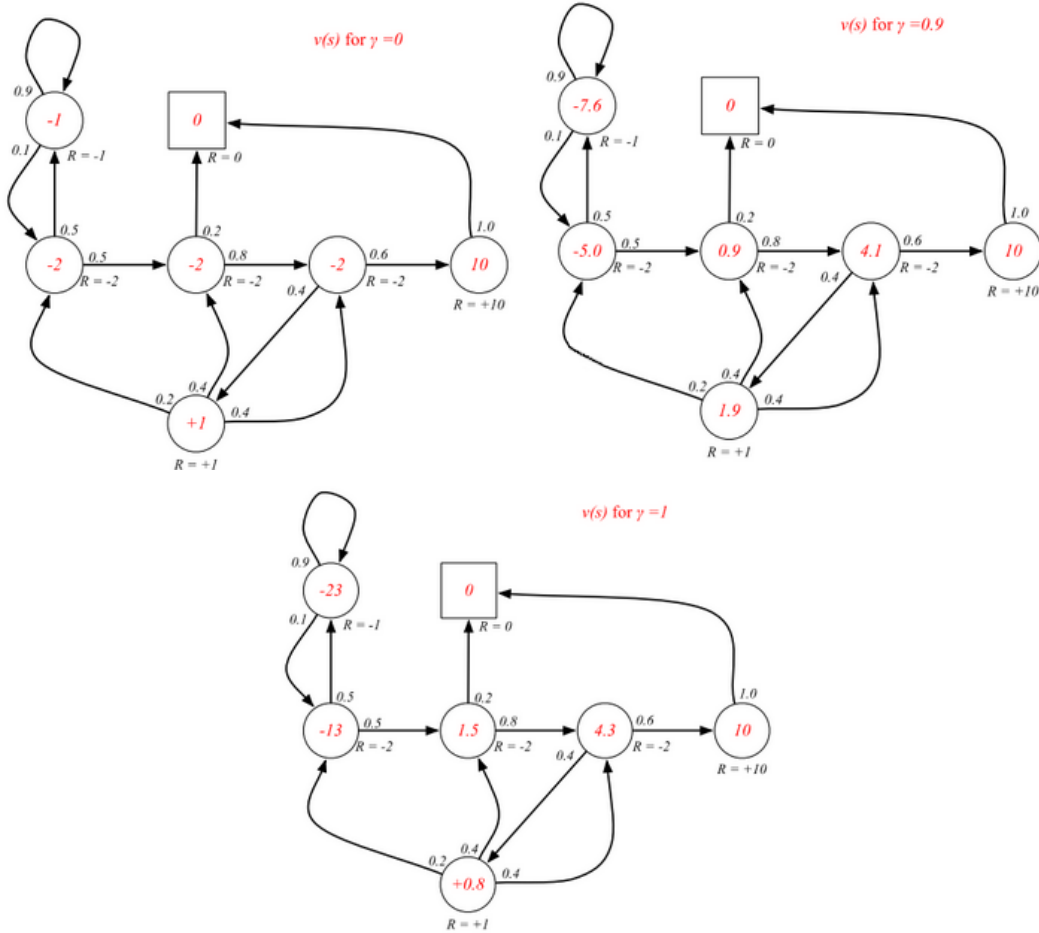


Figure 9.2: State-Value Function for Student MRP.

The value of a state and the value function can be split into two parts. If you are in state s the total value function $v(s)$ equals the immediate reward you get on the next transition, R_{t+1} , plus the discounted value from the next state, $\gamma v(S_{t+1})$:

$$\begin{aligned}
 v(s) &= E[G_t | S_t = s] \\
 &= E[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots | S_t = s] \\
 &= E[R_{t+1} + \gamma(R_{t+2} + \gamma R_{t+3} + \dots) | S_t = s] \\
 &= E[R_{t+1} + \gamma G_{t+1} | S_t = s] \\
 &= E[R_{t+1} + \gamma v(S_{t+1}) | S_t = s].
 \end{aligned}$$

future value is just an expected, discounted copy of the same quantity evaluated one time-step later.

Because the environment is stochastic, there may be many possible successor states and many possible rewards when you start from s . The Bellman equation averages over all these possibilities, weighting each by its probability. In words it says: the value of the starting state must equal the expected immediate reward plus the discounted expected value of where you end up next. The value of a state is tied to the values of its successors.

Start from the definition $v(s) = E[G_t | S_t = s]$ and substitute $G_t = R_{t+1} + \gamma G_{t+1}$. Then

$$v(s) = E[R_{t+1} + \gamma G_{t+1} | S_t = s].$$

The expectation of the sum is the sum of expectations. The immediate reward expectation is $R_s = E[R_{t+1} | S_t = s]$. The second term is γ times the expected return from the next state: $\gamma \sum_{s' \in S} P_{ss'} v(s')$, because the next state is s' with probability $P_{ss'}$ and from there the expected future return is $v(s')$. Put together:

$$v(s) = R_s + \gamma \sum_{s' \in S} P_{ss'} v(s').$$

That is the canonical Bellman equation for an MRP.

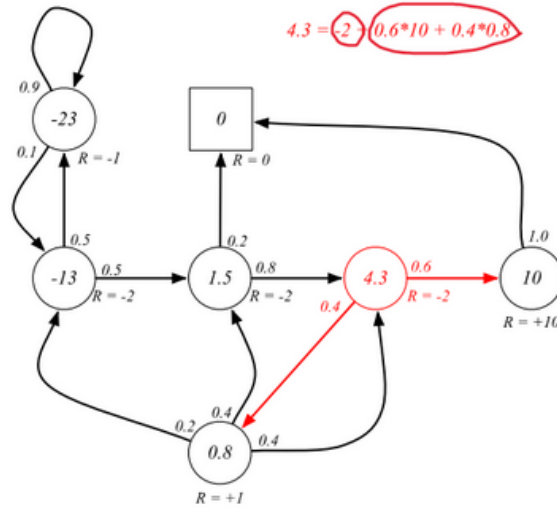


Figure 9.3: Bellman equation applied to the student example with $\gamma = 1$.

It's often cleaner to write the Bellman equation in vector/matrix notation. Let v be the column vector whose entries are $v(s)$ for each state s . Let R be the column vector with entries R_s . Let P be the transition matrix with entries $P_{ss'}$. Then the Bellman equation for the MRP becomes

$$v = R + \gamma P v.$$

This single vector equation contains all state equations at once. You can rearrange it into the familiar linear-algebra form $(I - \gamma P)v = R$. If $I - \gamma P$ is invertible, the closed-form solution is:

$$\begin{aligned} v &= R + \gamma P v \\ (I - \gamma P)v &= R \\ v &= (I - \gamma P)^{-1} R \end{aligned}$$

The Bellman equation is a linear system in the unknowns $v(s)$. For a state space of size n , solving the linear system directly by matrix inversion or Gaussian elimination has computational complexity roughly $O(n^3)$. That is fine for small MRPs but quickly becomes infeasible when n is large. Because of this, a variety of iterative methods are used for larger problems. Dynamic programming gives iterative policy-evaluation procedures that converge to the exact v when you have a complete model. Model-free methods such as Monte Carlo evaluation approximate values by sampling complete returns, and temporal-difference (TD) methods like TD(0) bootstrap the current estimate against sampled immediate rewards plus a discounted next-state value. All of these are ways to solve or approximate the Bellman equations without inverting a huge matrix.

9.3 Markov Decision Process

A Markov decision process is exactly a Markov reward process with actions: the environment is fully observable, all states are Markov, and the agent chooses actions that influence the next-state distribution and the immediate reward.

Formally, an MDP is the tuple $\langle S, A, P, R, \gamma \rangle$. The set S is the finite set of states. A is the set of actions; the transition matrix is $P_{ss'}^a = P(S_{t+1} = s' | S_t = s, A_t = a)$, and $R_s^a = E[R_{t+1} | S_t = s, A_t = a]$.

There is effectively one transition matrix per action. If you think of P as a block of matrices, each action a supplies a matrix P^a that tells you how the environment responds to that action across all states.

In the student MDP the states are the same as before, but now the agent chooses actions in some states: for instance the agent can choose to study or check Facebook. Each action might lead to different immediate rewards and different transition probabilities. The goal is to choose actions along the way so that the accumulated discounted reward is maximized.

9.3.1 Policy

A policy fully defines the agent's behavior. A policy π maps states to distributions over actions; formally

$$\pi(a|s) = P(A_t = a | S_t = s).$$

Policies are typically assumed stationary: the distribution depends on the current state but not on time. Stationarity makes the math simpler and often suffices for many tasks, though time-dependent policies can appear in certain contexts.

When a policy π is fixed, an MDP becomes an MRP: the stochastic dynamics together with the policy create effective transition probabilities and expected rewards (you marginalize over the actions chosen by π).

9.3.2 Value function

For a given policy π the state-value function $v_\pi(s)$ is the expected return starting in state s and following π afterwards:

$$v_\pi(s) = E_\pi[G_t | S_t = s].$$

The action-value function $q_\pi(s, a)$ measures the expected return starting from state s , taking action a once at that state, and then following π thereafter:

$$q_\pi(s, a) = E_\pi[G_t | S_t = s, A_t = a].$$

These two functions are linked by the simple identity

$$v_\pi(s) = \sum_{a \in A} \pi(a|s) q_\pi(s, a),$$

which simply says the value of a state under π is the average, under π , of the action-values for that state.

You can write Bellman-style recursive equations for both v_π and q_π . The state-value Bellman expectation equation decomposes $v_\pi(s)$ into the expected immediate reward plus the discounted expected value of the successor state, where the expectation averages both over the policy's action distribution and the environment's transition distribution. Formally:

$$v_\pi(s) = \sum_{a \in A} \pi(a|s) \left(R_s^a + \gamma \sum_{s' \in S} P_{ss'}^a v_\pi(s') \right).$$

You can see this as first choosing an action with probability $\pi(a|s)$; given that action, you get immediate expected reward R_s^a and then move to s' with probability $P_{ss'}^a$, from which you get value $v_\pi(s')$.

The action-value Bellman expectation equation is similar, but it conditions on having taken action a in the current state:

$$q_\pi(s, a) = R_s^a + \gamma \sum_{s' \in S} P_{ss'}^a \sum_{a' \in A} \pi(a'|s') q_\pi(s', a').$$

Here the inner sum $\sum_{a' \in A} \pi(a'|s') q_\pi(s', a')$ is simply $v_\pi(s')$, hence the action-value equation can be written as

$$q_\pi(s, a) = R_s^a + \gamma \sum_{s' \in S} P_{ss'}^a v_\pi(s').$$

9.4 Optimal policy

A policy is simply a way for an agent to behave: given the current state, the policy tells the agent what action to take (deterministically or stochastically). Up to now, we've often treated a policy as something that was given, and the task was to evaluate it — to compute the value function that tells us how good that policy is.

Reinforcement learning, however, has a more ambitious goal. We don't only want to evaluate a single policy; we want to discover the best policy possible. That is, we want a policy that, across time, gathers as much cumulative reward as possible. The rest of this chapter is about formalizing what "best" means, how to express it, and how to compute it.

To compare two policies we look at their value functions. A policy π is considered better than or equal to another policy π' if for every state s the expected return starting from s under π is at least as large as under π' . In symbols we write $\pi \geq \pi'$ when $V_\pi(s) \geq V_{\pi'}(s)$ for all s . Because of this partial ordering, we can ask whether there exists a policy that is better than or equal to every other policy. The answer is yes. For any Markov Decision Process (MDP) there exists at least one policy π_* such that $\pi_* \geq \pi$ for every policy π . That policy π_* is called an optimal policy. All optimal policies share the same optimal value functions (state-value and action-value), so finding those value functions is the key to finding optimal behavior.

Two central objects are the optimal state-value function v_* and the optimal action-value function q_* . The optimal state-value function $v_*(s)$ is defined as the maximum state-value achievable from state s by any policy:

$$v_*(s) = \max_{\pi} V_{\pi}(s).$$

This expresses the best possible expected return you can guarantee when starting from state s . The optimal action-value function $q_*(s, a)$ is defined similarly but conditions on taking a specific action a in state s first:

$$q_*(s, a) = \max_{\pi} q_{\pi}(s, a).$$

In words, $q_*(s, a)$ tells you the best expected return you can achieve if you start in state s , take action a now, and then follow the best possible policy afterwards.

If you have $q_*(s, a)$, finding an optimal policy is immediate: in each state choose an action that maximizes q_* :

$$\pi_*(a|s) = \begin{cases} 1 & \text{if } a = \operatorname{argmax}_{a \in A} q_*(s, a) \\ 0 & \text{otherwise} \end{cases}$$

Because the *argmax* may not be unique, there can be multiple optimal policies, but they all have the same values v_* and q_* . Importantly, there always exists at least one deterministic optimal policy for any finite MDP — so you can pick a single best action in every state and be optimal.

9.4.1 Bellman Optimality Equation

The optimal value functions satisfy recursive identities called the Bellman optimality equations. They are the same flavor as the Bellman equations for a fixed policy, but here we take the maximum over actions because we allow the agent to choose the best action at each step.

The optimal value functions satisfy recursive identities called the Bellman optimality equations. They are the same flavor as the Bellman equations for a fixed policy, but here we take the maximum over actions because we allow the agent to choose the best action at each step.

The Bellman optimality equation for the optimal state-value v_* is:

$$v_*(s) = \max_a E[R_{t+1} + \gamma v_*(S_{t+1}) | S_t = s, A_t = a],$$

or, written explicitly in terms of transition probabilities,

$$v_*(s) = \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma v_*(s')].$$

Read this as: the value of the best action in state s equals the expected immediate reward from that action plus the discounted value of the next state, where we then assume we continue optimally.

The Bellman optimality equation for the optimal action-value q_* is similar but starts from a state-action pair:

$$q_*(s, a) = E[R_{t+1} + \gamma \max_{a'} q_*(S_{t+1}, a') | S_t = s, A_t = a],$$

or explicitly

$$q_*(s, a) = \sum_{s', r} p(s', r | s, a) [r + \gamma \max_{a'} q_*(s', a')].$$

This says: the value of taking action a in state s equals the expected immediate reward plus the discounted value of the best action available in the next state.

9.4.2 Solving the Bellman Optimality Equation

The Bellman optimality equation is the precise mathematical statement of “act optimally at every step.” It tells us, for each state or state–action pair, what the optimal value must equal. But there’s a catch. The equation is nonlinear because it contains a max over actions. That nonlinearity means there is no general closed-form solution you can write down for v_* or q_* in an arbitrary MDP. Instead we solve it iteratively.

There are several well-known iterative families of methods. If you know the model (the transition probabilities and rewards), dynamic programming gives elegant and reliable algorithms; policy iteration alternates between evaluating a policy and improving it; if you don’t know the model and must learn from experience, you use sample-based methods such as Q-learning or Sarsa (and more generally TD-based methods). Each of these algorithms is a way of repeatedly applying the Bellman operator (or an approximation of it) until the value estimate stabilizes.

Temporal-Difference (TD) learning

Temporal-Difference learning is a family of methods that learn value functions directly from experience. The key idea is: instead of waiting until an entire episode finishes to compute a return, update value estimates using the difference between successive predictions — that is, the prediction error you see after one time step.

If you are estimating a state value function V_π for a fixed policy π , TD methods address the prediction problem: given observed transitions generated using π , how should you update your estimate of V_π ? At each time step, you have the previous estimate for the current state, you observe the immediate reward and the next state, and you compute a one-step target from that observation. The difference between that one-step target and your old estimate is called the TD error; it tells you how much to adjust the old estimate.

The simplest TD method is TD(0), also called one-step TD. It’s a special case of the more general TD(λ) family, and it forms the building block for TD control algorithms like Q-learning.

Example 9.4.1 – Driving-home story

Imagine every day you try to predict how long the drive home will take. In the morning you say “30 minutes.” Halfway there you’re stuck in traffic and realize 30 minutes was optimistic. Do you have to wait until you arrive to adjust your initial prediction? TD says: no. As soon as you get information (say, a traffic jam report or a much slower speed on the road), you update your prediction for the starting point using that intermediate information. TD(0) is exactly that — you update estimates step by step as new observations arrive, using the most recent successor as a sample of the next state.

For state-value prediction TD(0) uses this update:

$$V(S_t) \rightarrow V(S_t) + \alpha [R_{t+1} + \gamma V(S_{t+1}) - V(S_t)].$$

Here α is the learning rate, R_{t+1} is the reward observed after leaving S_t , and γ is the discount factor. The bracketed term is the TD error: the one-step target $R_{t+1} + \gamma V(S_{t+1})$ minus the old estimate $V(S_t)$. Notice TD(0) uses a single sampled successor rather than the full expected distribution over successors; that’s what makes it sample-based.

```

1 Input: the policy  $\pi$  to be evaluated
2 Algorithm parameters: step size  $\alpha \in [0, 1]$ 
3 Initialize  $V(s)$ , for all  $s \in S^+$ , arbitrarily except that  $V(\text{terminal}) = 0$ 
4
5 Loop for each episode:
6     Initialize  $S$ 
```

```

7   Loop for each step of episode:
8        $A \rightarrow$  action given  $\pi$  for  $S$ 
9       Take action  $A$ , observe  $R, S'$ 
10       $V(S) \rightarrow V(S) + \alpha[R + \gamma V(S') - V(S)]$ 
11       $S \rightarrow S'$ 
12  until  $S$  is terminal

```

Listing 9.1: Tabular TD(0) for estimating v_π

Q-learning

Q-learning is a TD method extended to the control problem: it directly learns estimates of the optimal action-value function $q_*(s, a)$ from experience, without needing a model.

Q-learning is an off-policy TD control algorithm. “Off-policy” means the policy used to generate behavior (the behavior policy) can be different from the policy being evaluated and improved (the target policy). Concretely, while the agent might explore randomly or follow an exploratory policy to collect data, Q-learning always updates its table toward the greedy target (the max over next-state actions). This makes Q-learning a very flexible algorithm: you can explore arbitrarily and still learn the optimal q_* .

The TD error for Q-learning uses the standard one-step target but with a max over next actions. For a transition $(S_t, A_t, R_{t+1}, S_{t+1})$ the TD error is

$$\delta_t = R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t).$$

Think of this term piecewise. The first part, R_{t+1} , is the immediate reward received after taking action A_t in state S_t . The second part, $\gamma \max_a Q(S_{t+1}, a)$, is the discounted estimate of the best possible future value from the next state. Subtracting the current estimate $Q(S_t, A_t)$ yields the error — how far the current Q is from the one-step backed-up target.

The Bellman equation for optimal Q implies that, in expectation and at convergence, δ_t will be zero for every state–action pair.

Q-learning applies the TD error with a step size α :

$$Q(S_t, A_t) \rightarrow Q(S_t, A_t) + \alpha \delta_t,$$

which expanded is

$$Q(S_t, A_t) \rightarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)].$$

The update mixes the old value with a new sample-based target. The learning rate α controls how much weight you give the new sample. If α is 1 you jump fully to the sample target (unstable); if α is very small you learn slowly.

You can read the Q-learning update as an application of a Bellman backup: the new desired value for the state–action pair (s, a) is the immediate reward plus the discounted value of the best action in the successor state. The algorithm uses the sampled successor to approximate that expected backup. Over many visits and with sufficient exploration, these sample backups approximate the expectation and Q converges to the fixed point that satisfies the Bellman optimality equation for q_* .

```

1  Algorithm parameters: step size  $\alpha \in [0, 1]$ , small  $\epsilon > 0$ 
2  Initialize  $Q(s, a)$ , for all  $s \in S^+$ ,  $a \in A(s)$ , arbitrarily except that  $Q(\text{terminal}, \cdot) = 0$ 
3
4  Loop for each episode:
5      Initialize  $S$ 
6      Loop for each step of episode:
7          Choose  $A$  from  $S$  using policy derived from  $Q$  (e.g.  $\epsilon$ -greedy)
8          Take action  $A$ , observe  $R, S'$ 
9           $Q(S, A) \rightarrow Q(S, A) + \alpha[R + \gamma \max_a Q(S', a) - Q(S, A)]$ 
10          $S \rightarrow S'$ 
11  until  $S$  is terminal

```

Listing 9.2: Q-learning (off-policy TD control) for estimating $\pi \approx \pi_*$.

Chapter 10

Performance Evaluation

Now we'll walk through a practical, systematic way to evaluate the performance of any systems — especially networking protocols — and we'll explain the decision points you'll meet: what to measure, how to measure it, what trade-offs you're making, and how to present the results.

10.1 Practical steps

1. Begin by writing down the question you want the evaluation to answer. For example, you might want to compare a brand new MAC protocol for sensor networks against the current standard. Saying that aloud helps avoid wasting time. Be specific about what “better” means in your context: is it lower energy consumption, higher packet delivery ratio, lower latency, or longer network lifetime? Often you want a mix of those.
2. Next list the concrete services your system provides and the outcomes you'll observe. For a MAC protocol the service is simple: nodes send packets to their neighbors. The outcomes you can directly observe are whether each packet was delivered successfully, how long it took, how much energy was spent to send or receive it, and whether the network is still alive after a long period.

Your evaluation will rest on a handful of performance metrics. Common ones for sensor-network MAC protocols include latency, packet delivery ratio (PDR), energy consumption, and network lifetime. *Parameters* are things you have to set for the system: number of nodes, duty cycle, transmission power, and so on. *Factors* are the subset of parameters you will vary deliberately in your experiments so you can see how the system behaves.

There are three broad ways to evaluate performance.

- *Analytical modeling* uses math to derive performance formulas or bounds. It's fast and cheap but requires simplifying assumptions; if you want a quick insight or to explore parameter relationships, modeling is excellent.
- *Simulation* builds a virtual version of the system in software and runs experiments with controllable randomness and repeatability. It captures more detail than analytic models and is relatively inexpensive, but results depend on the simulator's fidelity.
- *Measurement* (running on real hardware) gives results in the real world, but it's costly in time, money, and logistics. Measurement can reveal issues simulations missed, but it's vulnerable to uncontrolled environmental variation.

Which one to choose depends on where you are in the lifecycle, how much time and money you have, what tools you can access, and how accurate you need to be. If your system is brand new and you don't have hardware, modeling or simulation are essentially your only choices. If you already have a prototype, measurements become an option and often a necessity. If you need results extremely quickly, analytical modeling tends to be fastest because it often requires only pencil, paper, and some math. If you can afford a bit more time, simulation provides richer, more realistic results.

Analytical models are cheap and quick but often less accurate because of simplifying assumptions. Simulations are more costly but can include more system details and therefore usually produce results closer to reality. Measurements cost the most in terms of equipment and time, but they're easiest to convince stakeholders with.

Think through these aspects as you plan.

- *Stage*: Are you evaluating a conceptual design, a prototype, or a production-ready system? Early stage favors modeling or simulation.
- *Time*: How quickly do you need answers? Tight deadlines push toward analytic modeling and careful approximations.
- *Tools*: What existing modeling, simulation, or measurement tools do you or your lab have? Use what you know unless there's a compelling reason to learn a new tool.
- *Accuracy*: How precise must the results be to support your claim? If small differences matter, you'll need simulations or measurements.
- *Trade-off evaluation*: Do you need to compare many designs or parameter settings? Modeling can let you explore broad spaces cheaply.
- *Cost*: What budget is available for hardware, testbeds, or researcher time? Balance ambition against resources.
- *Sellability*: Who needs to be convinced? If your audience is skeptical, measurement results can be more convincing than equations or simulation traces.

Designing experiments means deciding which factors to vary and how, what workloads to run, and which combinations of settings give you the most information with minimal effort.

3. Start by choosing a baseline — a fixed set of parameter values that represent a realistic or commonly used configuration. Then create a small set of experiments that explore meaningful deviations from that baseline, focusing on the factors you think will matter most.
4. Workload selection is often overlooked, but it can make or break your study. Workload describes the traffic patterns and events that occur during the experiment: how many flows, where they start and end, packet sizes, and inter-arrival times. Choose workloads that mimic real application scenarios your protocol is intended for.
5. Once you have data, plotting is your friend. Don't hide the variance: reporting means without variability is misleading. When comparing two protocols, determine whether observed differences are statistically significant and practically meaningful.
6. When preparing plots and tables, make them answer the question you set out at the start. Explain what the observed trade-offs mean for a system designer. For example, "Protocol A reduces average energy per packet by 20% compared to Protocol B when inter-arrival times are long, but at high traffic loads Protocol A's packet delivery drops sharply due to collisions."

Finally, be transparent about limitations. Discuss what you did not model, what assumptions you made, and how those might affect the conclusions.

10.2 Three rules of validation

1. *Do not trust a simulation result until you validate it against either an analytical model or measurements.* Simulators can have bugs or unrealistic parameters.
2. *Do not trust an analytical model until you validate it with simulation or measurement.* Models make simplifying assumptions that can hide important behavior.
3. *Do not trust a measurement until you validate it with an analytical model or a simulation.* Real systems have noise and unique environmental factors; cross-checking helps you separate systemic behavior from artifacts.

10.3 Selecting performance metrics

When you evaluate something you're essentially asking: does it do the job I want, and how well? To answer that you need to observe outcomes and summarize them with metrics that people understand and trust. Good metrics make your results comparable, reproducible, and useful for decision making. Bad metrics hide important trade-offs and can mislead.

When a system is asked to perform a service, there are three high-level outcomes. It can succeed and do the service correctly, it can do it incorrectly (an error), or it can refuse/fail to perform the service at all (unavailable).

These three outcomes are important because they suggest three kinds of metrics you should track: metrics that describe *speed* (how fast successful services complete), metrics that describe *reliability* (how often incorrect behaviors occur, and what kinds of errors), and metrics that describe *availability* (how often the system refuses or fails).

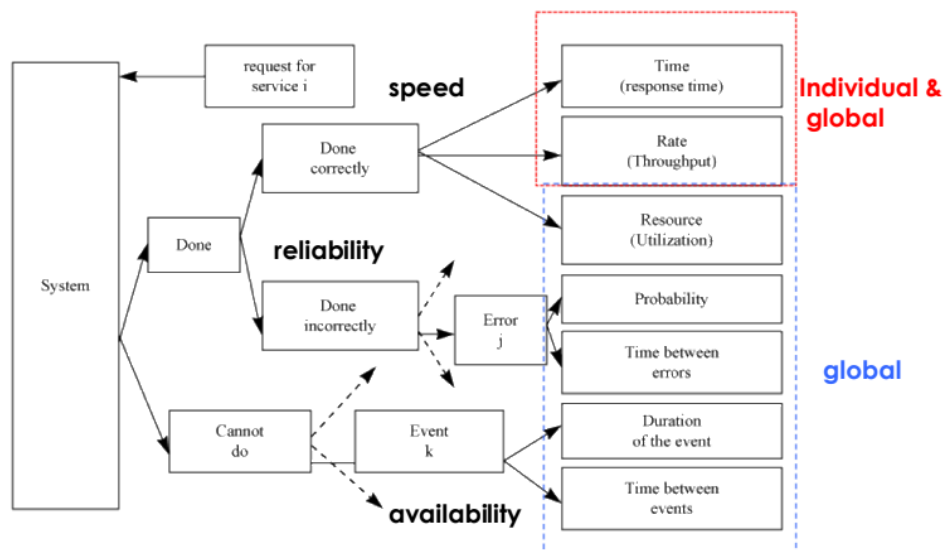


Figure 10.1: Selecting performance metrics.

10.3.1 Speed, reliability, availability

If the service is performed correctly, we measure:

- *Speed*: how long the service takes. For a packet, this is latency: the time between packet arrival and its successful delivery (or acknowledgment).
- *Productivity*: how many services are completed per unit time. For a gateway, throughput is the number of packets forwarded per second.
- *Utilization*: what fraction of system resources are busy serving requests. For a gateway this could be CPU busy time, radio on time, or channel occupancy as a percentage of elapsed time.

If the system performs incorrectly, errors occur. It helps to classify different error types and measure their probabilities. For a gateway you might measure single-bit errors, whole packet corruption, misrouting, etc., and estimate the fraction of packets in each error class.

If the system fails or is unavailable, classify the failure modes (hardware failure, software crash, network partition) and estimate how often each mode happens. For example, the gateway may be unavailable 0.01% of the time due to processor failure and 0.03% due to software failure.

A network is shared by multiple users, which means there are two perspectives to measure performance.

- An individual metric tells how well the system serves one user or one flow — user-centric utility. Examples are per-user throughput, per-flow latency, or per-node energy consumption.
- A global metric captures the system-wide utility — total throughput across all users, fairness indices, or aggregate energy consumed by the whole network.

Optimizing for an individual metric doesn't always optimize the global metric. A classic example is throughput: a single user grabbing the channel aggressively may improve their own throughput but lower others' throughput so much that total throughput or fairness gets worse.

Lots of metrics are possible, and many are correlated. Two useful principles when selecting which metrics to report are non-redundancy and completeness.

- *Non-redundancy* means don't report metrics that say the same thing in slightly different words. If two measures convey essentially the same information, pick the clearer one.
- *Completeness* means your metric set should collectively reflect all possible outcomes: success (speed/productivity/utilization), error (reliability), and unavailability (availability). If you ignore one of those outcomes you may miss important behavior.

Example 10.3.1 – Evaluating a new MAC protocol

You design a new MAC protocol for wireless sensor networks and want to show it's better than existing protocols. You choose simulation as the evaluation technique (because you probably don't have hardware yet), implement the protocol in the simulator, choose a scenario and workload (number of nodes, traffic pattern), pick metrics, run multiple repeated simulations varying some parameters, and get a pile of numbers.

What now? You have to summarize and present the measured data in a way that speaks clearly to the question: "Is the new MAC better, and under what conditions?"

10.4 Summarizing measured data

A measurement campaign can generate hundreds, thousands, or millions of observations for any variable (latency, energy per packet, throughput, etc.). You'll rarely want to dump raw values into a paper or report. Instead you summarize central tendency and variability, and use figures that make comparisons obvious.

10.4.1 Summarizing data by a single number

Two questions are central: how to summarize a dataset by a single number that is meaningful, and how to summarize its variability so readers understand the confidence or spread.

Empirical mean

The most common single-number summary is the *empirical mean*. Given observations $x_1, x_2, \dots, x_n$, the sample mean is the sum of the observations divided by n :

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i.$$

For many variables and large enough samples with limited skew, the mean is informative. But remember: the mean is only a summary; it can be misleading when data are highly skewed or have extreme outliers.

Suppose you measured two packet latencies: one was $l_1 = 2$ ms and the other $l_2 = 5$ s. Be careful with units. The mean is

$$\frac{(0.002 + 5)}{2} = \frac{5.002}{2} = 2.501\text{s}$$

Reporting "mean latency = 2.501 seconds" is correct arithmetically, but it's useless as a description of the system because the two observations reflect two very different behaviors (one very short, one very long).

Geometric mean

If your performance numbers are multiplicative improvements (factors like "2× faster", "0.5× energy"), the *geometric mean* is a more appropriate average because it treats multiplicative changes

symmetrically. For numbers $x_1, x_2, \dots, x_n$ the geometric mean is

$$\frac{\prod_{i=1}^n x_i}{n}.$$

10.4.2 Summarizing variability

Knowing the mean without variability is dangerous. Two systems may have the same mean but very different consistency. Usually you prefer the system whose performance is predictable and low variance.

Common measures of dispersion are range, variance/standard deviation, coefficient of variation, and percentiles (quantiles). Each has pros and cons.

Range

Range is the difference between the maximum and minimum observed values. It's simple to compute and sometimes useful when you know the variable is bounded. But range is highly sensitive to outliers and sample size: as you collect more samples, the maximum tends to get larger (if the tail is unbounded), making range unstable. Use range only when you have reason to believe the variable is bounded or when min/max themselves are meaningful.

Variance and Standard deviation

Variance measures the average squared deviation from the mean:

$$s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2,$$

where s^2 is the sample average ($\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$). The standard deviation s is the square root of variance and has the same units as the data, making it easier to interpret.

Because variance is expressed in squared units, it's not directly comparable to the mean. The standard deviation tells you how tightly values cluster around the mean. If you need a dimensionless measure to compare dispersion across variables with different units, use the coefficient of variation:

$$CV = \frac{s}{\bar{x}}.$$

A low CV means low relative variability.

Percentiles and Quantiles

Percentiles describe the value below which a given fraction of data lies. The p -th quantile is the value below which fraction p of the data falls. The median is the 0.5 quantile (50th percentile). The 95th percentile is the value exceeded by only 5% of observations — useful for describing tail behavior like latency spikes.

To compute an empirical α -quantile from a sample of size n , one simple approach is to sort the n observations and take the element indexed at $(n-1)\alpha + 1$ in the ordered list. For example, with $n = 32$ observations:

- the 10th percentile index is $(31 \times 0.10) + 1 = 4.1$, so you take roughly the 4th ordered element (or interpolate between 4th and 5th if you prefer interpolation);
- the 90th percentile index is $(31 \times 0.90) + 1 = 28.9$, so roughly the 29th element;
- the first quartile (25th percentile) has index $(31 \times 0.25) + 1 = 8.75$, so about the 9th element;
- the median index computes to $(31 \times 0.5) + 1 = 16.5$, so the median is typically taken as the average of the 16th and 17th ordered values.

Different software packages use slightly different interpolation rules for quantiles, but the idea is to map a fraction p into a sorted index and take or interpolate at that index.

10.4.3 What to report

A good practice is to report a small set of central tendency measures plus variability measures that capture shape and tails. When showing numbers in a table, include units and sample sizes. When reporting averages computed across runs, say how many repetitions you did and include error bars or confidence intervals where possible.

People read graphics faster than tables. Choose the chart type based on the variable type and on what you want to emphasize. If the independent variable is discrete (for example, protocol A, protocol B, protocol C), a column or bar chart is natural. If the independent variable is continuous (for example, message inter-arrival time), use a line chart to show trends.